Title Page

Lab Title: Lab 3 — Data Carving with XXD, Binwalk and Scalpe

Student Name: Athanasius Orikeze Alekwe

Student ID: 2025/FWSD/11230.

Course Name: Computer and Digital Forensics

Instructor Name: Aminu Idris

Date of Submission: 04/09/2026

Introduction

This laboratory exercise focused on practical data-carving and file-recovery techniques used when normal file-system information is missing, damaged, or no longer dependable. The work involved examining raw binary data, identifying file signatures, reconstructing files from hexadecimal data, recovering deleted content, and validating recovered evidence. The exercise also required the use of file headers and footers, metadata examination, direct sector-level analysis, embedded-file discovery, and signature-based carving.

The practical work was carried out in a controlled forensic workspace so that the original evidence could be preserved. Evidence files were downloaded into a dedicated laboratory structure, their sizes and cryptographic hashes were recorded, and the original copies were made read-only before analysis. Working copies were then used for the investigative procedures shown throughout the report.

The investigation progressed from JPEG examination with xxd and strings to file-system analysis with The Sleuth Kit, embedded-content discovery with Binwalk, and deleted-file recovery with Scalpel. MD5 and SHA-256 hashing was repeatedly used to check integrity and confirm that reconstructed or recovered files remained consistent with the data from which they were obtained.

Objectives

By completing this lab, students will demonstrate their ability to explain why data carving is required when file-system metadata is missing or unreliable; use xxd to examine binary file content and identify headers and footers; recognise JPEG SOI and EOI signatures; reconstruct binary content from a hexadecimal dump; use The Sleuth Kit to examine inode, block and sector-level data; identify and extract embedded content using Binwalk; recover deleted files from a forensic USB image using Scalpel; verify recovered evidence using cryptographic hashes; and prepare a professional forensic report containing commands, screenshots, findings and limitations.

Laboratory / Tools Used

Tool / Environment	How it was used in the laboratory
Kali Linux in VMware Workstation	Provided the working environment in which the forensic commands and analysis were performed.
wget	Downloaded the authorised evidence files used in the practical.
md5sum / sha256sum	Generated cryptographic hashes for original, reconstructed, extracted, carved, and packaged evidence.
file	Identified file types after downloading, reconstruction, extraction, and recovery.
xxd	Examined binary content, identified JPEG signatures, created a hexadecimal dump, and reconstructed the JPEG.
strings / grep	Located readable metadata-related strings, dates, software details, camera information, and other text inside the JPEG.
cmp	Compared the reconstructed JPEG with the working copy to confirm byte-for-byte equality.
The Sleuth Kit	img_stat, mmls, fsstat, fls, istat, icat, and blkcat were used to examine disk images, file systems, metadata entries, sectors, and deleted files.
7z	Inspected and extracted the 120M.7z training archive.
Binwalk	Detected embedded signatures and structures and performed automatic extraction attempts.
dd	Manually extracted the identified Office Open XML structure using offsets obtained from Binwalk.
unzip	Tested and listed the contents of the manually recovered OOXML/ZIP structure.
Scalpel	Carved EXIF- and JFIF-based JPEG files from the USB forensic image.
Kali Image Viewer	Opened recovered JPEG files to confirm that usable images had been successfully recovered.
zip / unzip -t	Created and validated the final supporting-evidence archive.

Key Findings

The hexadecimal examination of J_ub_law.jpg confirmed a standard JPEG start signature of FF D8, followed by an FF E1 APP1 marker containing EXIF information. The reconstructed copy produced the same MD5 and SHA-256 hashes as the source working copy, and cmp returned exit status 0, confirming that the reconstruction was byte-for-byte identical.

Metadata-style examination identified NIKON CORPORATION, NIKON D4, Adobe Photoshop 21.1 (Macintosh), the name-related string HOWARD KORN, the lens-related value 17.0-35.0 mm f/2.8, and date/time values including 2020:08:20 10:20:05 and 2013:10:08 18:56:21. Adobe/XMP information and an embedded camera-profile reference were also present.

The Sleuth Kit examination of Ch01InChap01.dd identified a FAT12 filesystem. A deleted Billing Letter.doc entry at metadata address 8 occupied sectors 237–283 and had a size of 24,064 bytes. Recovery with icat and direct extraction of the same 47 sectors with blkcat produced identical results, while sector 237 contained the Microsoft Compound File signature D0 CF 11 E0 A1 B1 1A E1.

The instructor-referenced File_carving.docx could not be located, including inside 120M.7z, so the permitted alternative Binwalk workflow was performed on the other supplied evidence. The USB evidence was confirmed as a raw 124,780,544-byte image with a FAT16 partition beginning at sector 128. Binwalk subsequently helped identify an Office Open XML structure at decimal offset 421888, which was manually extracted and successfully validated with unzip.

Finally, Scalpel was configured for both EXIF- and JFIF-based JPEG signatures and recovered 17 JPEG artefacts from the USB forensic image. Hashes were generated for all recovered JPEGs, and several—including 00000009.jpg, 00000010.jpg, and 00000011.jpg—were successfully opened in the Kali image viewer.

Practical Evidence

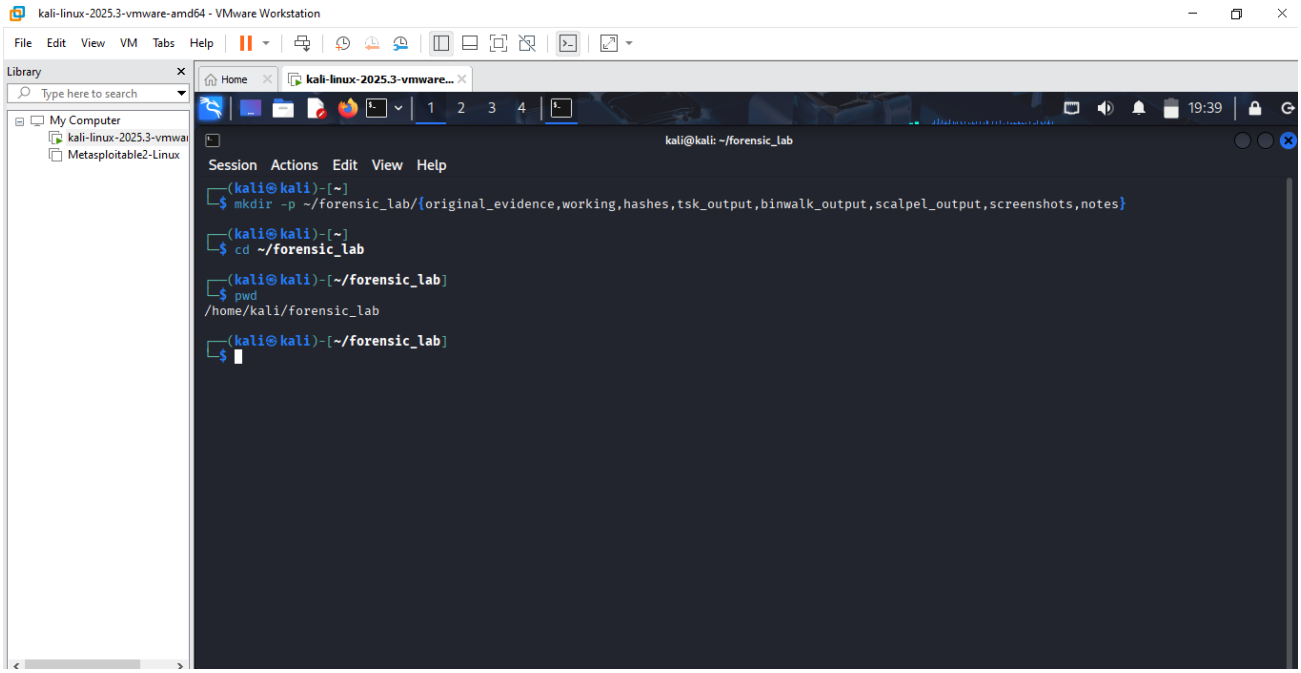

Fig 1.0 Creating the Lab folder.

Explanation

Here I created the lab folder and all its contents that are needed for this lab.

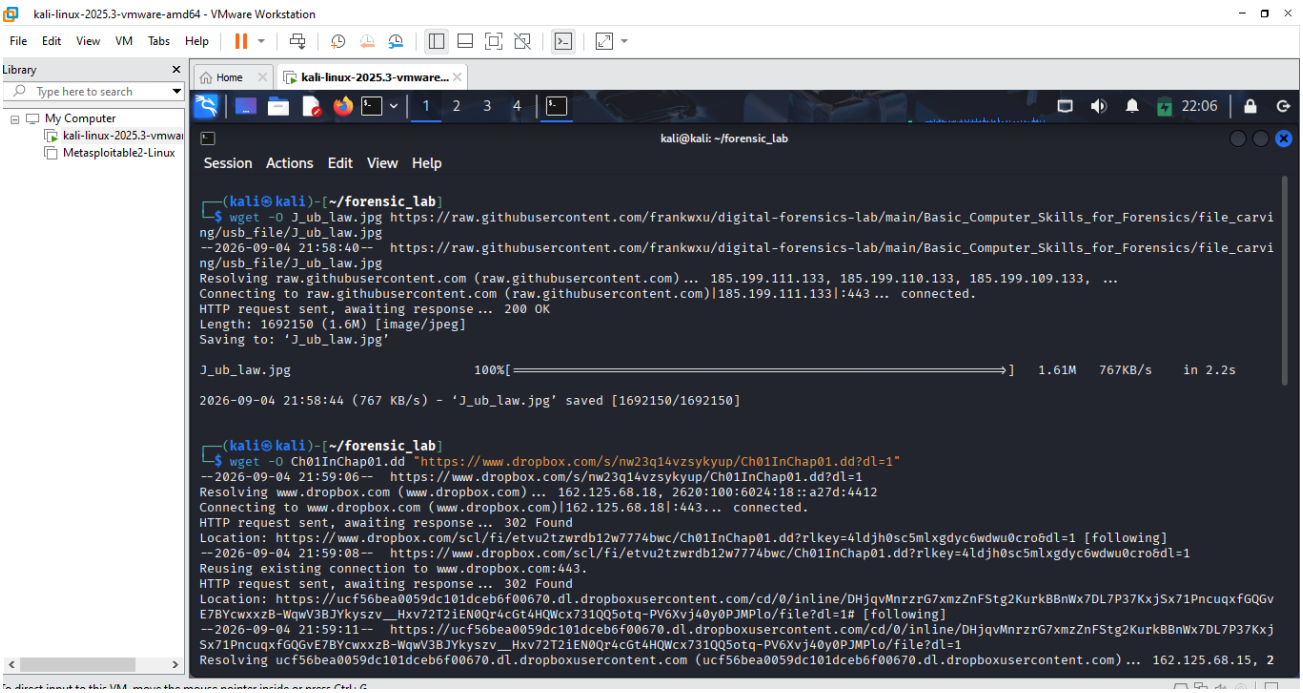

Fig 1.1 Download the authorized training evidence files.

Explanation

This screenshot records the beginning of the evidence-acquisition stage, where the authorised laboratory files were downloaded into the working environment. It provides a visible record that the practical used the evidence specified for the exercise.

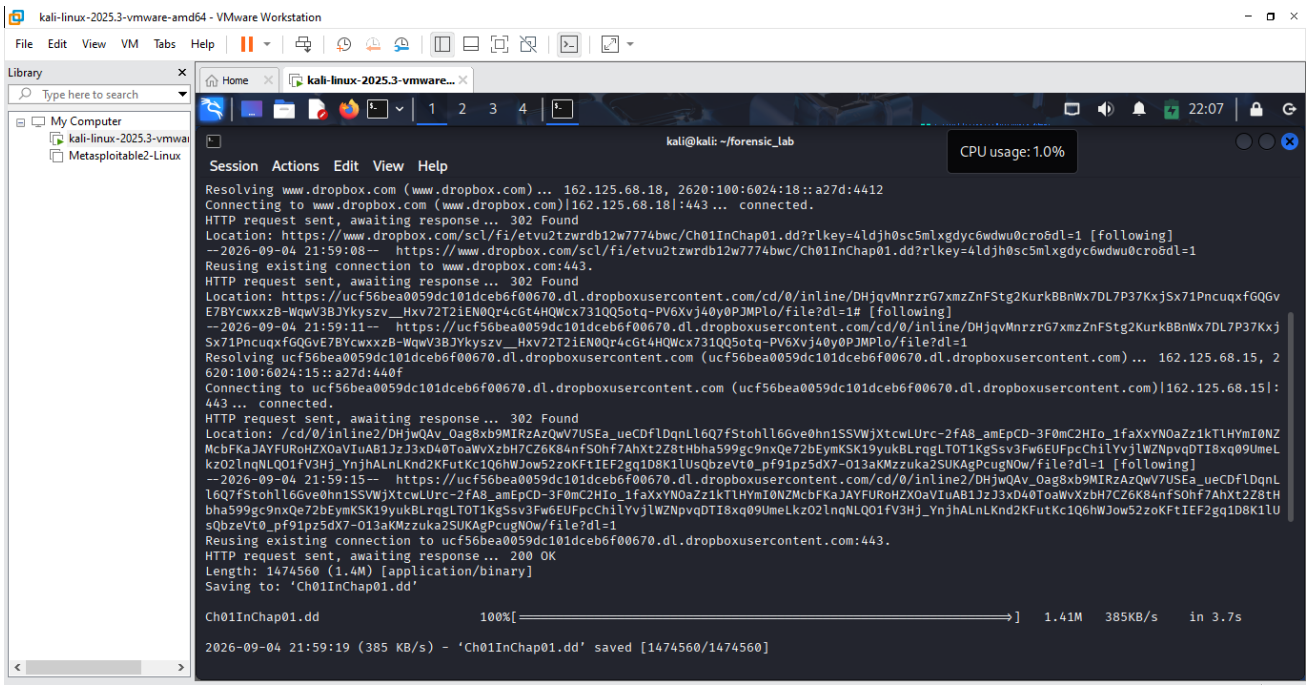

Fig 1.2 Download the authorized training evidence file page 1.

Explanation

This continuation shows the download process for Ch01InChap01.dd. The progress information demonstrates that the disk image was obtained before any forensic examination was started.

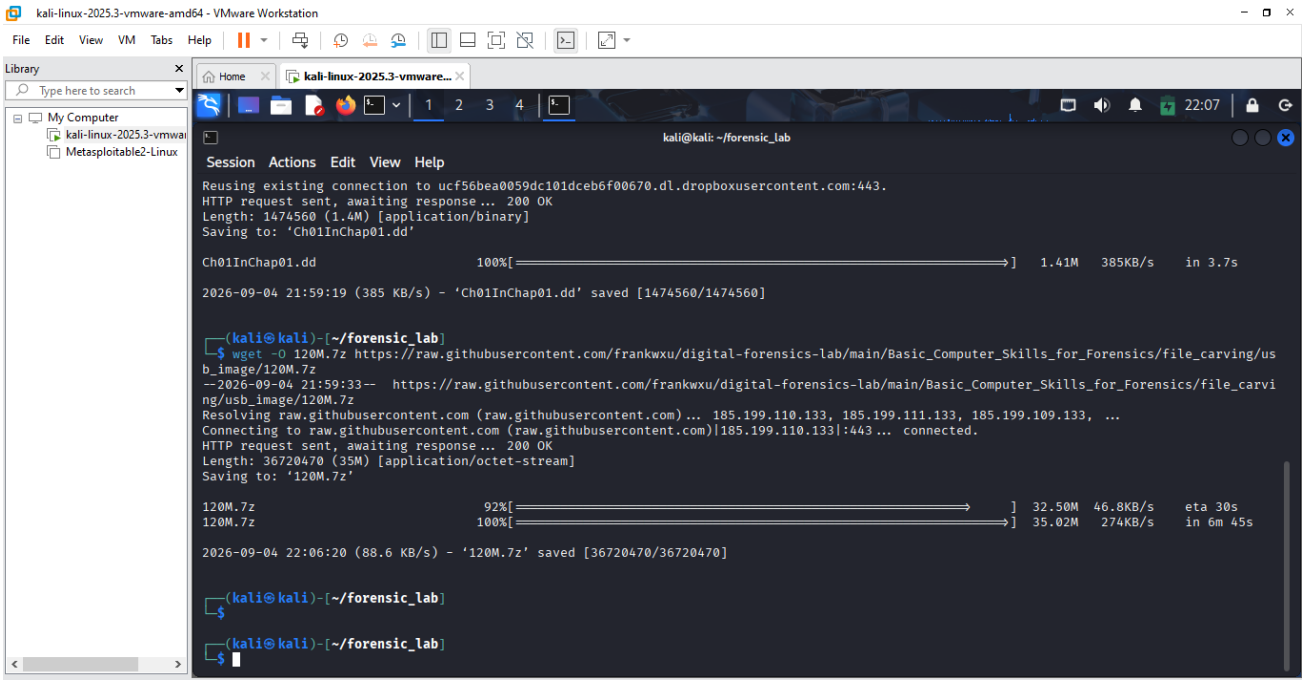

Fig 1.3 Download the authorized training evidence file page 2.

Explanation

This screenshot continues the evidence download process and shows the acquisition of the 120M.7z archive. It completes the documented acquisition of the main evidence resources used later in the laboratory.

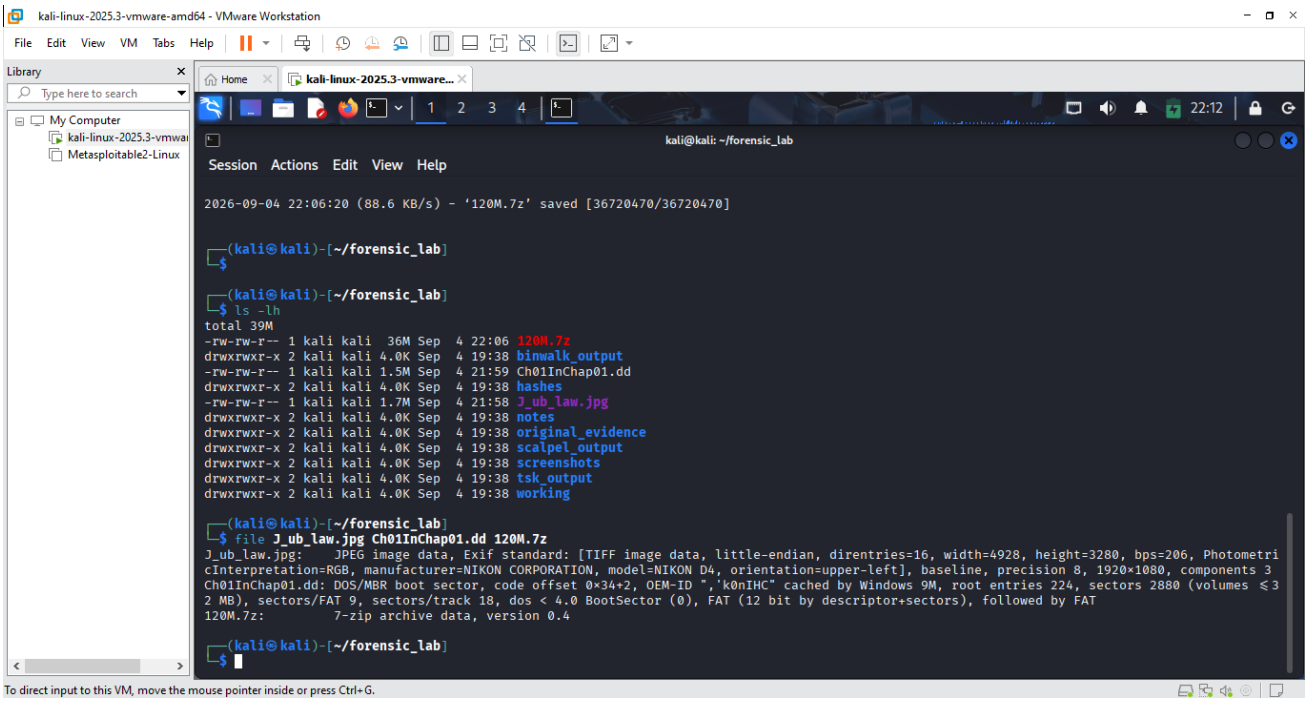

Fig 1.4 Checking that all three downloaded correctly.

Explanation

The downloaded files were listed and examined with the file command to confirm that they were recognized in their expected formats. This check helped ensure that the downloads were genuine evidence files rather than incomplete or incorrect downloads.

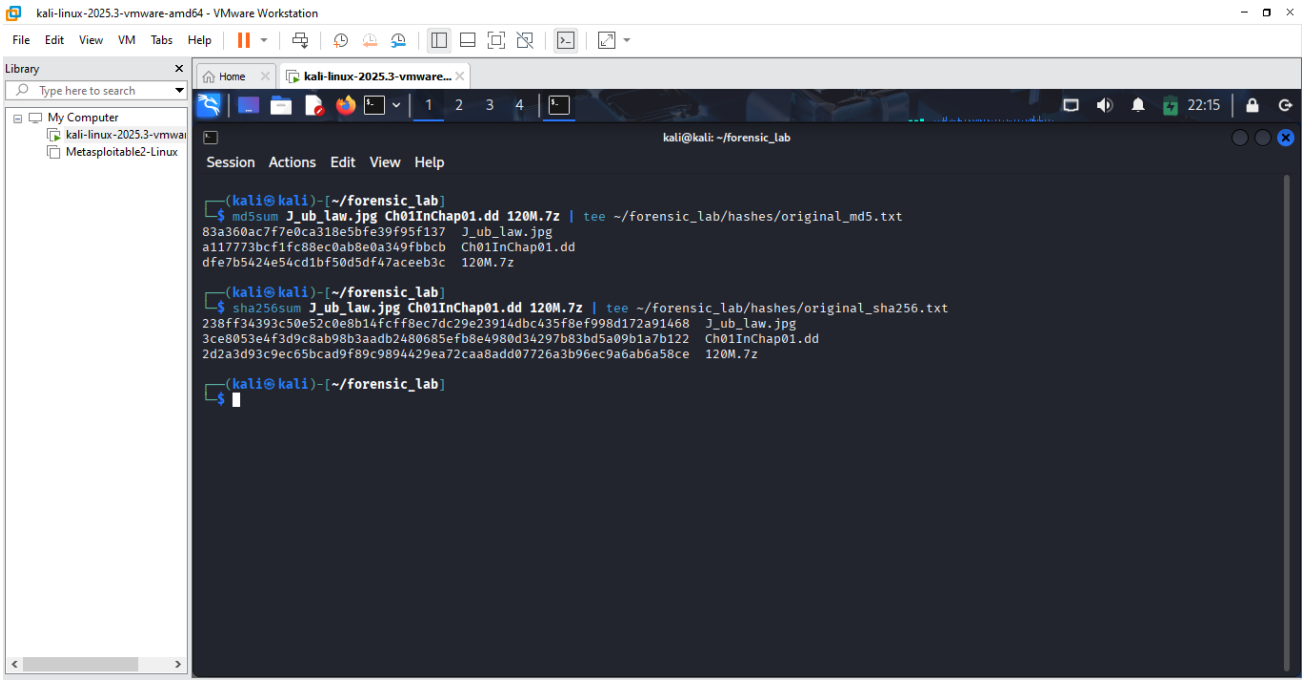

Fig 1.5 calculating the original hashes for md5sum and sha256sum.

Explanation

This screenshot records the first cryptographic integrity checks performed on the evidence. MD5 and SHA-256 values were calculated so that later copies or extracted data could be compared against known values.

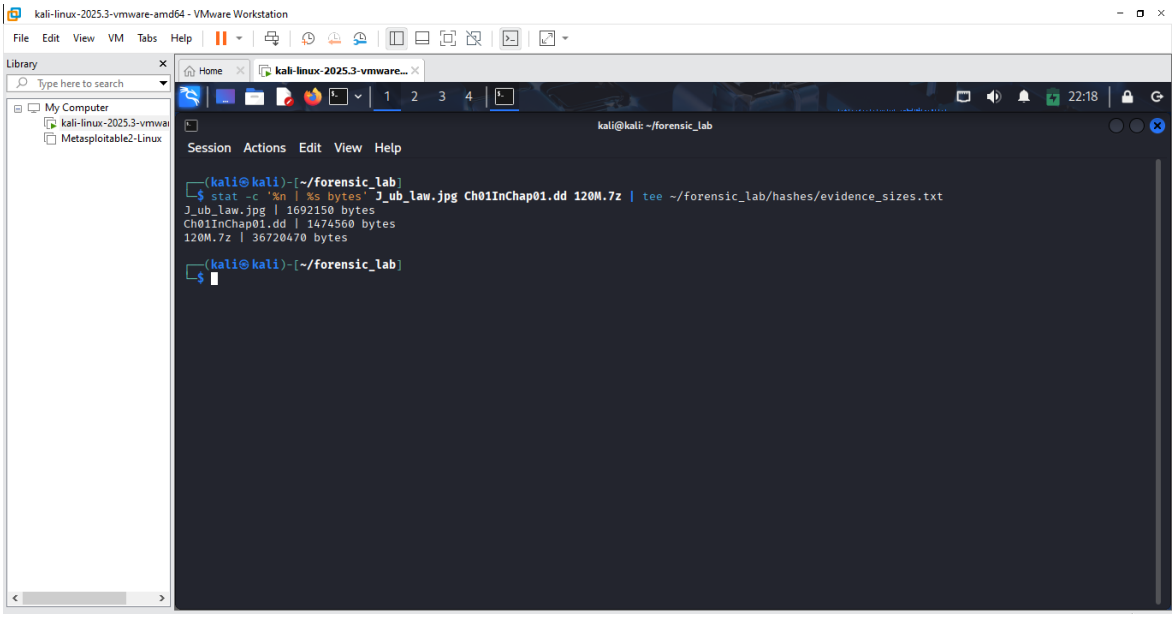

Fig 1.5.1 Recording the exact size for the files.

Explanation

The exact byte size of each evidence file was recorded at this stage. This added another basic integrity and identification reference alongside the cryptographic hashes.

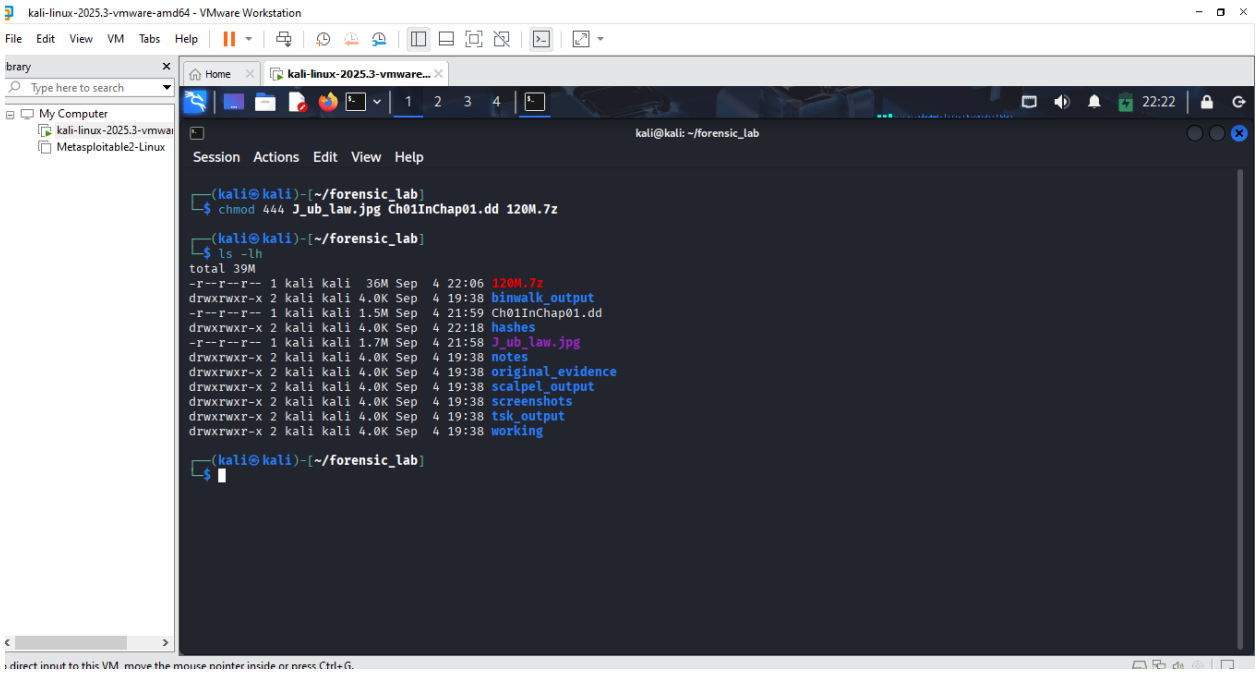

Fig 1.6 Making the originals read-only.

Explanation

The evidence files were given read-only permissions before analysis. This reduced the risk of accidental modification and helped preserve the originals while subsequent work was carried out on separate copies.

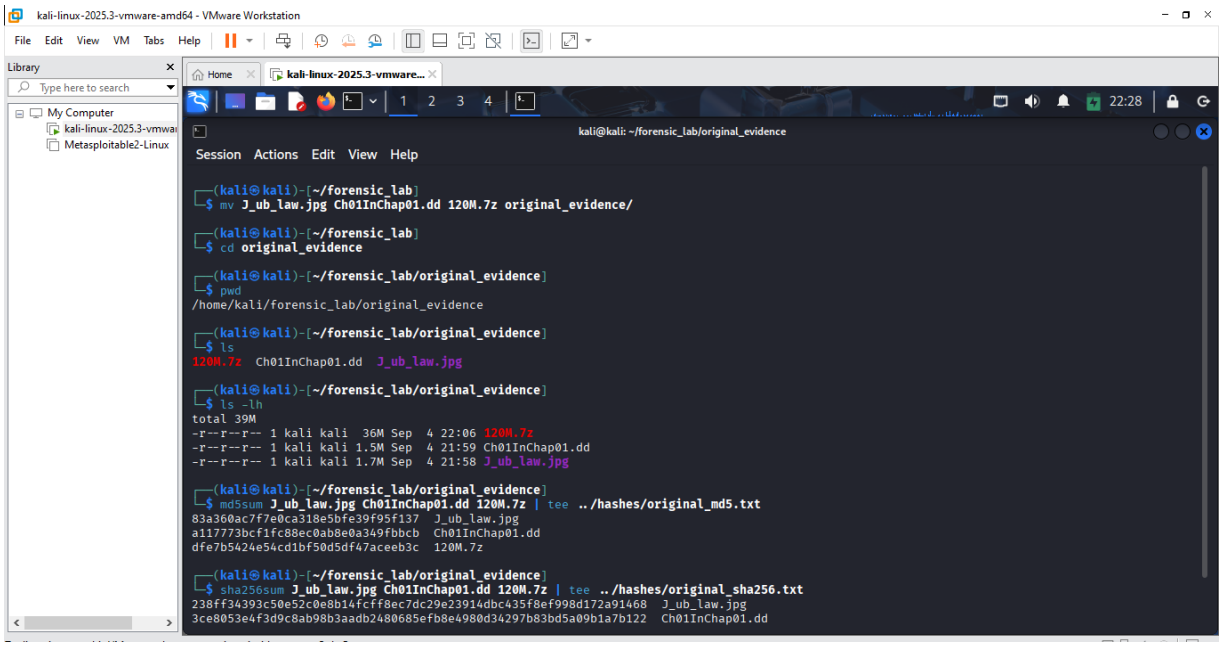

Fig 1.7 Moving the evidence into original_evidence.

Explanation

This screenshot shows the authorised files being placed in the dedicated original_evidence directory. Separating original evidence from working material helped maintain a clear distinction between source data and files created during analysis.

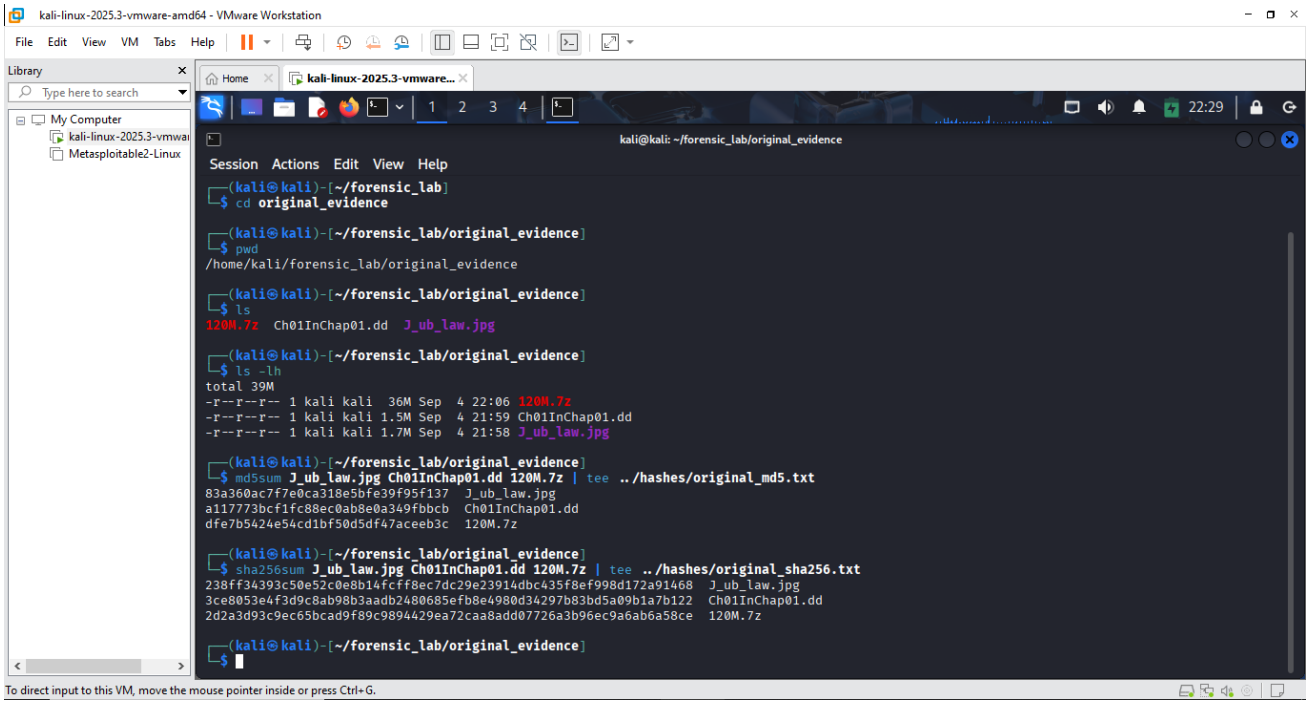

Fig 1.8 Recording the MD5 and SHA-256 hashes.

Explanation

After organizing the files inside the original-evidence directory, their cryptographic hashes were recorded again and saved into dedicated hash files. This created a permanent integrity record for the evidence used throughout the lab.

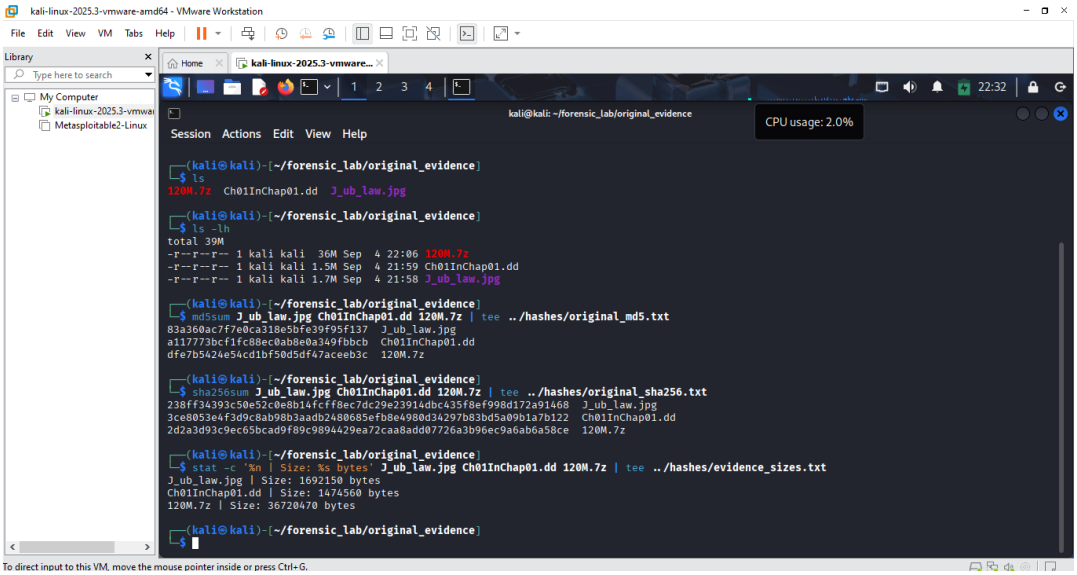

Fig 1.9 Recording the exact file sizes**.**

Explanation

This screenshot records the exact sizes of the evidence files after they had been organised in the evidence directory. The values provided an additional reference for confirming that the files had not unexpectedly changed.

Part A - File Signatures and Hex Analysis

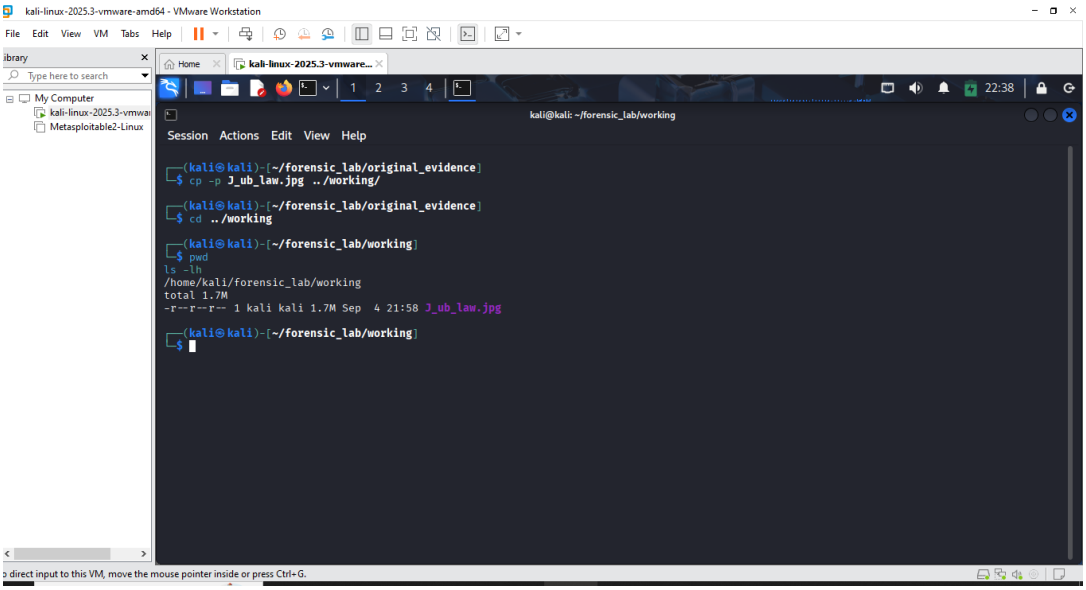

Fig 1.10 Making a working copy of the JPEG.

.

Explanation

A separate copy of J_ub_law.jpg was created in the working directory before hexadecimal analysis began.

This ensured that the original evidence remained untouched while the JPEG was examined and reconstructed.

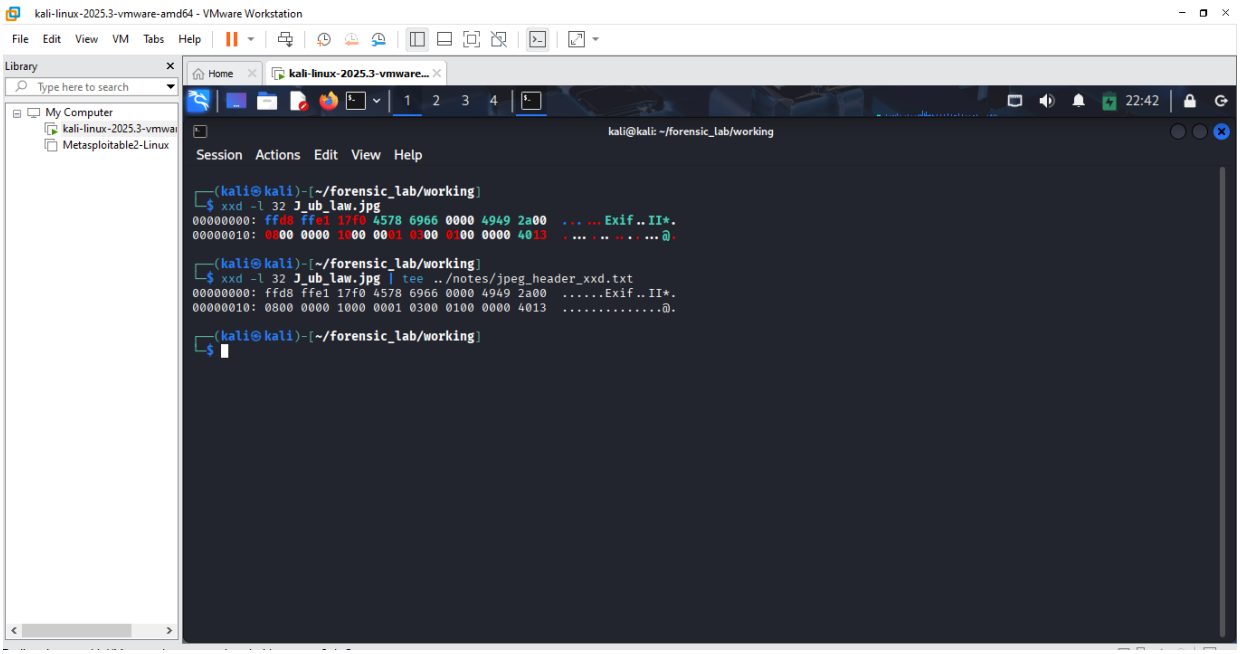

Fig 1.11 Examine the JPEG header with xxd.

Explanation

The beginning of the JPEG was displayed in hexadecimal form using xxd. The output revealed the standard FF D8 JPEG Start of Image marker followed by the FF E1 APP1 marker and readable Exif data.

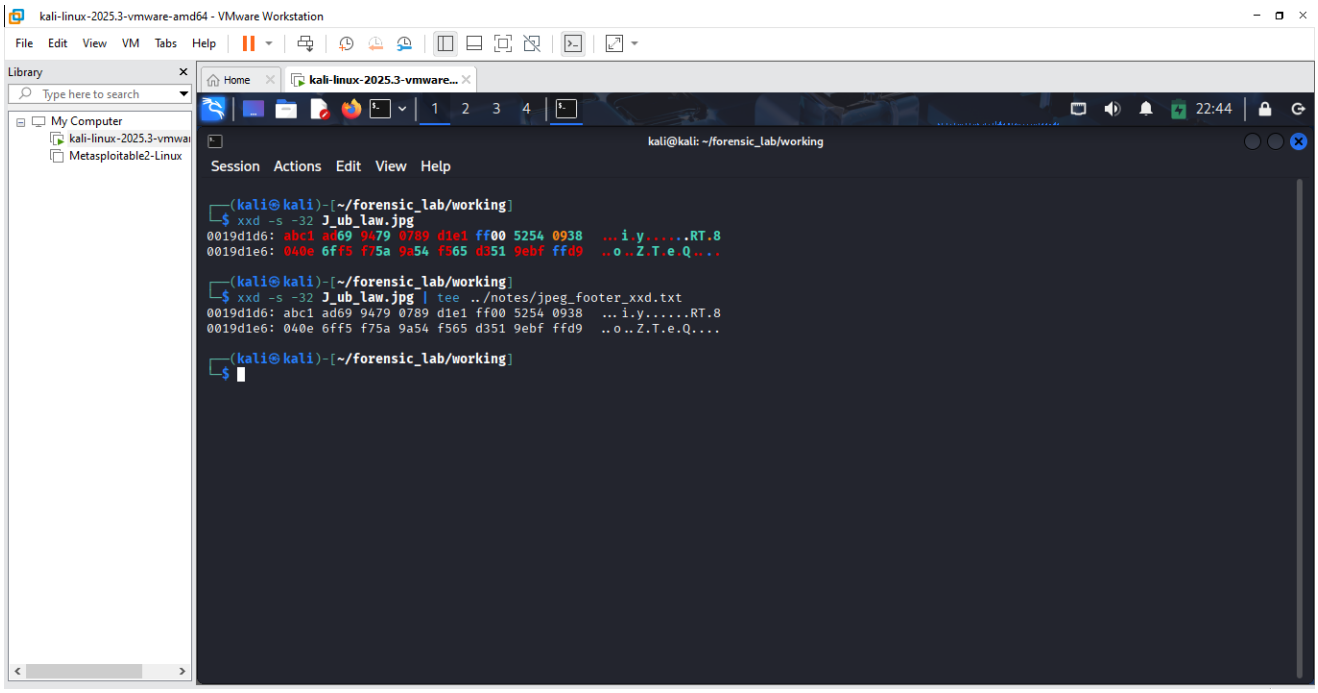

Fig 1.12 Examine the JPEG footer.

Explanation

The JPEG begins with the standard FF D8 SOI signature. Immediately following it is an FF E1 APP1 marker containing an EXIF structure.

The final bytes of the image were examined to locate the JPEG end marker. The FF D9 value at the end provided evidence that the expected JPEG footer was present.

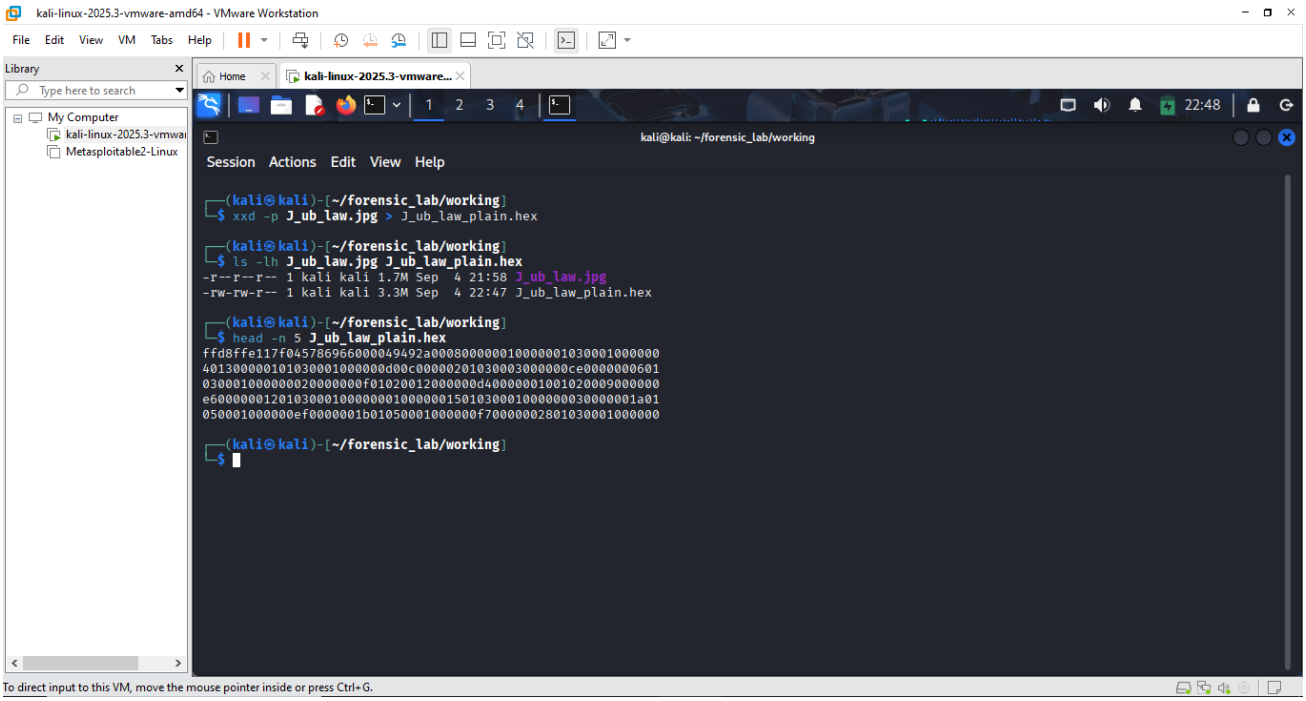

Fig 1.13 Creating a plain hexadecimal dump.

Explanation

The JPEG was converted into a plain-text hexadecimal representation. This demonstrated how binary image content can be represented as hexadecimal data while preserving the underlying byte values needed for reconstruction.

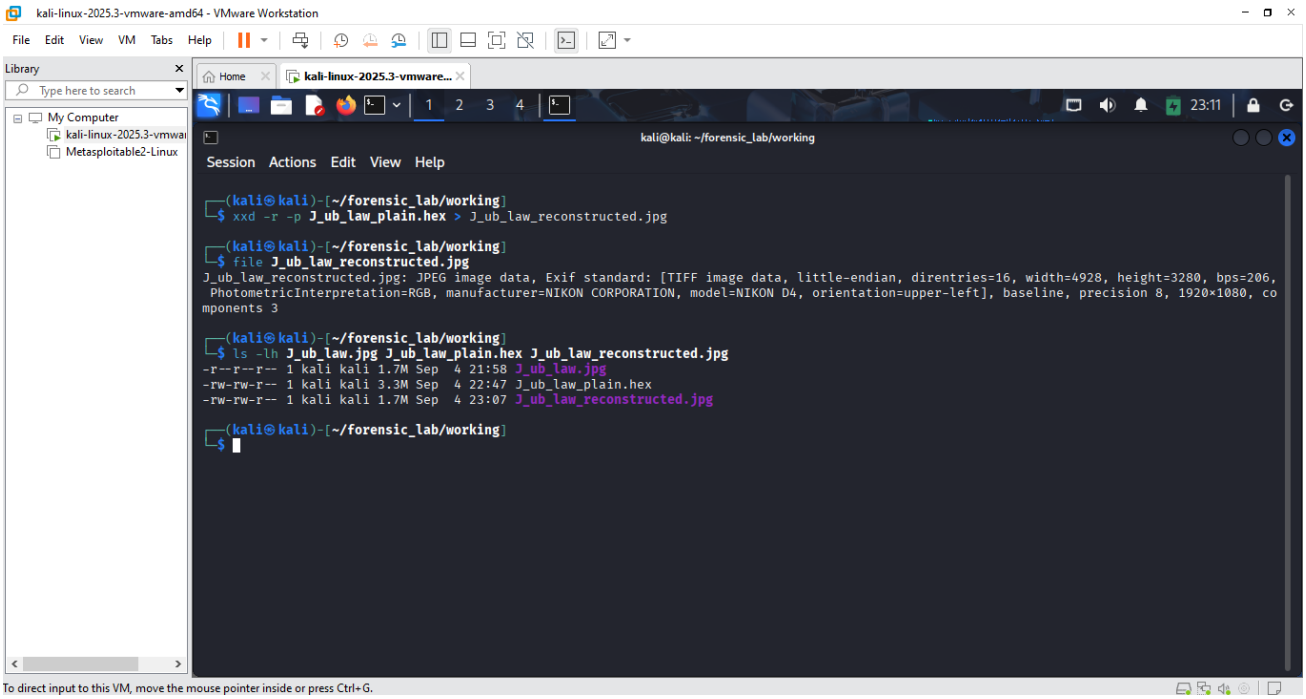

Fig 1.14 Reconstructing the JPEG from the hexadecimal dump.

Explanation

The plain hexadecimal dump was reversed using xxd to produce a reconstructed JPEG file. The resulting file was identified as a JPEG and could then be compared against the original working copy.

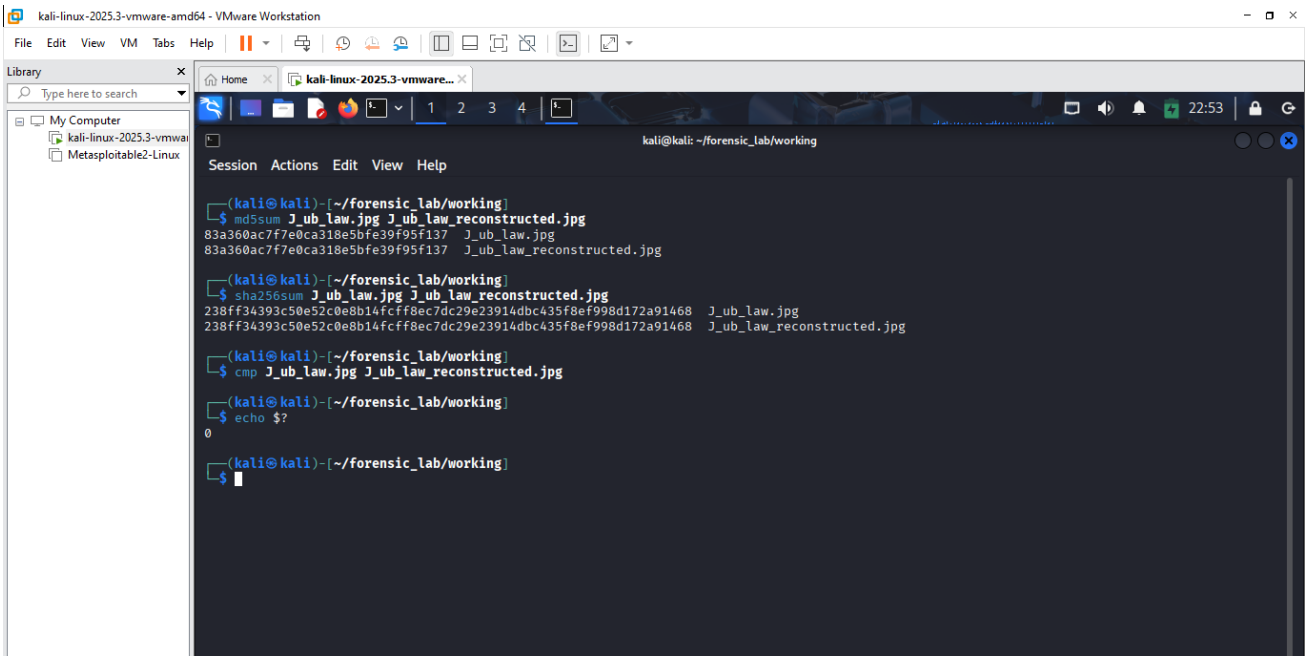

Fig 1.15 Cryptographically verify the reconstruction.

Explanation

The JPEG reconstructed from the hexadecimal dump produced identical MD5 and SHA-256 hashes to the source JPEG. A binary comparison using cmp also returned exit status 0, confirming that the reconstructed

file was identical to the source working copy.
MD5 and SHA-256 values for the working JPEG and reconstructed JPEG were identical. A cmp comparison also returned exit status 0, demonstrating that the reconstructed file matched the source working copy byte for byte.

Part B — examining metadata-related strings inside

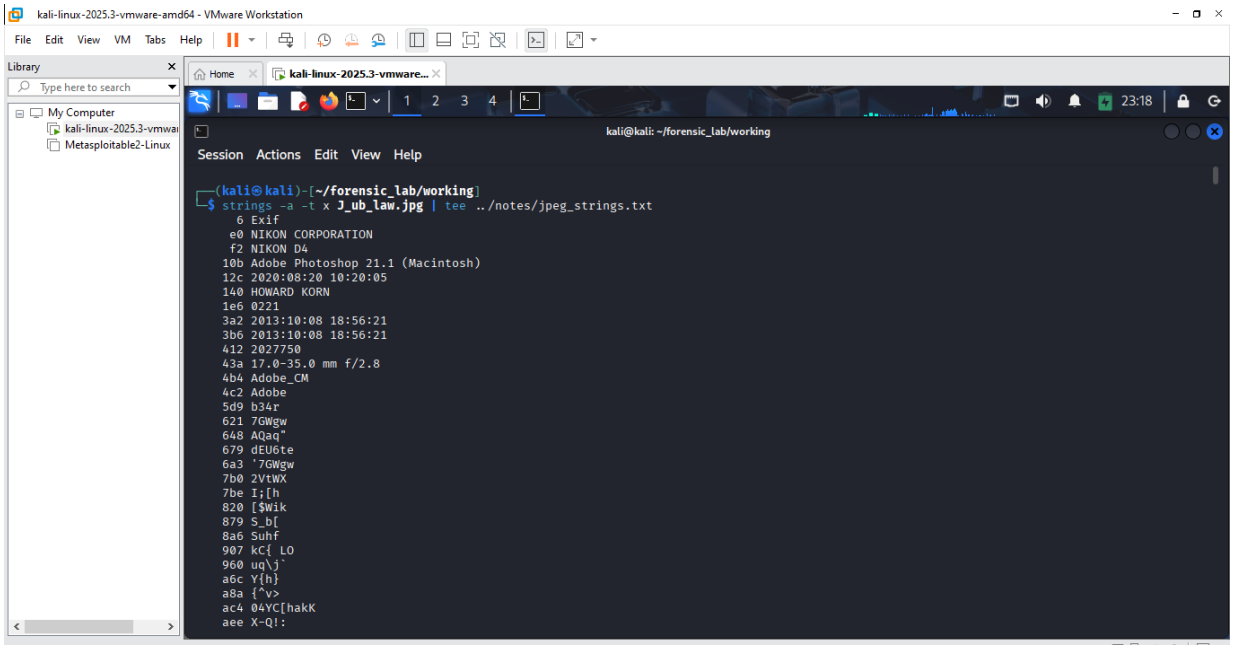

Fig 1.16 The metadata-like content examination.

Explanation

The strings command was used to extract readable text from the JPEG together with hexadecimal offsets. The output immediately revealed EXIF-related data and several useful strings, including camera, software, name, date, and lens-related information.

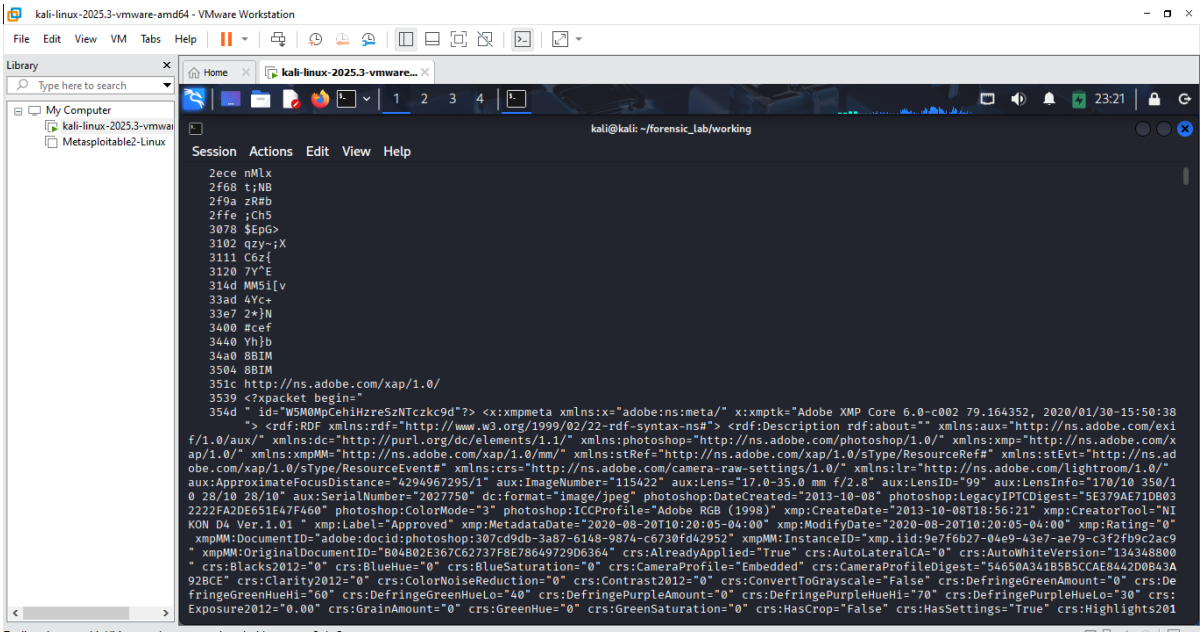

Fig 1.17 The metadata-like content examination page 1.

Explanation

This screenshot continues the readable-string examination and shows a larger section of embedded XML/XMP-style information. It demonstrates that considerably more information was stored within the JPEG than was visible from the image alone.

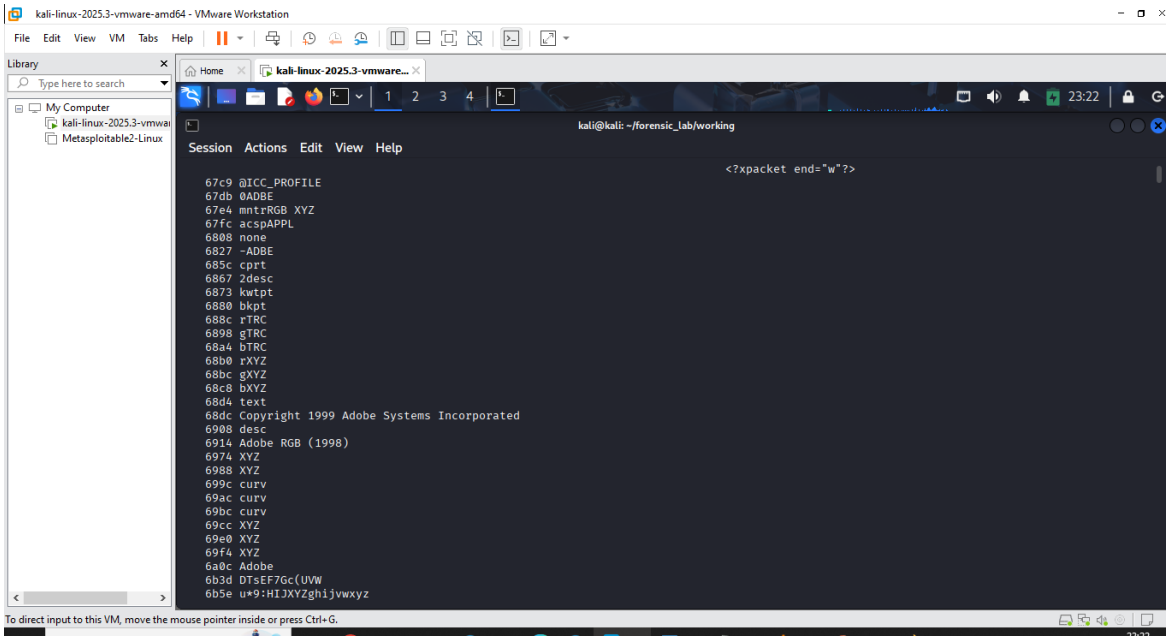

Fig 1.18 The metadata-like content examination page 2.

Explanation

Further readable strings were displayed from the JPEG, including Adobe-related information. The output also contained a readable Adobe copyright string, which became part of the documented metadata findings.

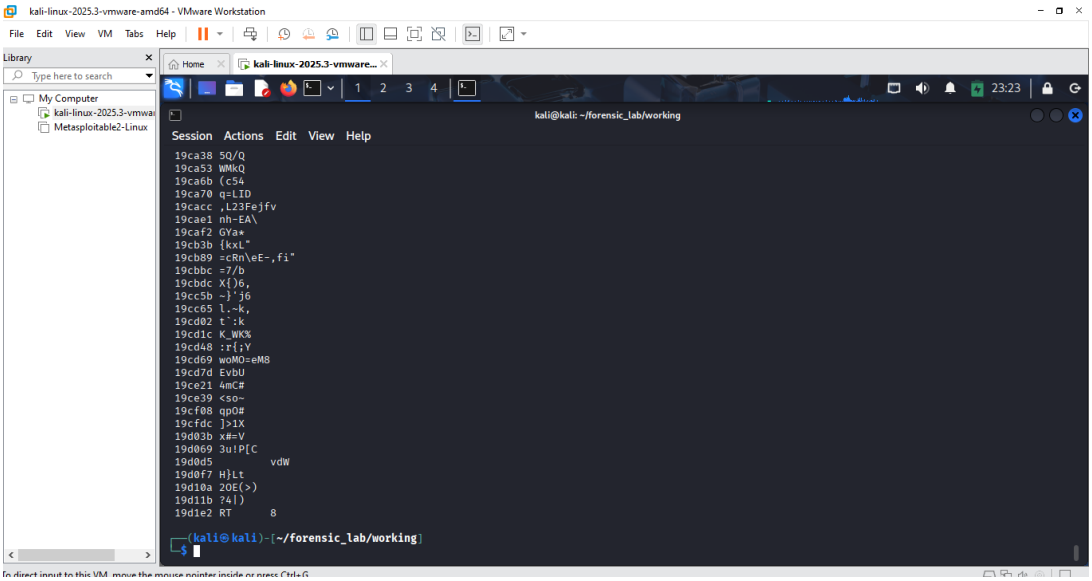

Fig 1.19 The metadata-like content examination page 3.

Explanation

This screenshot records the continuation of the string output through later offsets in the JPEG. Much of this

content was not immediately meaningful on its own, but retaining it provided a complete record of the strings examination.

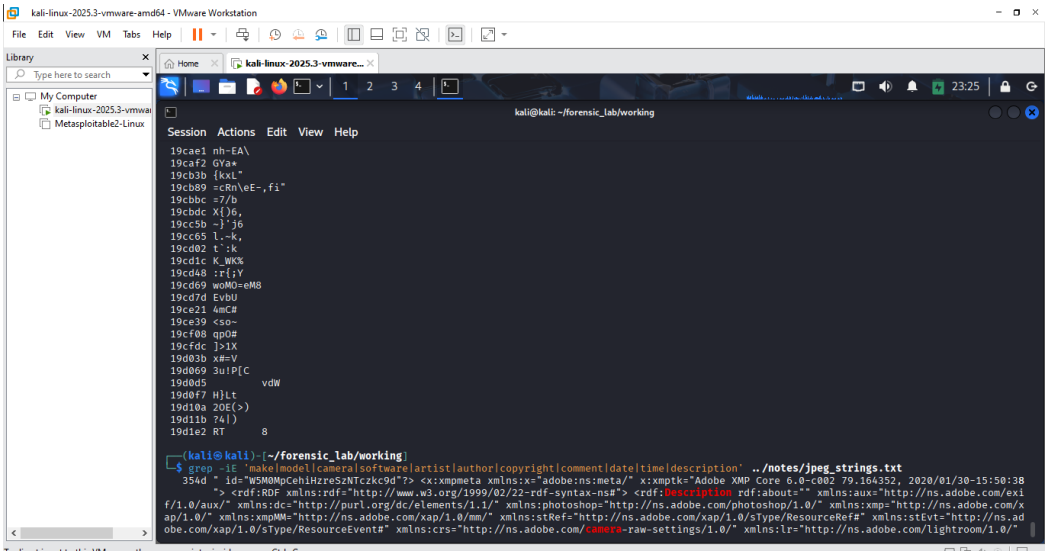

Fig 1.20 The metadata-like content examination page 4.

Explanation

A targeted search was used to reduce the larger strings output to terms connected with metadata, including software, camera information, dates, and related fields. This made the relevant information easier to isolate from unrelated readable data.

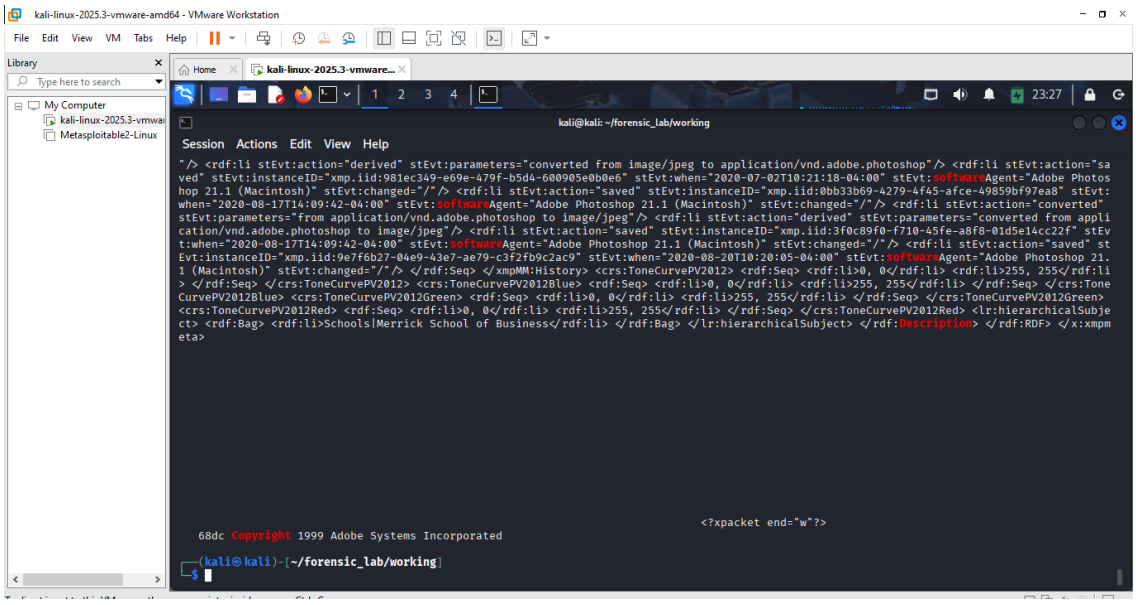

Fig 1.21 The metadata-like content examination page 5.

Explanation

This screenshot shows additional XMP information associated with Adobe software and processing history. It also displays the copyright-related text recovered from within the file.

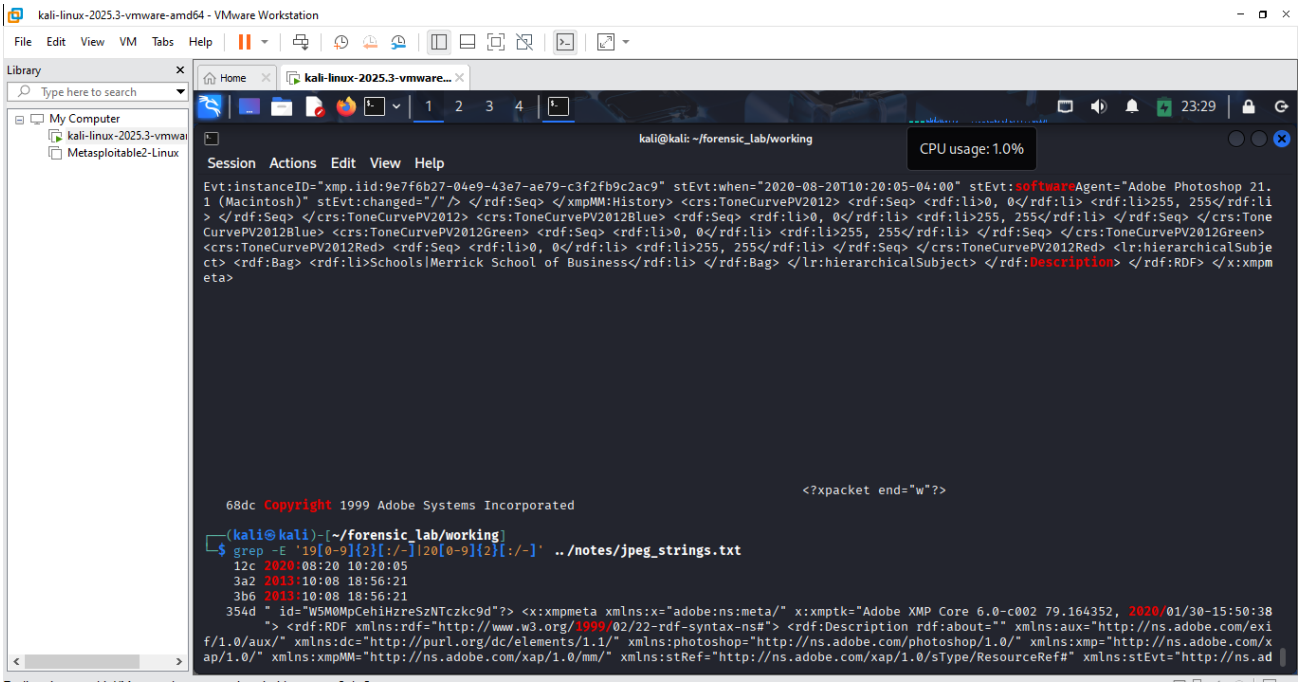

Fig 1.22 The metadata-like content examination page 6.

Explanation

The search was narrowed further to identify date-like values and metadata terms. This helped confirm the dates actually present in the JPEG rather than assuming values that were not supported by the data.

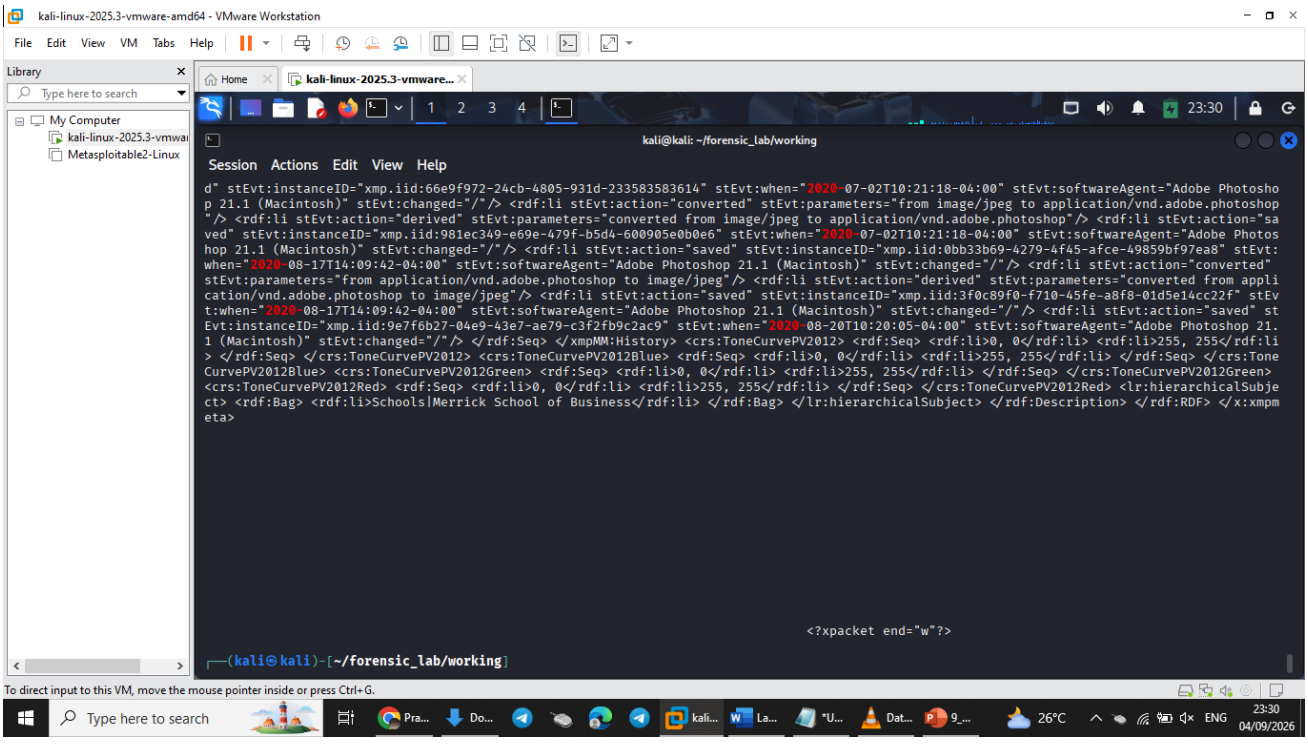

Fig 1.23 The metadata-like content examination page 7.

Explanation

This screenshot continues the detailed XMP output and shows embedded processing information connected with the image. It provides supporting evidence that Adobe-related metadata was present inside the file.

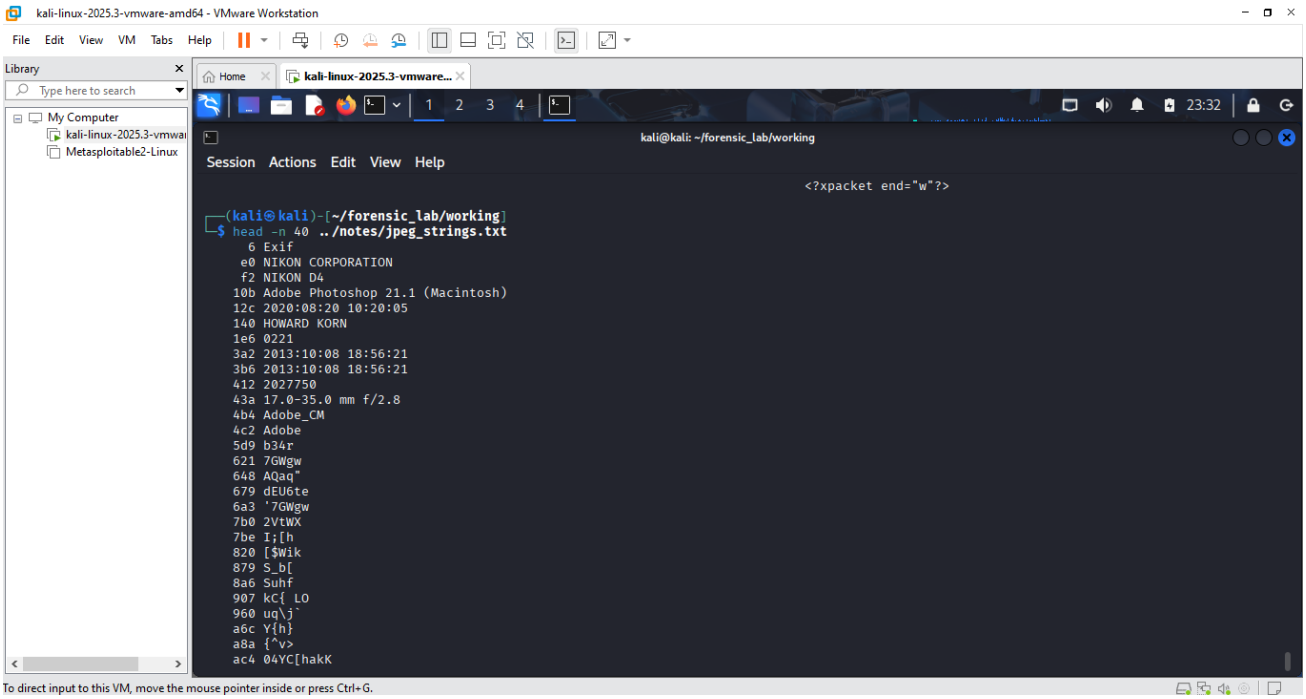

Fig 1.24 The metadata-like content examination page 8.

Explanation

The beginning of the saved strings output clearly displays several important findings, including NIKON CORPORATION, NIKON D4, Adobe Photoshop 21.1 (Macintosh), HOWARD KORN, date/time values, and the lens-related string.

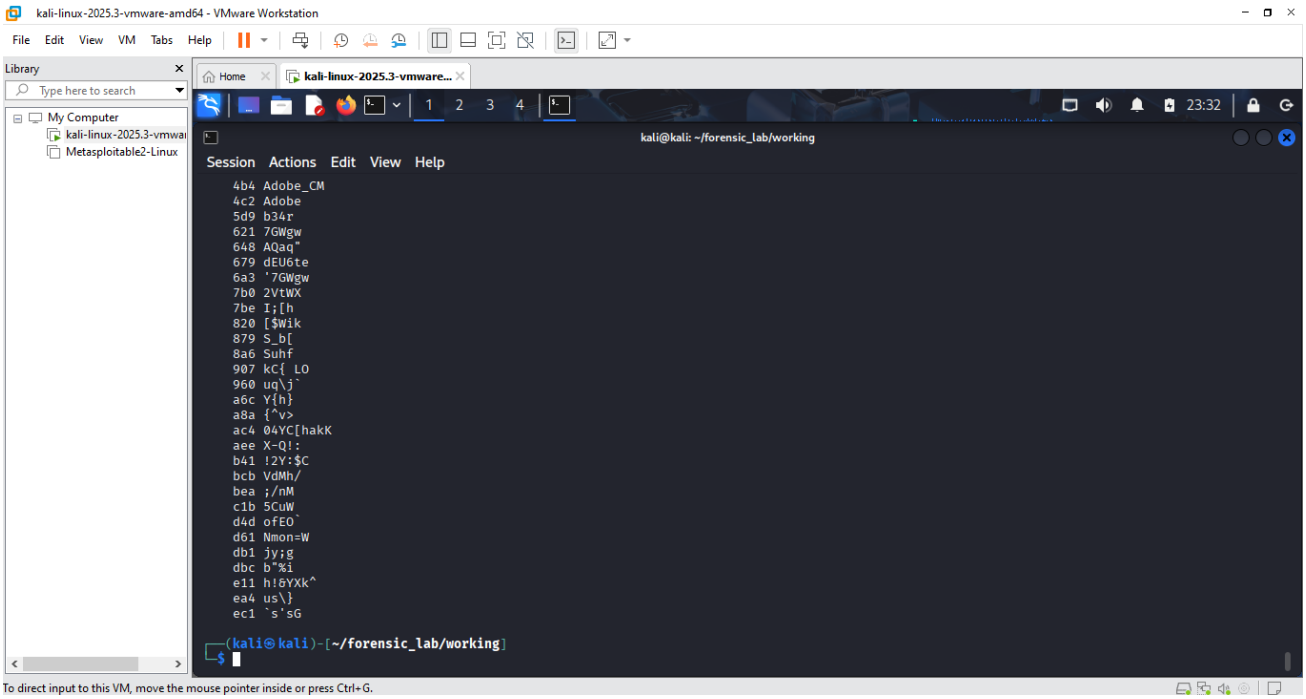

Fig 1.25 The metadata-like content examination page 9.

Explanation

This screenshot shows the continuation of the extracted strings beyond the main readable metadata fields. It documents that the examination was not limited to a single matched value but covered the wider file contents.

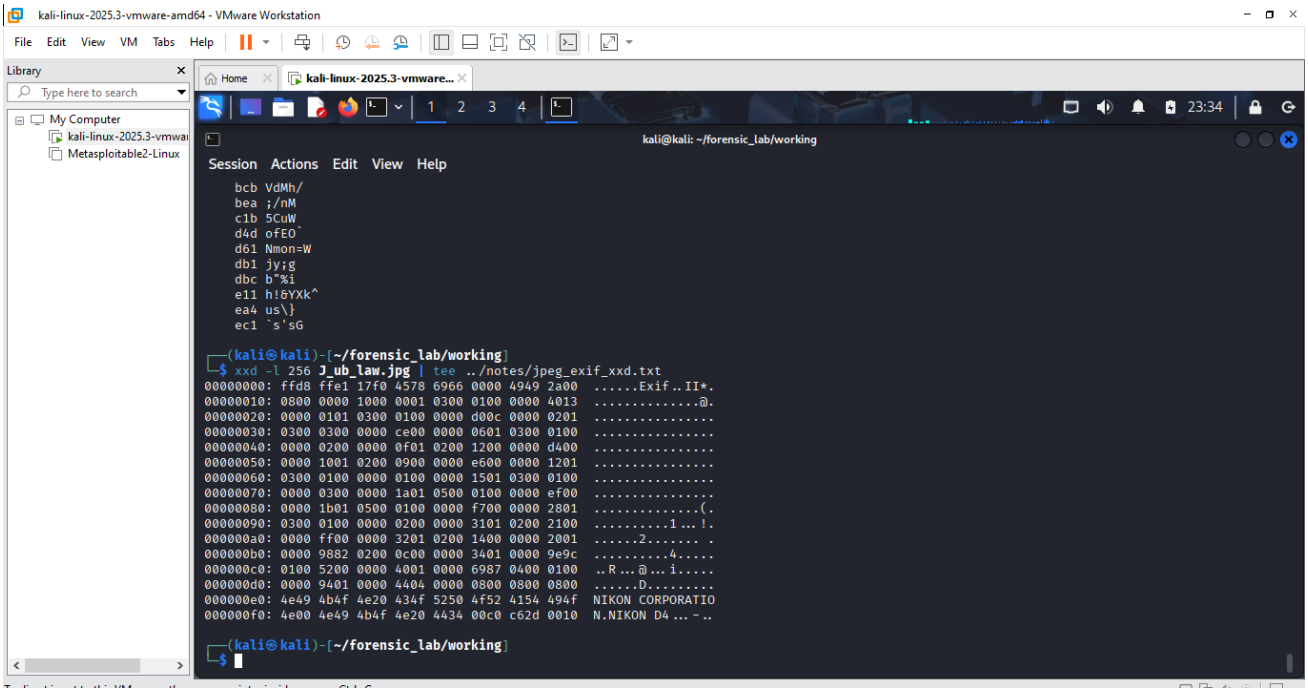

Fig 1.26 The metadata-like content examination page 10.

Explanation

The first 256 bytes of the JPEG were inspected with xxd, where readable camera information could be seen within the EXIF area. NIKON CORPORATION and NIKON D4 were visible in the hexadecimal output.

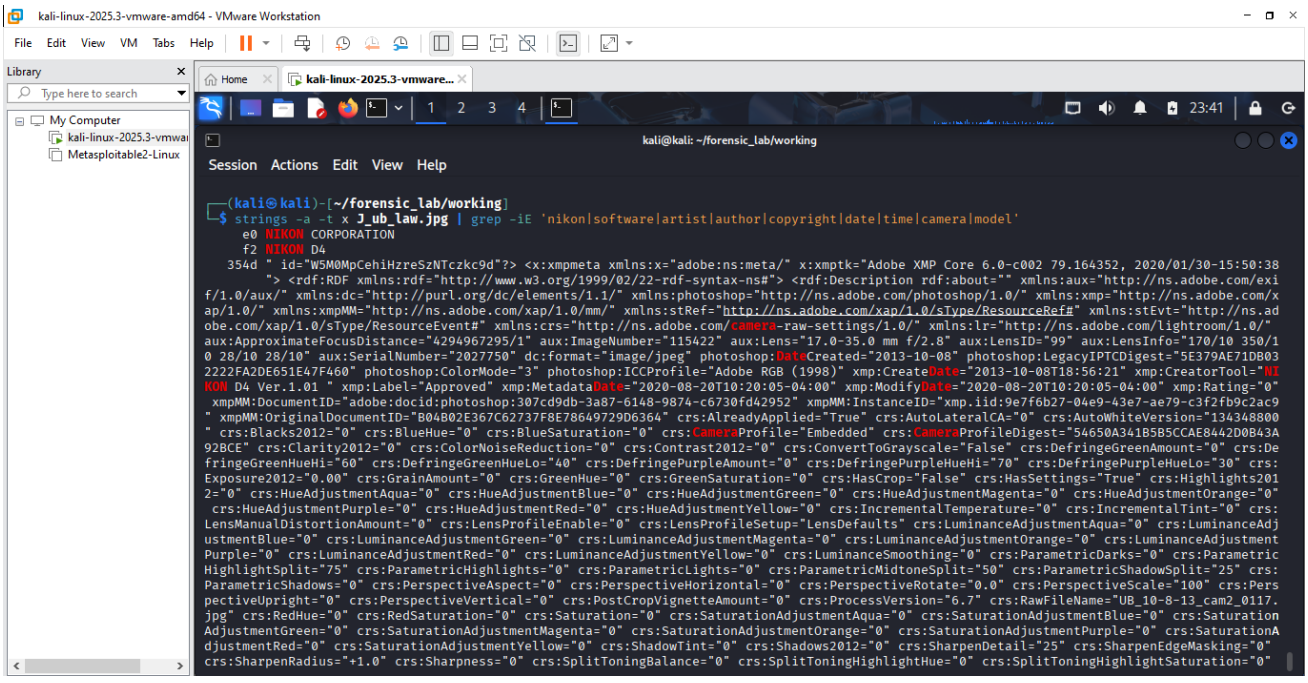

Fig 1.27 The metadata-like content examination page 11.

Explanation

A targeted metadata search brought together Nikon, software, date, camera, and other matching strings from the JPEG. The screenshot provides a focused view of the information that was considered relevant to the investigation.

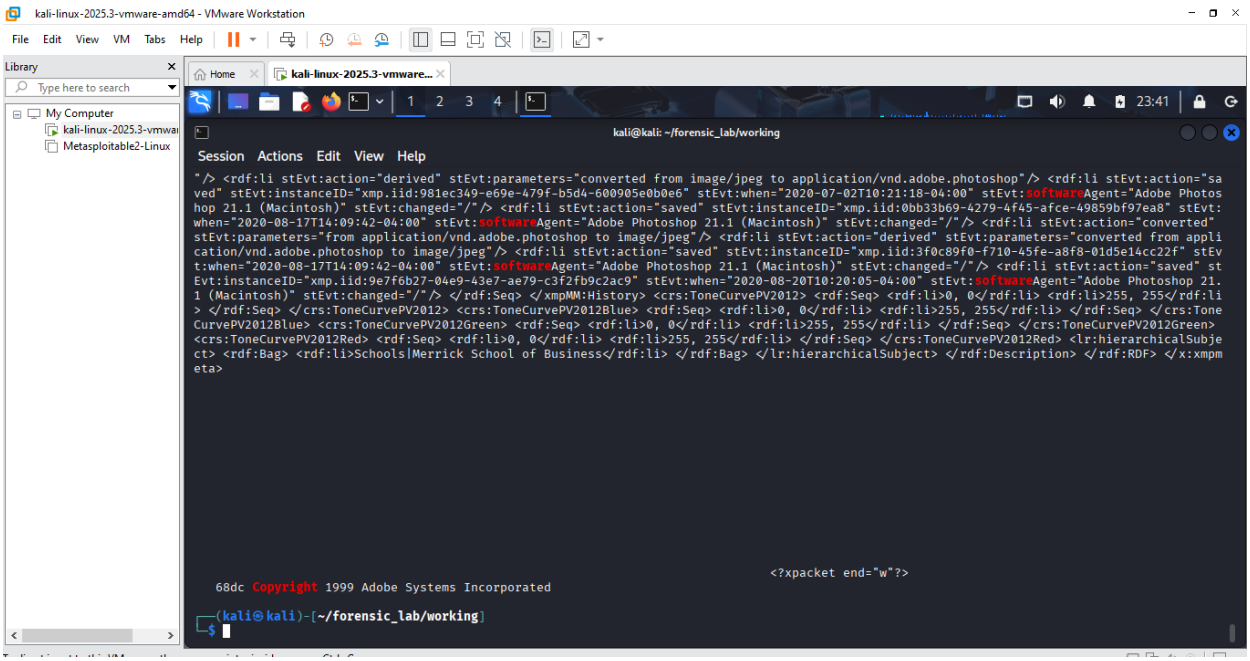

Fig 1.28 The metadata-like content examination page 12.

Explanation

This continuation displays more of the XMP and Adobe-related metadata identified by the targeted search. The presence of processing and software information supported the conclusion that the image contained embedded editing-related metadata.

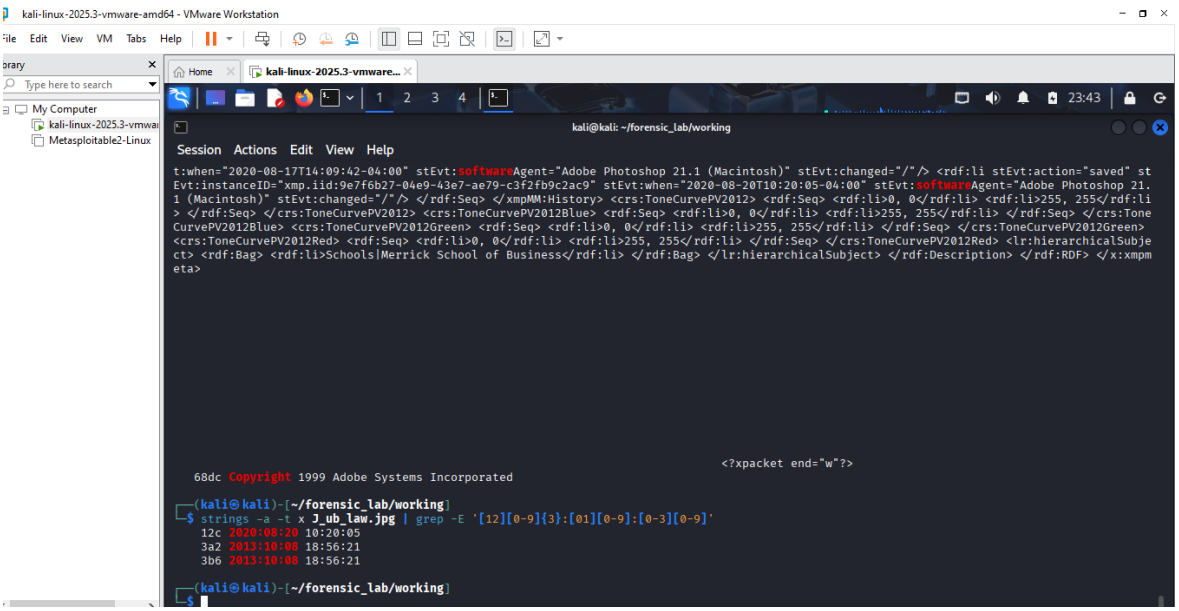

Fig 1.29 The metadata-like content examination page 13.

The final metadata screenshot records the date search and additional metadata output. Together, the metadata examination identified Nikon camera information, Adobe Photoshop information, the name-related string, lens information, and multiple date/time values.

Explanation

Examination of J_ub_law.jpg using xxd and strings identified an EXIF structure and several readable metadata-related strings. The camera manufacturer was identified as NIKON CORPORATION and the model as NIKON D4. The file contained references to Adobe Photoshop 21.1 (Macintosh), a name-related string HOWARD KORN, the lens-related string 17.0-35.0 mm f/2.8, and date/time values including 2020:08:20 10:20:05 and 2013:10:08 18:56:21. Adobe/XMP metadata and an embedded camera-profile reference were also observed. Only information actually present in the examined data was recorded.

Part C - Direct Data-Unit Examination with The Sleuth Kit

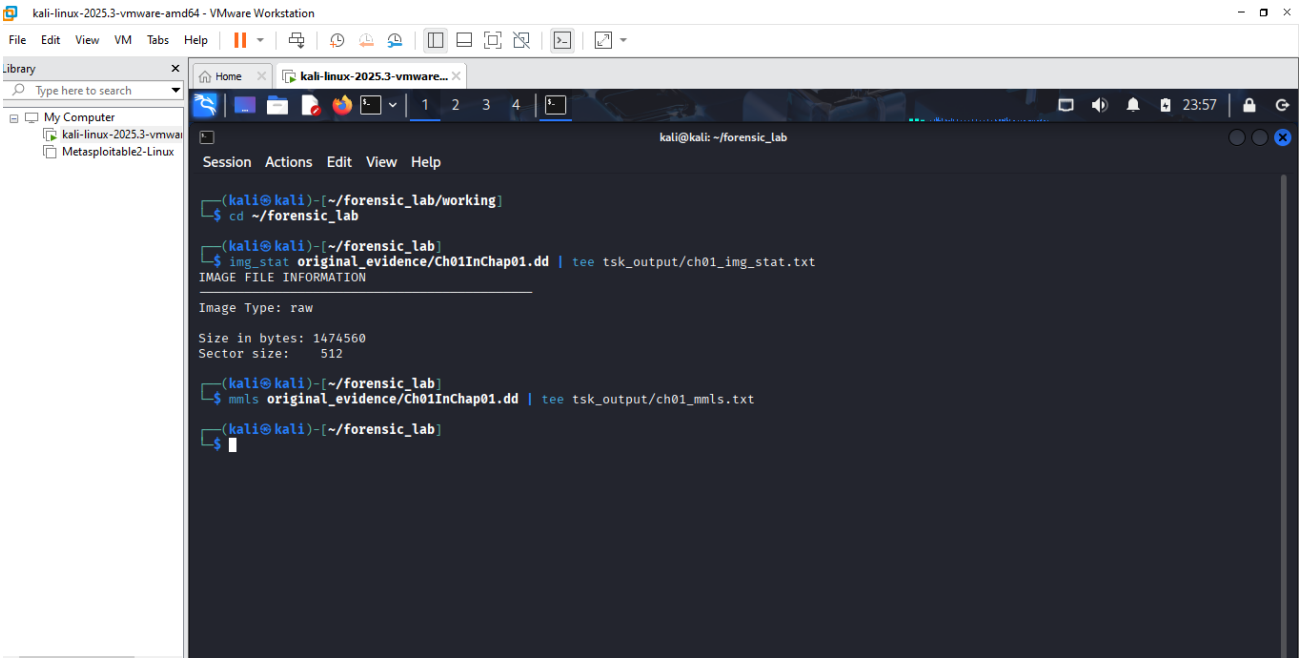

Fig 2. Examining the Ch01InChap01.dd.

img_stat identified the evidence as a raw image with a size of 1,474,560 bytes and 512-byte sectors. mmls produced no partition table, so the investigation did not assume an artificial partition offset.

Explanation

mmls was executed against Ch01InChap01.dd; no partition table was returned. Therefore, no arbitrary partition offset was assumed, and subsequent filesystem analysis was performed directly against the image.

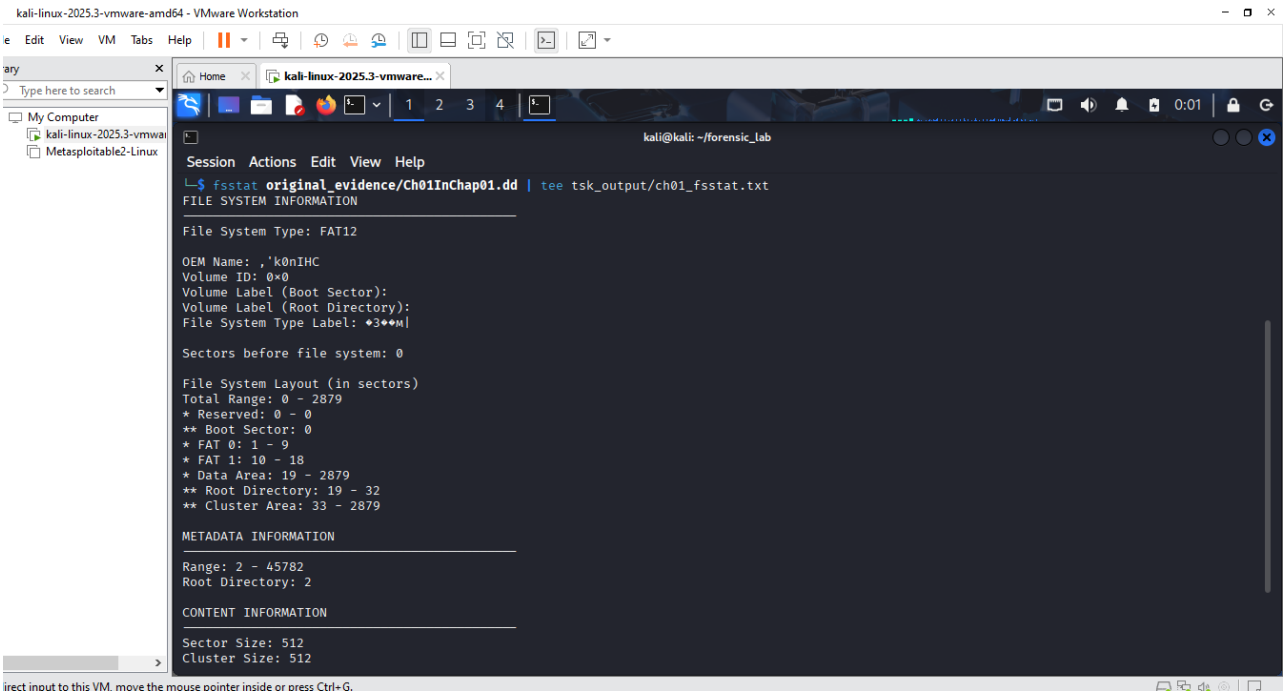

Fig 2.1 Examining the Ch01InChap01.dd page 2.

Explanation

fsstat identified the file system as FAT12 and displayed its layout, including FAT areas, the root directory, data area, and cluster region. This provided the structural information needed for later file and sector examination.

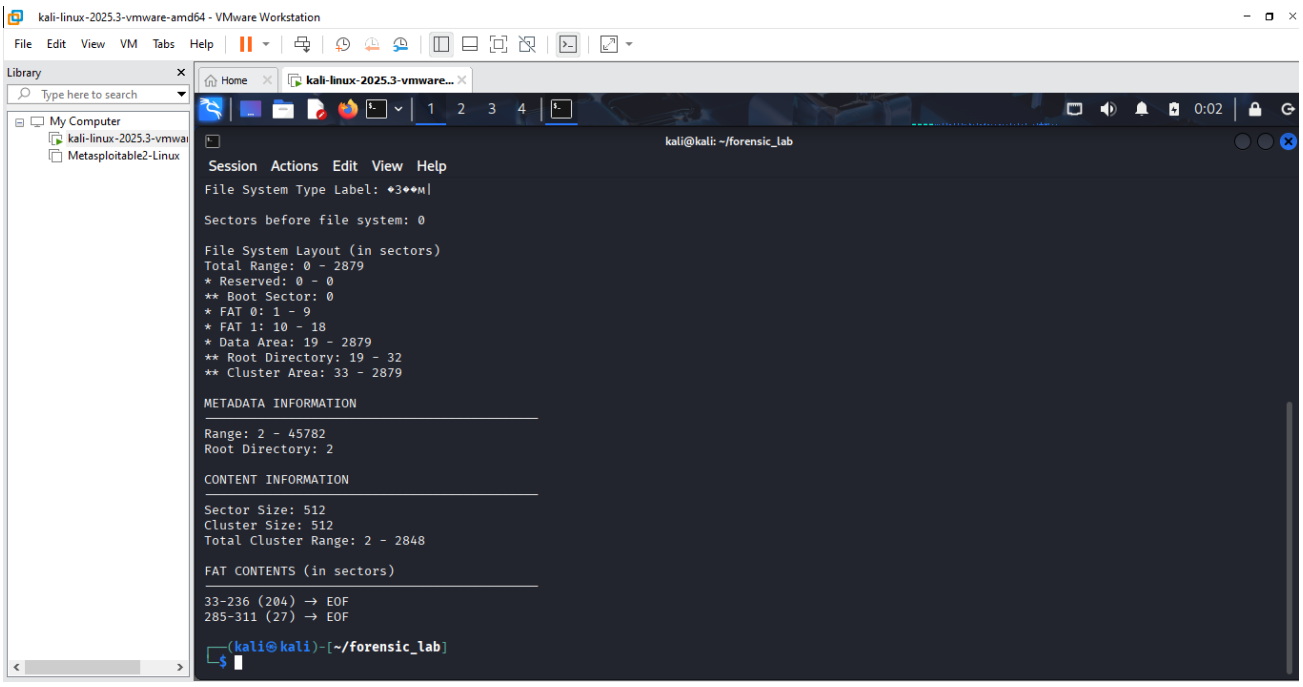

Fig 2.2 Examining the Ch01InChap01.dd page 3.

Explanation

This continuation records the remaining FAT12 layout and content information, including sector and cluster

sizes. It completed the initial file-system description of the image.

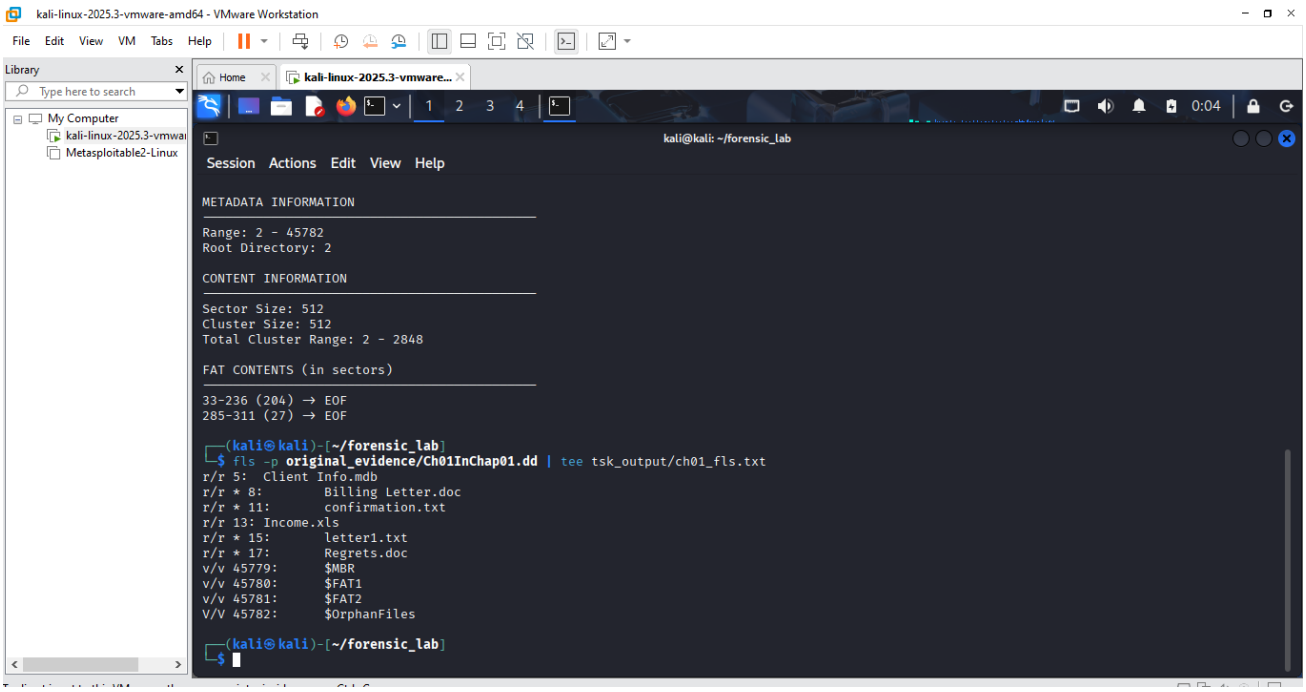

Fig 2.3 Examining the Ch01InChap01.dd page 4.

Explanation

fls was used to list directory entries from the FAT12 image. The output included normal and deleted files together with their metadata addresses, giving specific values that could be used for subsequent recovery.

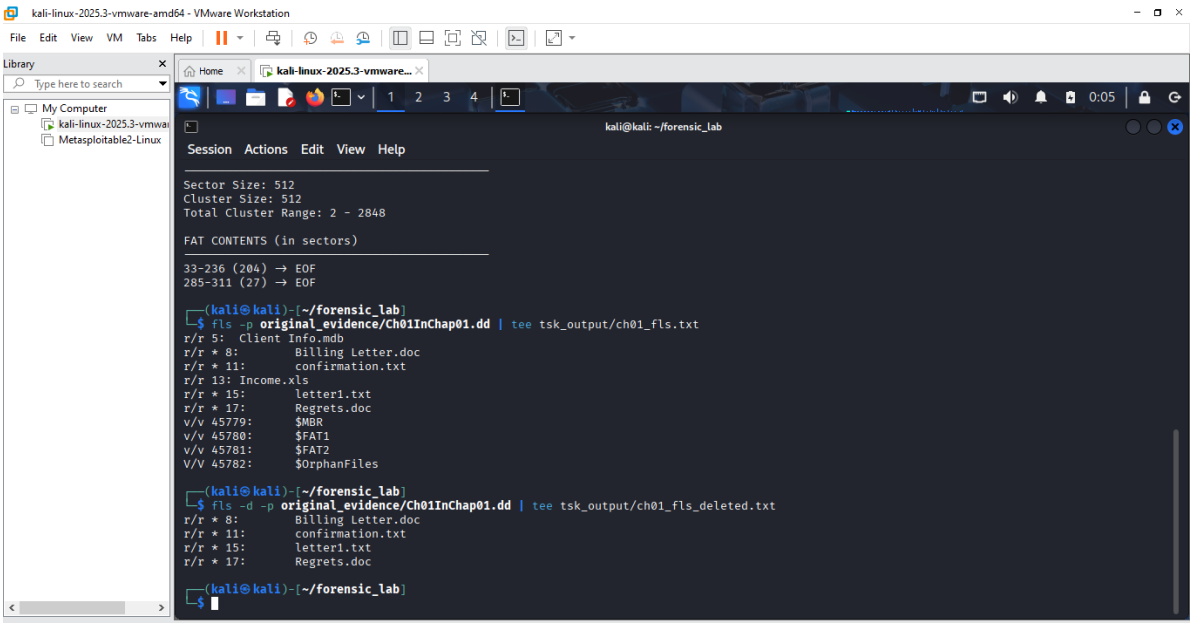

Fig 2.4 Examining the Ch01InChap01.dd page 5.

Explanation

The deleted-file listing was narrowed to entries marked as deleted. Files such as Billing Letter.doc, confirmation.txt, letter1.txt, and Regrets.doc were visible with their actual metadata addresses.

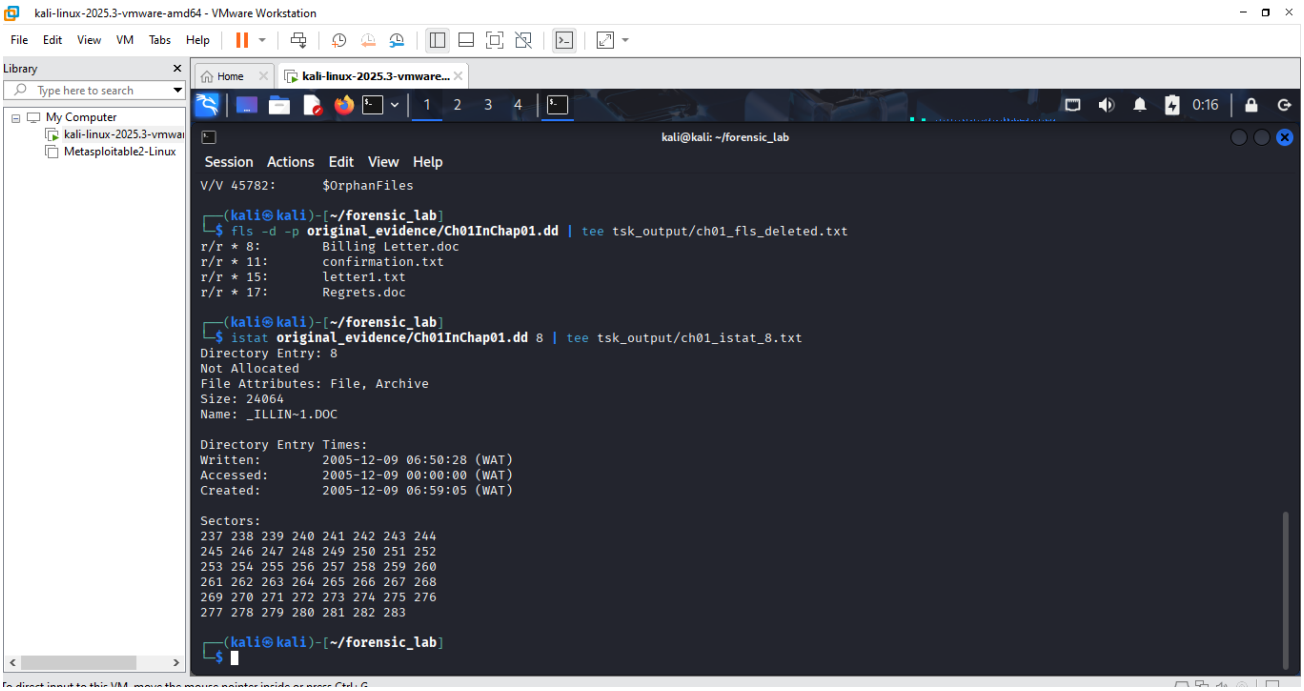

Fig 2.5 inspecting one deleted file with istat.

Explanation

Metadata entry 8 was examined with istat. The output showed that the entry was not allocated, had a size of 24,064 bytes, and occupied sectors 237 through 283.

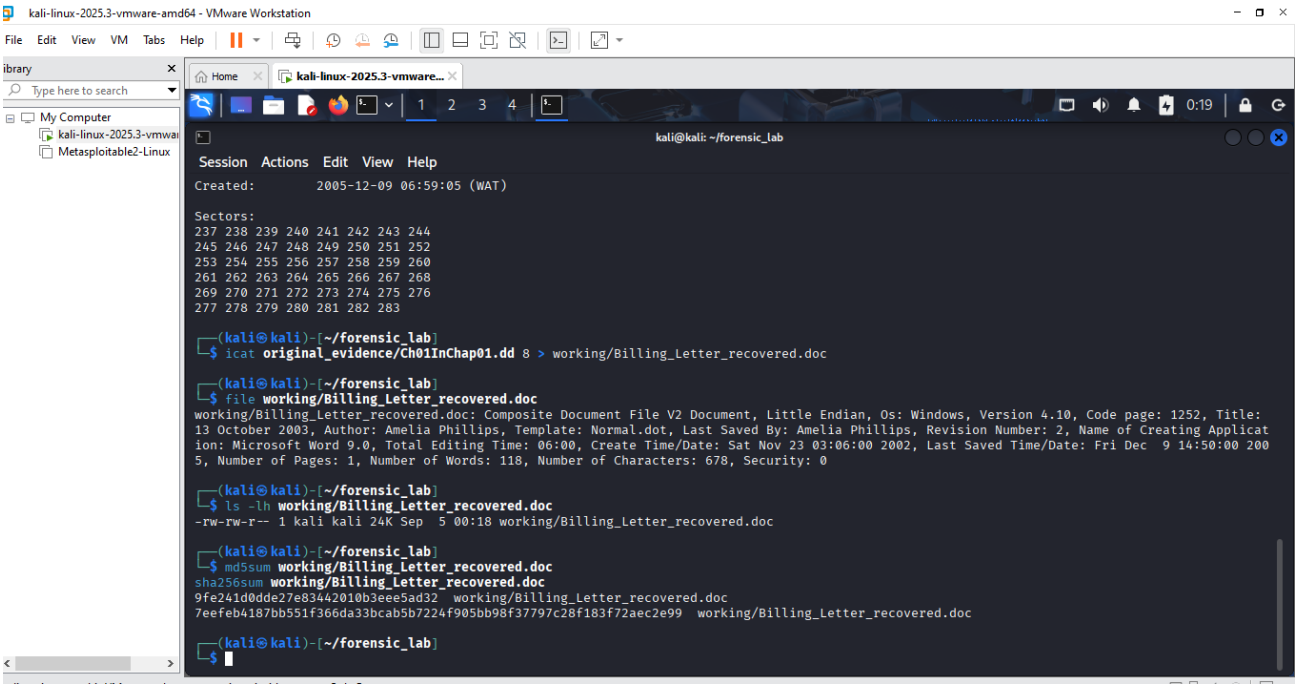

Fig 2.6 Extracting it with icat.

Explanation

icat was used with metadata address 8 to recover the deleted document. The recovered file was recognised as a Microsoft Compound Document, and MD5 and SHA-256 hashes were generated to preserve its integrity

information.

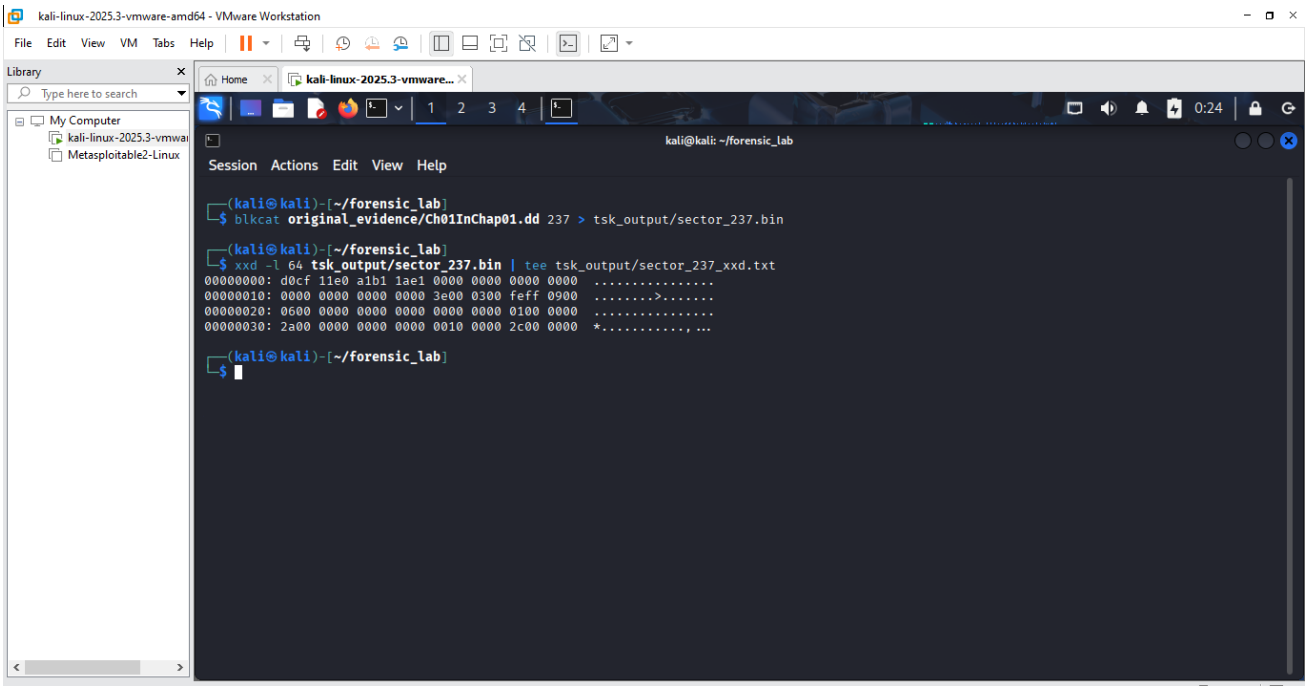

Fig 2.7 Completing the blkcat requirement.

Explanation

Sector 237, identified from the istat result, was extracted directly with blkcat and examined with xxd. The output began with D0 CF 11 E0 A1 B1 1A E1, matching the Microsoft Compound File signature.

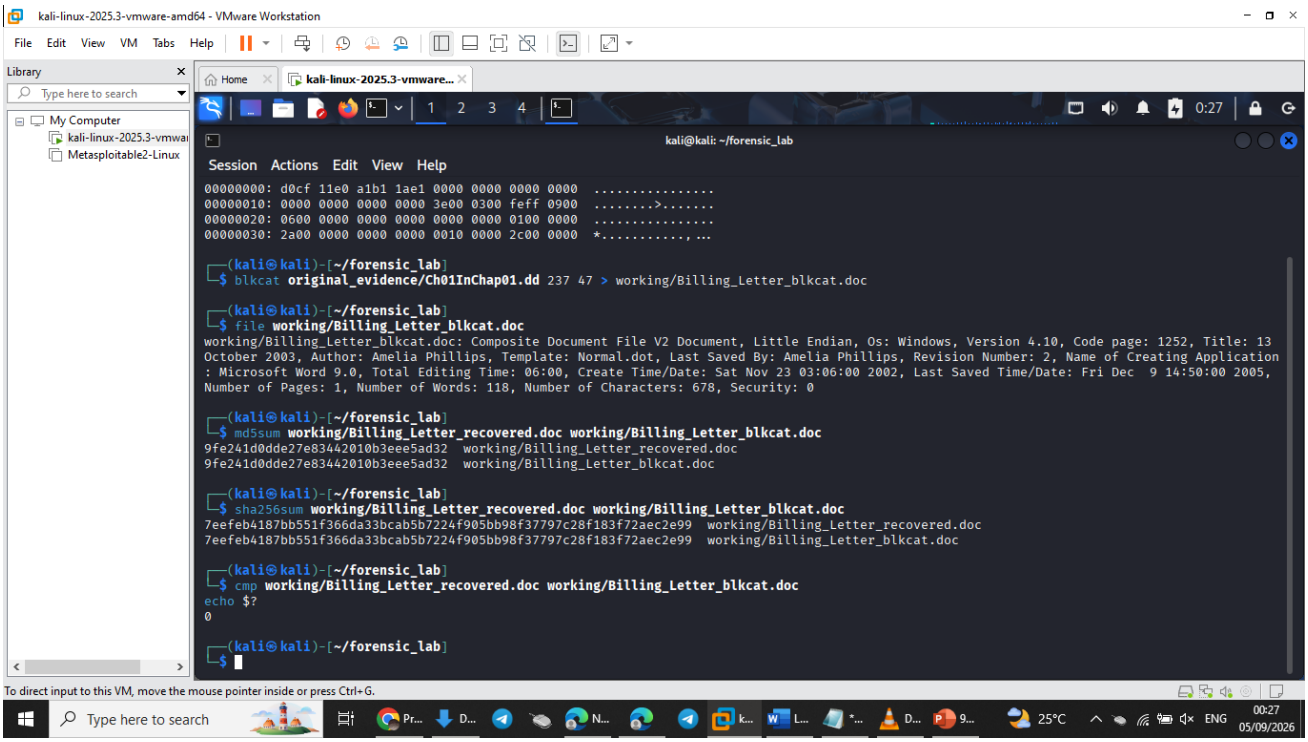

Fig 2.8 Completing the blkcat requirement page 1.

All 47 sectors from 237 to 283 were directly extracted and reconstructed into a document. Its MD5 and SHA-256 hashes matched the icat recovery, and cmp confirmed that both recovery approaches produced identical content.

Explanation

The deleted file was first recovered logically using icat and metadata address 8. istat identified its physical data sectors as 237–283. Direct extraction of the same 47 sectors with blkcat produced a byte-for-byte identical file.

The Sleuth Kit was used to examine Ch01InChap01.dd. img_stat identified the evidence as a raw 1,474,560-byte image with 512-byte sectors. mmls returned no partition table, so no arbitrary offset was applied. fsstat identified a FAT12 filesystem. fls identified several deleted files, including Billing Letter.doc at metadata address 8. istat confirmed that entry 8 was not allocated and occupied sectors 237–283, with a file size of 24,064 bytes. The file was recovered using icat. Sector 237 was then examined directly with blkcat, revealing the Microsoft Compound File signature D0 CF 11 E0 A1 B1 1A E1. Direct extraction of all 47 sectors with blkcat produced a file whose MD5 and SHA-256 hashes were identical to the icat recovery.

Part D - Embedded File Discovery with Binwalk

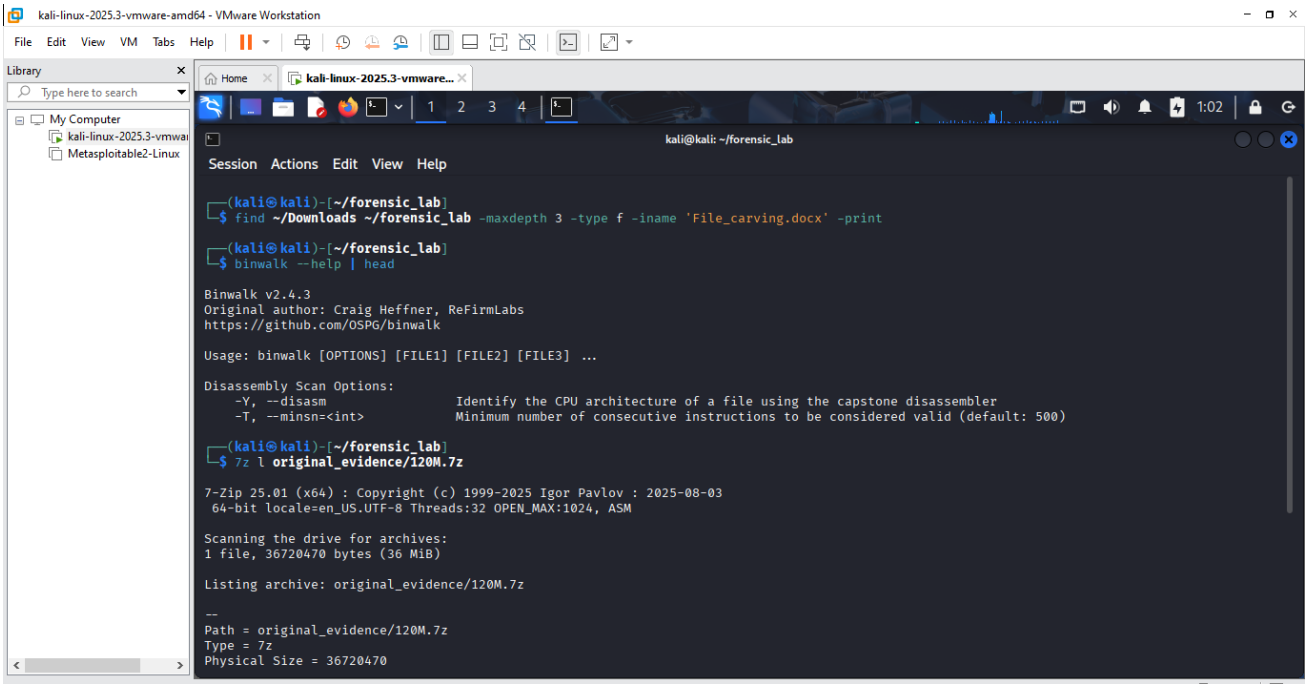

Fig 2.9 Searching for the doc file.

Explanation

A recursive search was carried out for the instructor-referenced File_carving.docx, while Binwalk availability was also checked. The screenshot begins the process of confirming whether the required DOCX resource had actually been supplied.

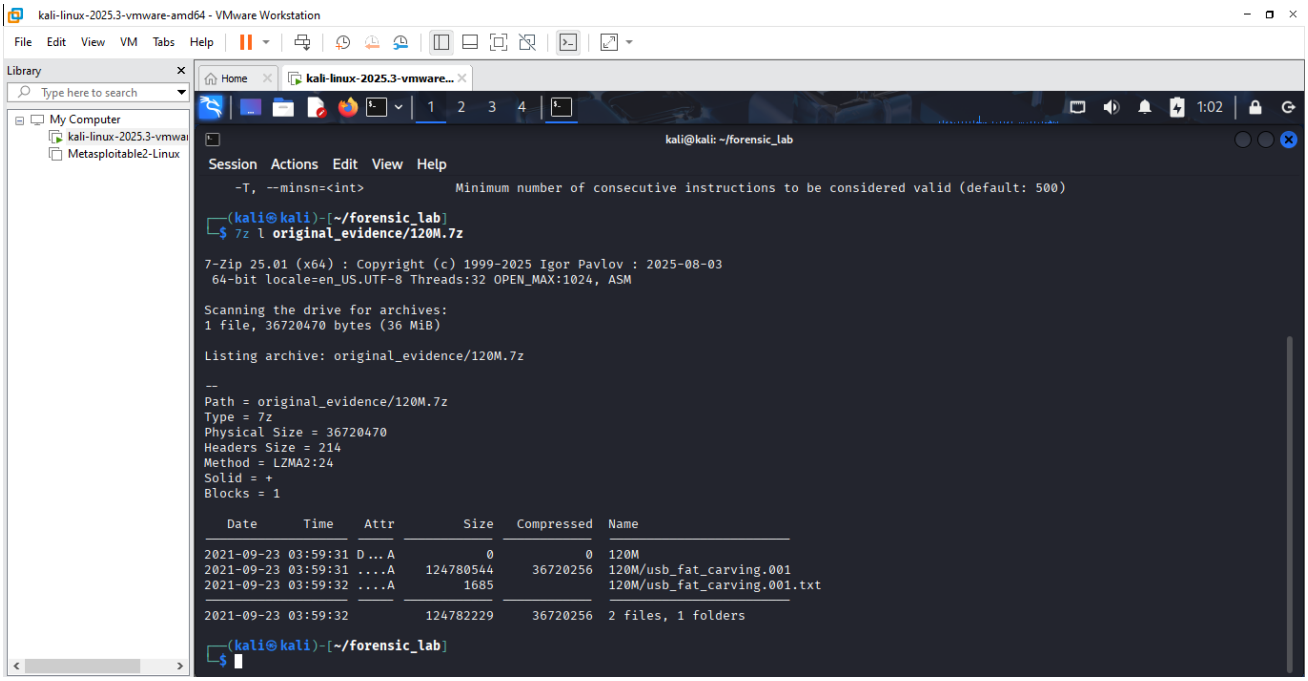

Fig 3.0 Searching for the doc file page 1.

Explanation

The contents of 120M.7z were inspected using 7z l. The archive contained usb_fat_carving.001 and its associated text file, but the required File_carving.docx was not listed.

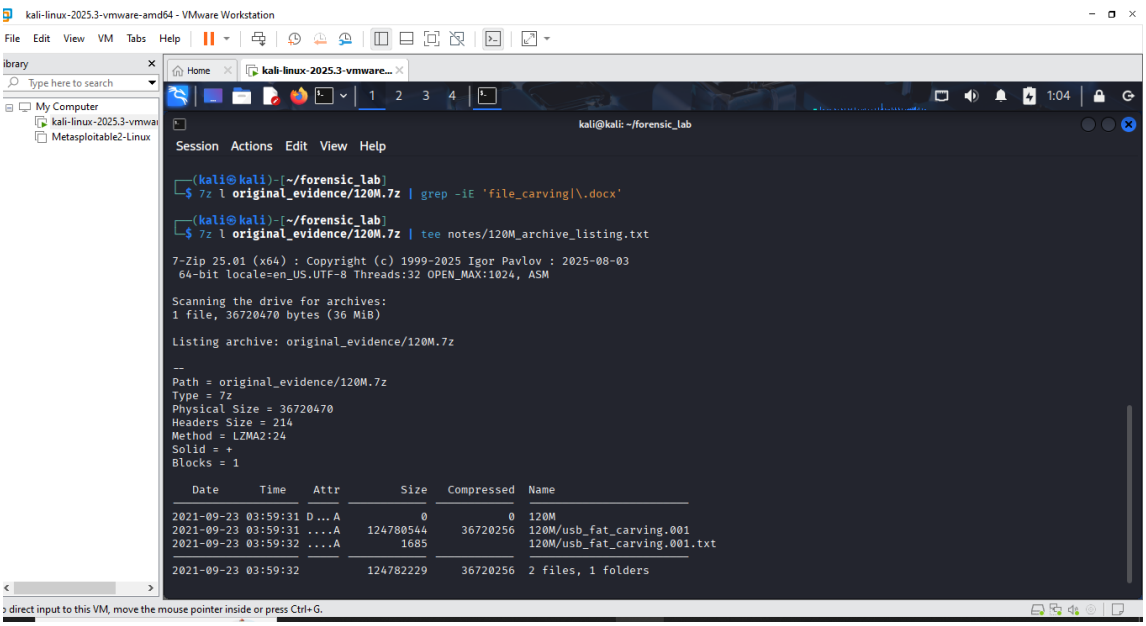

Fig 3.1 Searching for the doc file page 2.

Explanation

Binwalk detected JPEG/EXIF data, a little-endian TIFF structure, and an Adobe copyright string inside J_ub_law.jpg. No recognised Binwalk signatures were returned for Ch01InChap01.dd, so the investigation continued with the supplied USB forensic image.

Limitation – File_carving.docx:

The laboratory instructions reference an instructor-provided File_carving.docx for the demonstrated manual and automatic Binwalk extraction workflow. A recursive search of the available working directories did not locate the file. The supplied 120M.7z archive was also inspected using 7z l; it contained usb_fat_carving.001 and an associated text file, but no File_carving.docx. Consequently, the DOCX-specific workflow could not be reproduced. In accordance with the laboratory instructions, Binwalk practice was instead completed using the other authorised evidence files.

Complete the Binwalk practice using the other supplied lab evidence.

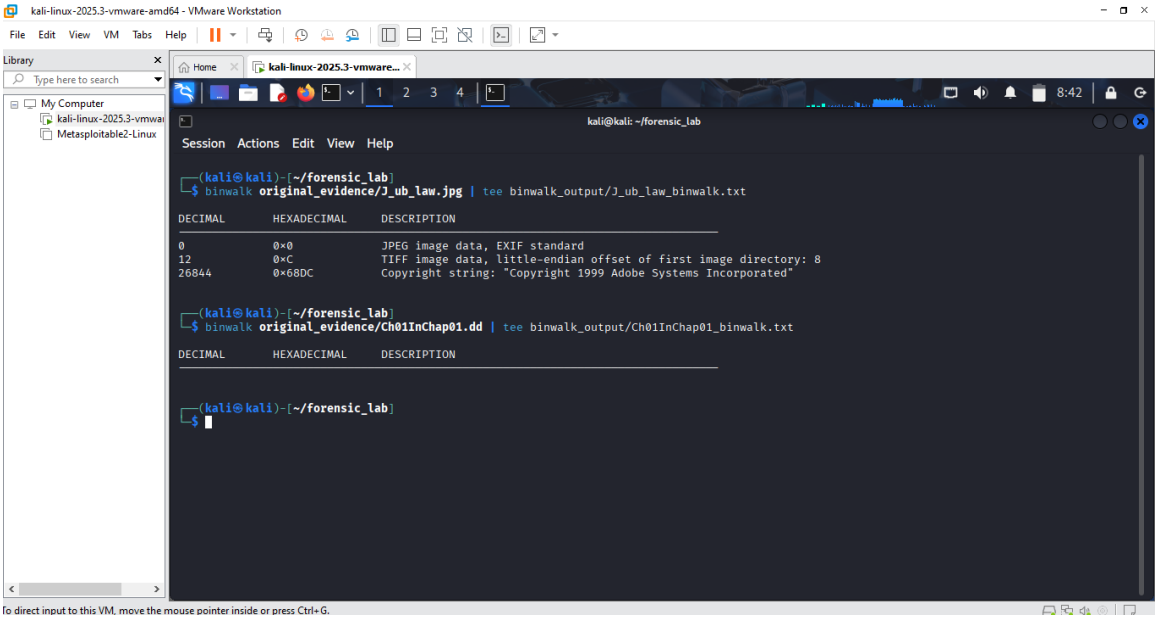

Fig 3.1 Using binwalk for both authorize files.

DECIMAL	HEXADECIMAL	DESCRIPTION
0	0x0	JPEG image data, EXIF standard
12	0xC	TIFF image data, little-endian offset of first image directory: 8
26844	0x68DC	Copyright string: "Copyright 1999 Adobe Systems Incorporated"

File_carving.docx was not available among the supplied evidence or within 120M.7z. In accordance with the laboratory instructions, Binwalk analysis was therefore performed using the other authorised evidence. Binwalk identified JPEG/EXIF data at offset 0, an embedded little-endian TIFF structure at decimal offset 12 (0xC), and an Adobe copyright string at decimal offset 26,844 (0x68DC) within J_ub_law.jpg. No recognised Binwalk signatures were reported for Ch01InChap01.dd. Further Binwalk examination and extraction was performed on the supplied USB forensic image after preparation.

Part E — Prepare the USB image

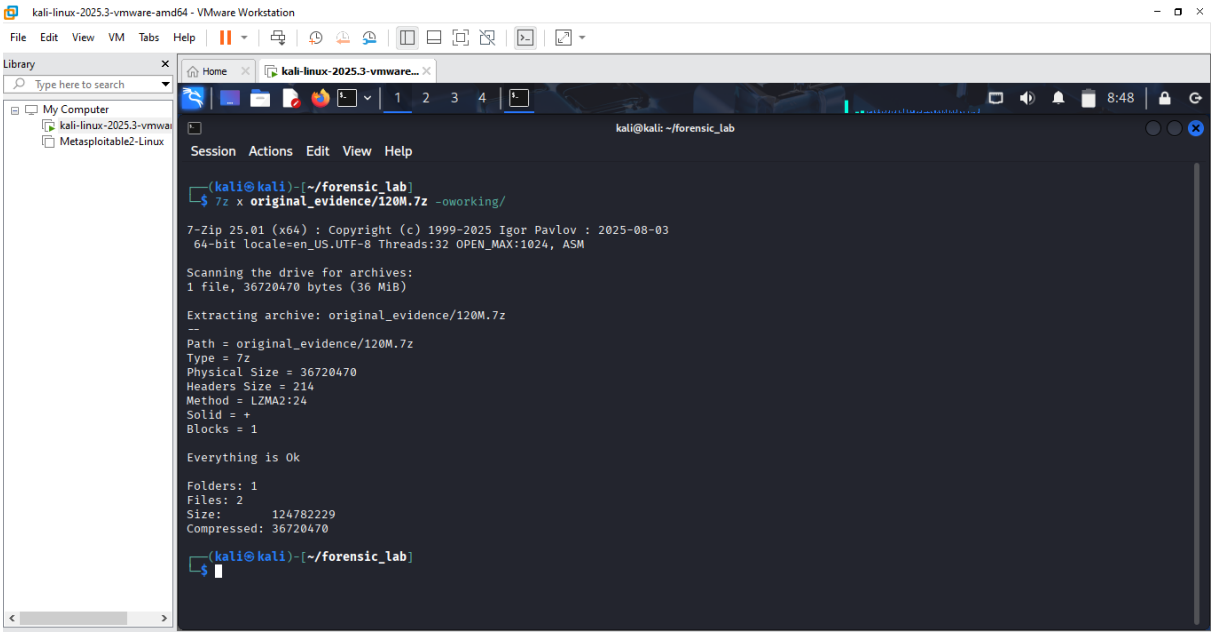

Fig 3.2 Extracting into working and keeping the .7z untouched.

Explanation

The 120M.7z archive was extracted into the working area rather than modifying the preserved archive. This produced the USB forensic image required for the remaining analysis.

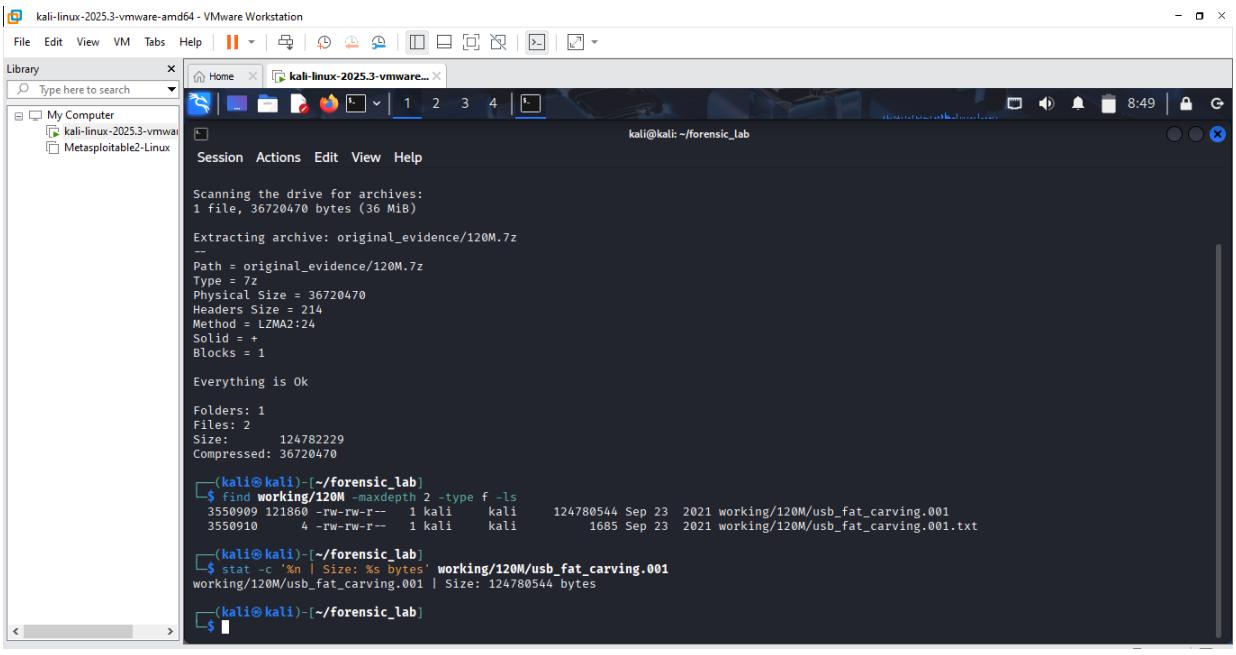

Fig 3.3 Seeing exactly what was extracted and identifying the image type.

Explanation

The extracted contents were listed, showing usb_fat_carving.001 and its accompanying text file. The image was then examined to identify its type before further analysis.

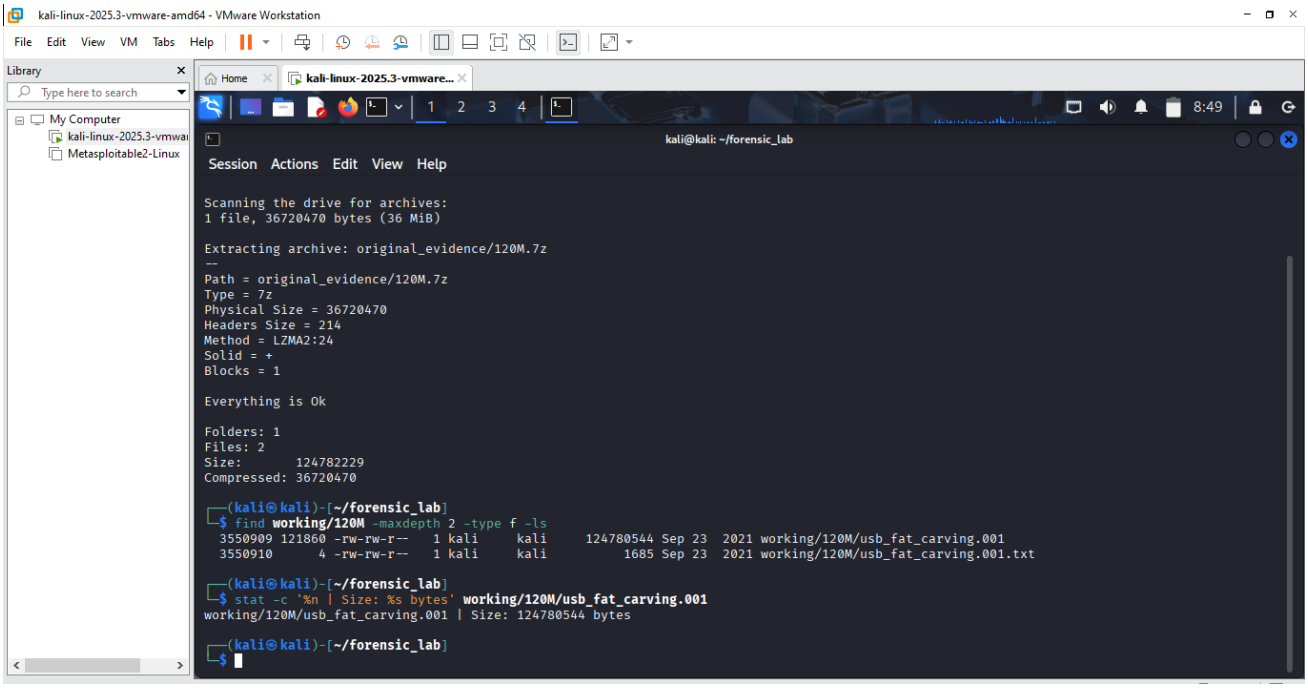

Fig 3.4 Recording the exact size for usb_fat_carving.001.

Explanation

The extracted USB image size was recorded as 124,780,544 bytes. Recording the exact size provided another reference point for evidence identification and integrity.

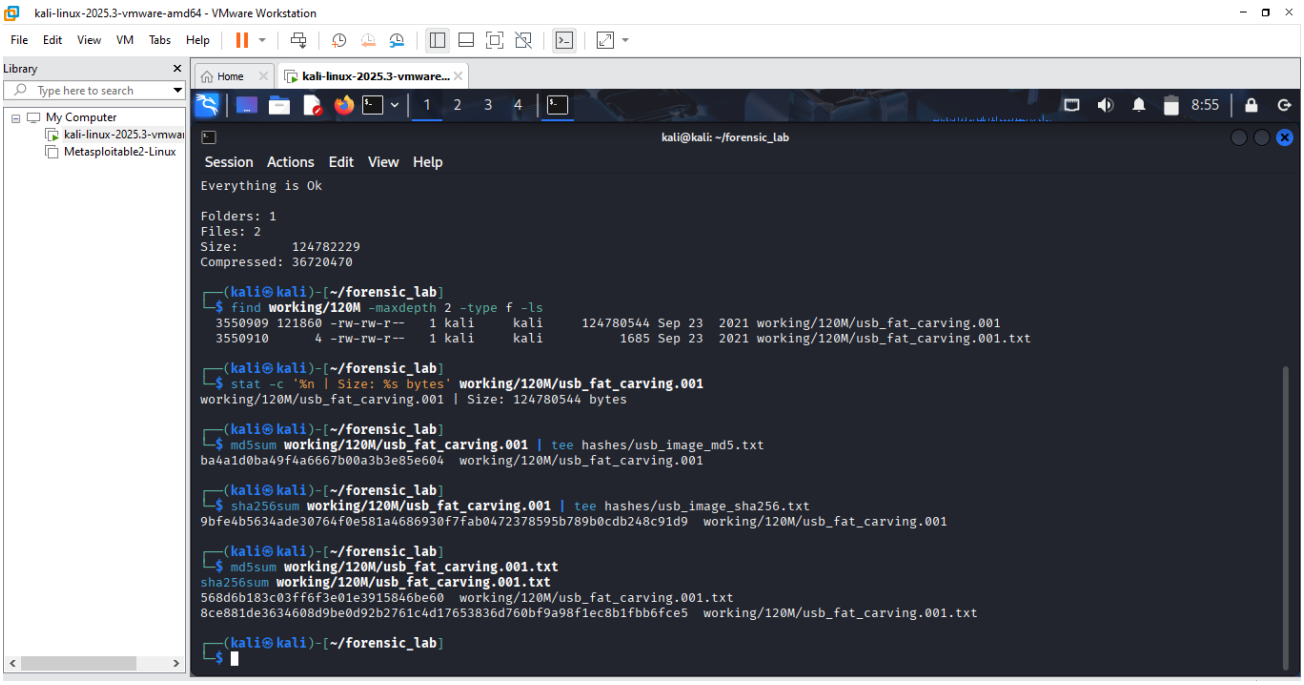

Fig 3.5 Hashing the extracted forensic image with md5sum and sha256sum.

Explanation

MD5 and SHA-256 hashes were generated for usb_fat_carving.001 and its accompanying text file. The USB image MD5 was ba4a1d0ba49f4a6667b00a3b3e85e604, while its SHA-256 was 9bfe4b5634ade30764f0e581a4686930f7fab0472378595b789b0cdb248c91d9.

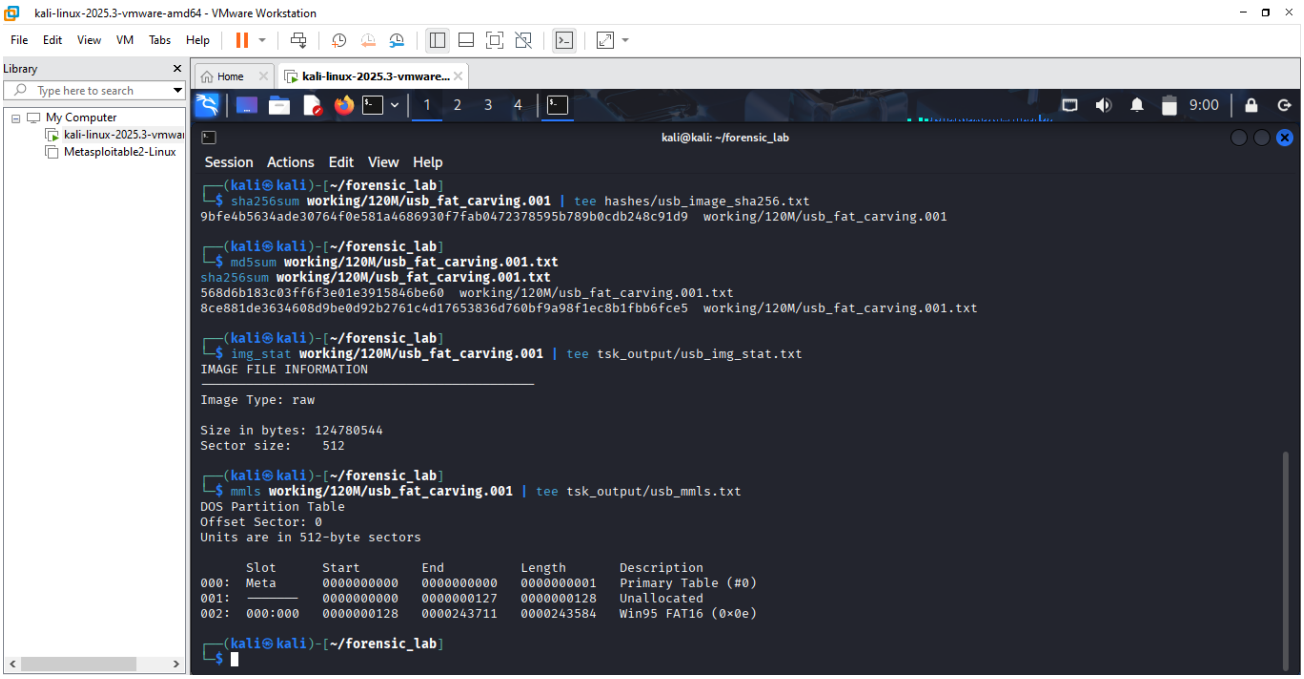

Fig 3.6 Determining the partition structure.

Explanation

img_stat and mmls were used to examine the image structure. mmls showed 128 unallocated sectors followed by a Win95 FAT16 partition beginning at sector **128**, confirming the offset required for later TSK commands.

mmls output shows:

Slot	Start	End	Length	Description
001	0	127	128	Unallocated
002	128	243711	243584	Win95 FAT16 (0x0e)

USB image preparation

Image size: 124,780,544 bytes

Image type: Raw

Sector size: 512 bytes

MD5: ba4a1d0ba49f4a6667b00a3b3e85e604

SHA-256: 9bfe4b5634ade30764f0e581a4686930f7fab0472378595b789b0cdb248c91d9

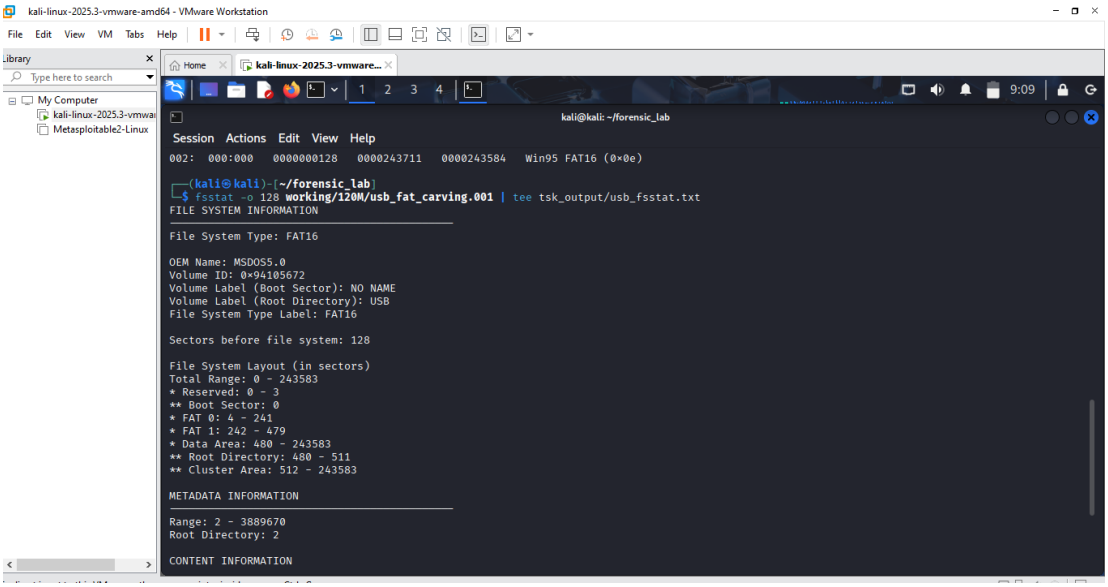

Fig 3.7 Inspecting the FAT16 filesystem at the confirmed offset.

Explanation

fsstat was executed using offset 128 and identified the file system as FAT16 with the volume label USB. The output also began to show the FAT and data-area layout.

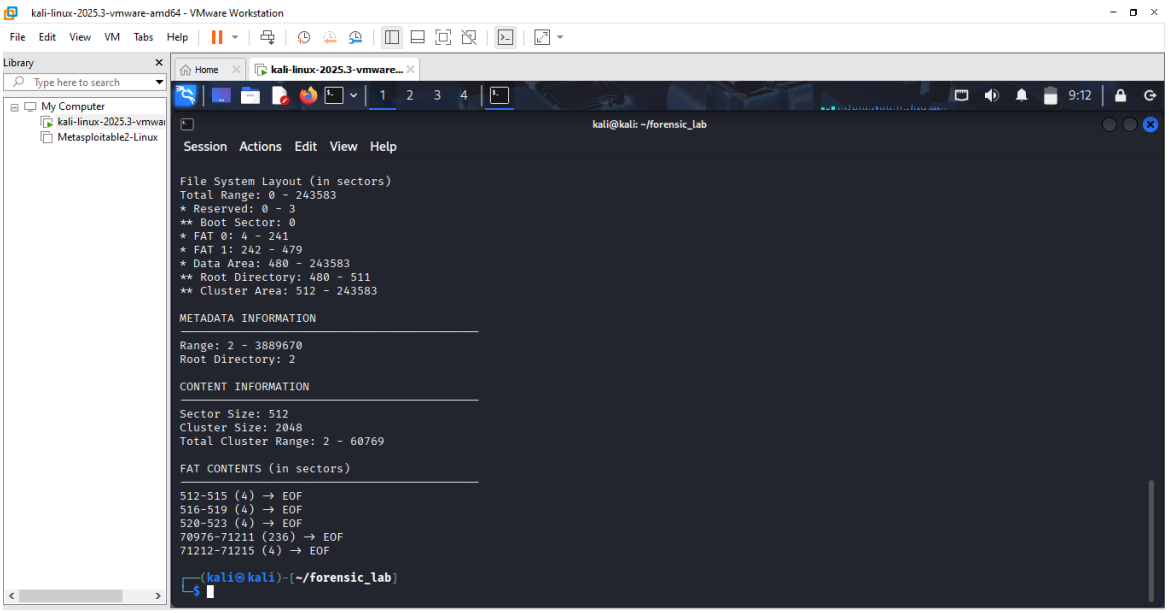

Fig 3.8 Inspecting the FAT16 filesystem at the confirmed offset page 1.

Explanation

The remaining fsstat output shows a 512-byte sector size, 2048-byte cluster size, FAT locations, root directory, data area, and cluster area. These values describe the structure of the USB filesystem used in the later recovery work.

Filesystem: FAT16

OEM: MSDOS5.0

Volume label: USB

Sectors before filesystem: 128

Sector size: 512 bytes

Cluster size: 2048 bytes

Filesystem sector range: 0–243583

FAT 0: sectors 4–241

FAT 1: sectors 242–479

Data area: sectors 480–243583

Root directory: sectors 480–511

Cluster area: sectors 512–243583

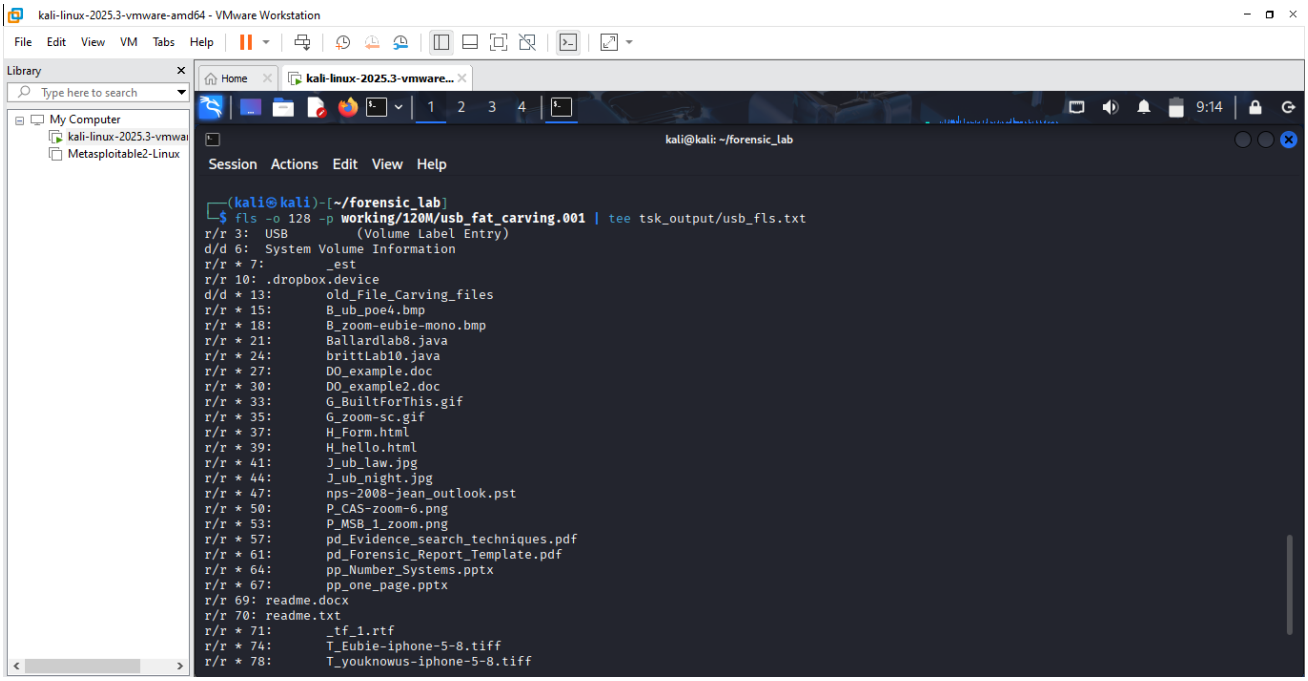

Fig 3.9 listing the files.

Explanation

fls was run at the confirmed offset to display files and directories found on the USB image. The listing included a range of document, image, audio, archive, and other file types.

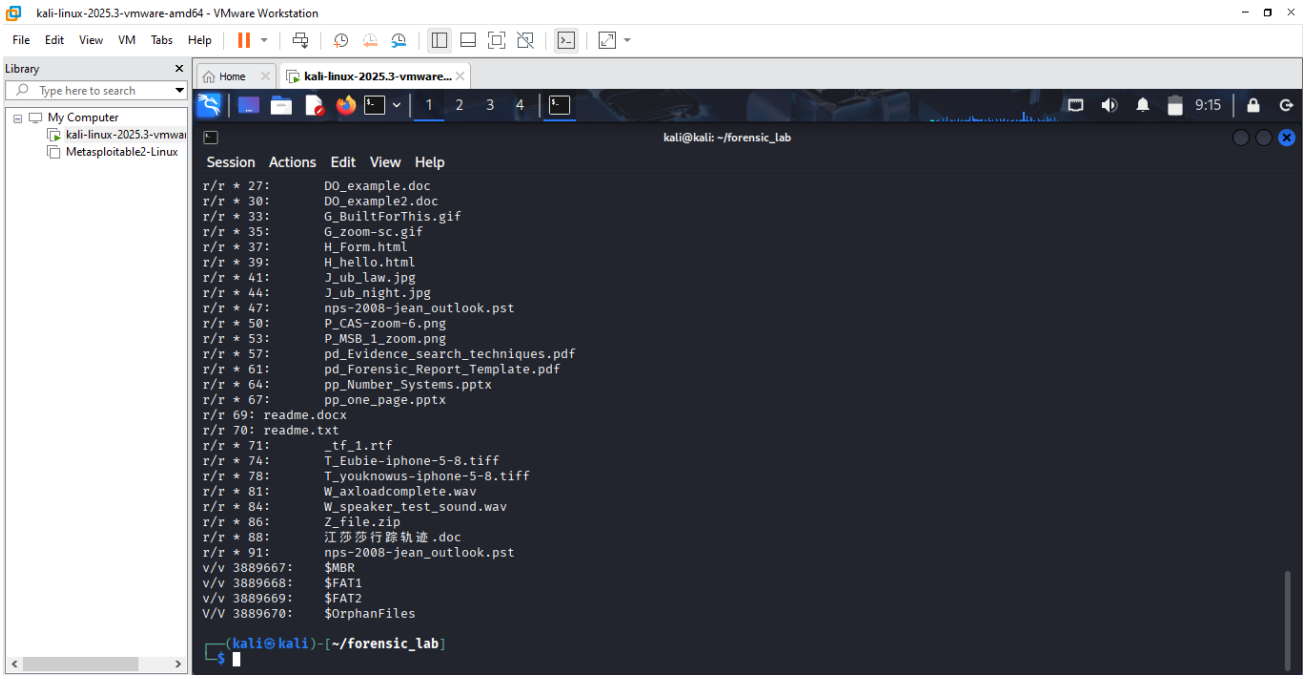

Fig 4.0 listing the files page 1.

Explanation

This screenshot continues the fls output and records additional files and special FAT entries. Together with the previous screenshot, it provides a more complete view of the accessible file-system contents.

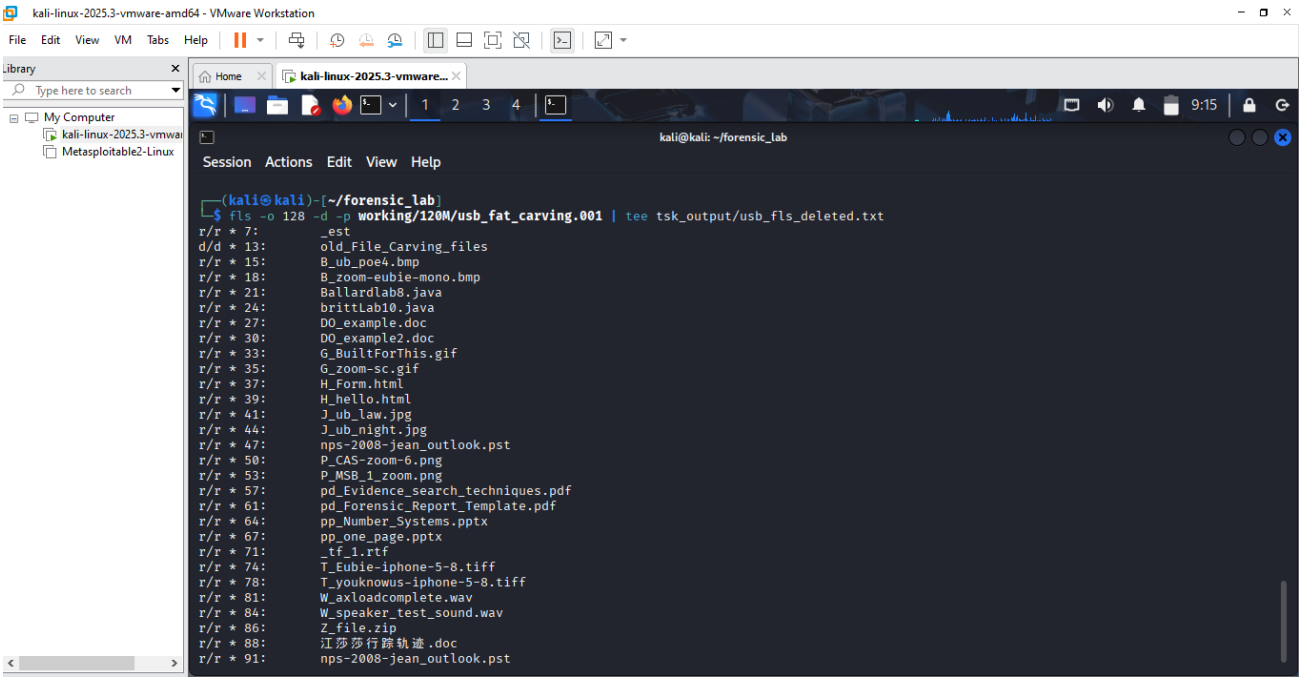

Fig 4.1 Showing deleted entries.

Explanation

fls was run specifically to display deleted entries. The results showed several deleted files, including JPEGs that could later be recovered through signature-based carving.

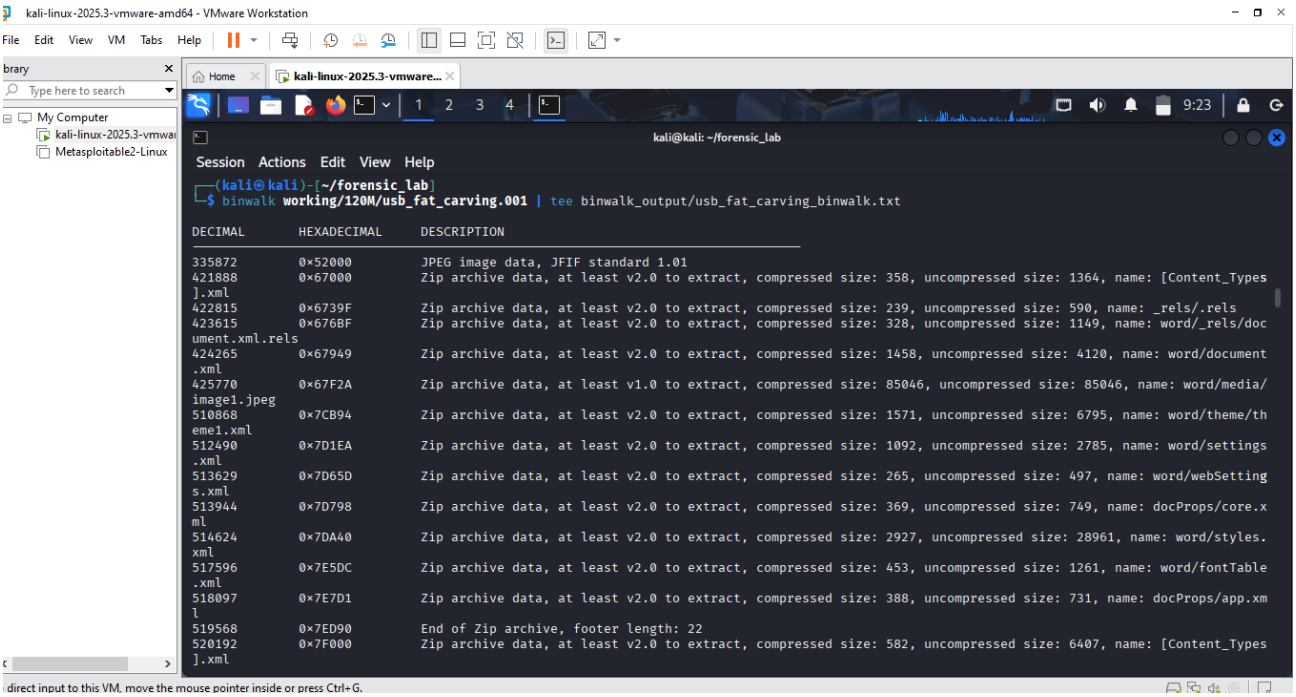

Fig 4.2 Examining fat_carving.001 using binwalk.

Explanation

Binwalk identified numerous embedded signatures in the USB image, including JPEG and ZIP/Office-related structures. The output also showed internal Word OOXML components such as [Content_Types].xml and word/document.xml.

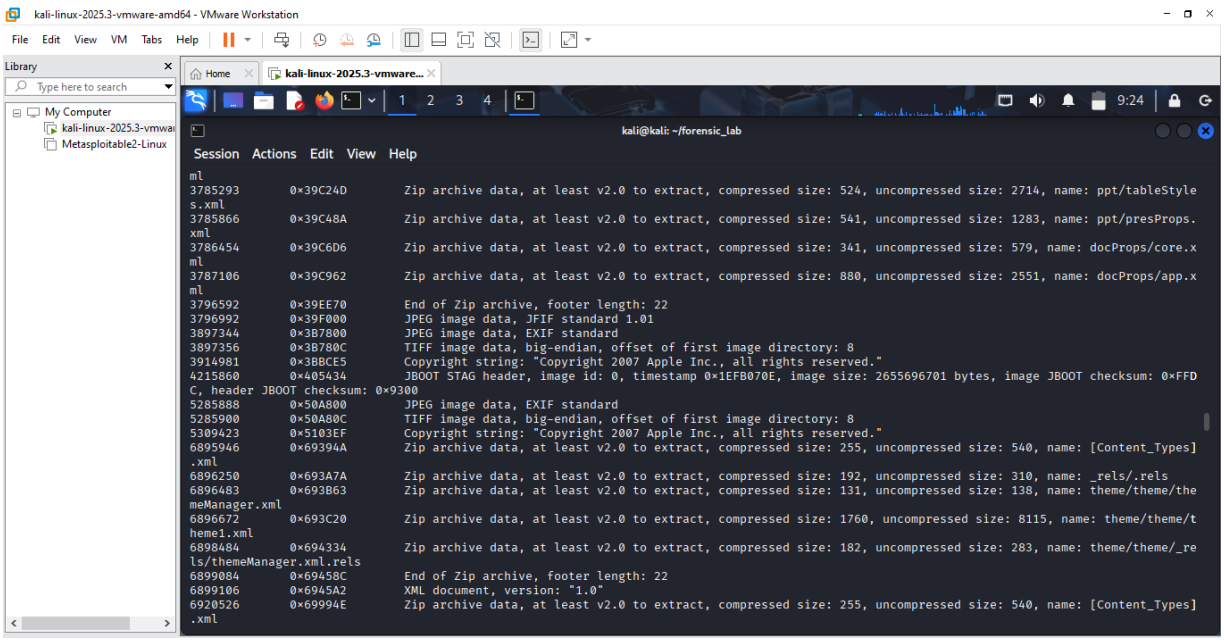

Fig 4.3 Examining fat_carving.001 using binwalk page 1.

Explanation

This continuation shows further signatures and embedded structures at additional offsets. JPEG, TIFF/EXIF,

ZIP, XML, and copyright-related data were among the items detected.

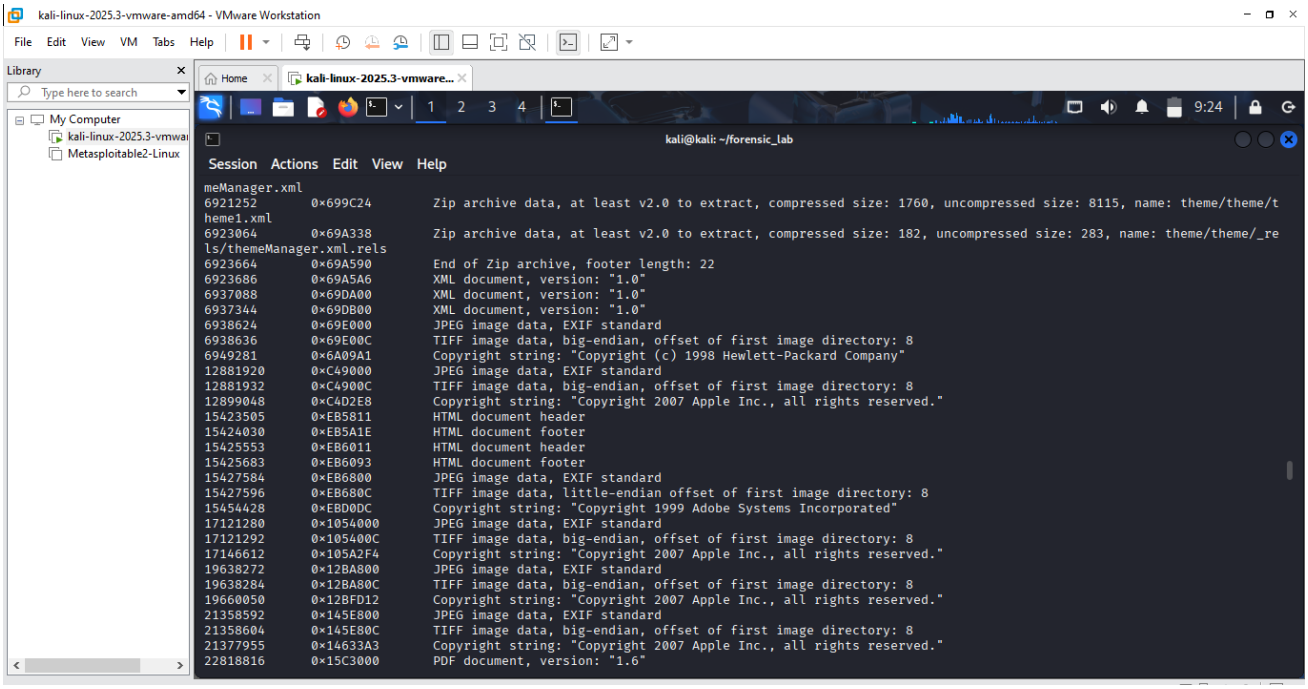

Fig 4.4 Examining fat_carving.001 using binwalk page 2.

Explanation

More embedded content was identified deeper in the image, including image structures, HTML headers and footers, PDF content, and additional metadata-related strings. This demonstrated that many different file signatures were present within the raw image.

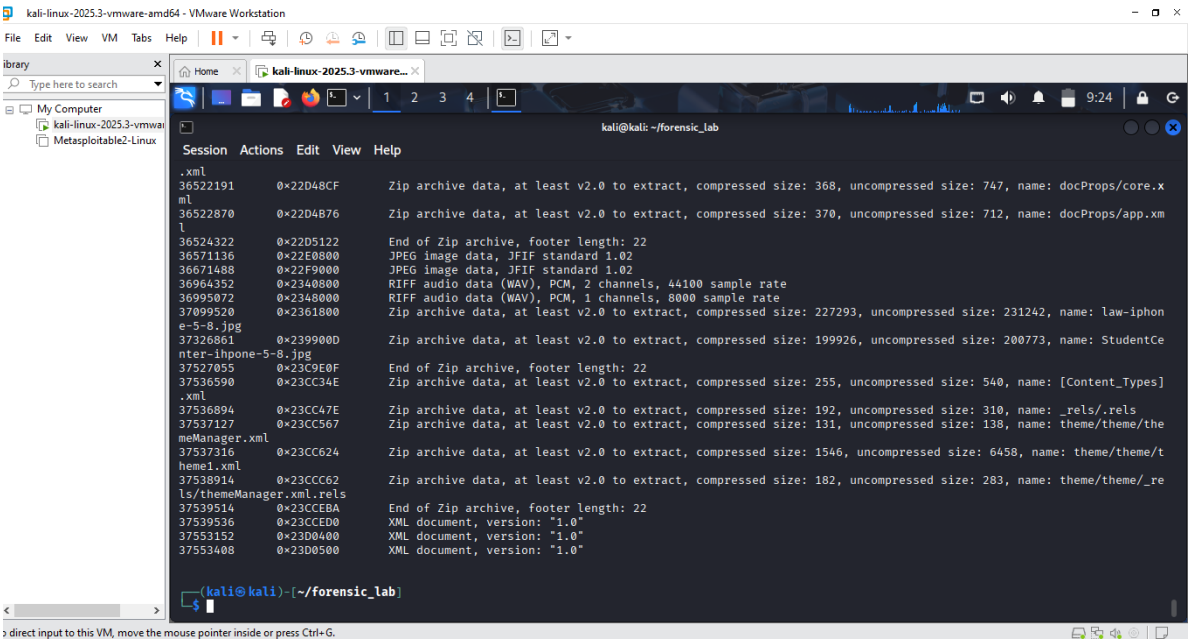

Fig 4.5 Examining fat_carving.001 using binwalk page 3.

Explanation

The final part of the Binwalk scan shows still more ZIP, image, audio, and XML-related signatures. The broad range of results provided useful offsets for both automatic and manual extraction attempts.

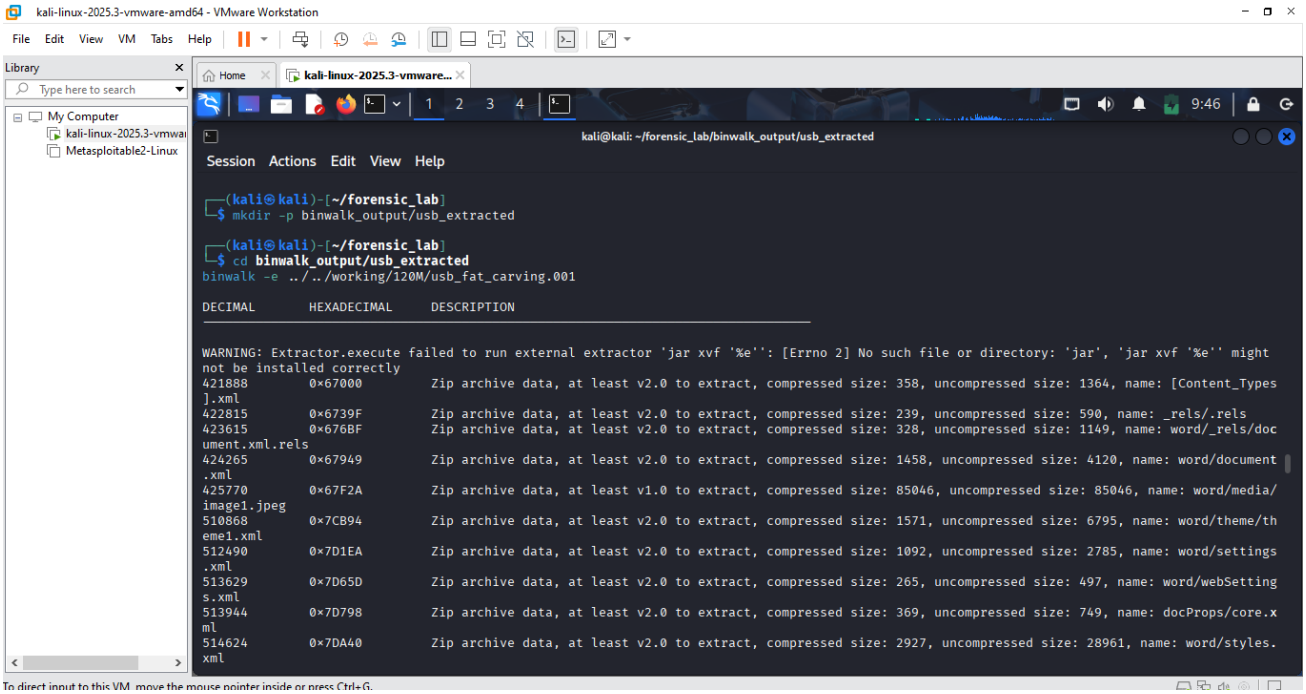

Fig 4.6 Automatic Binwalk extraction.

Explanation

Binwalk automatic extraction was run against the USB image. The process began extracting recognised structures but also reported that the external jar extractor was unavailable.

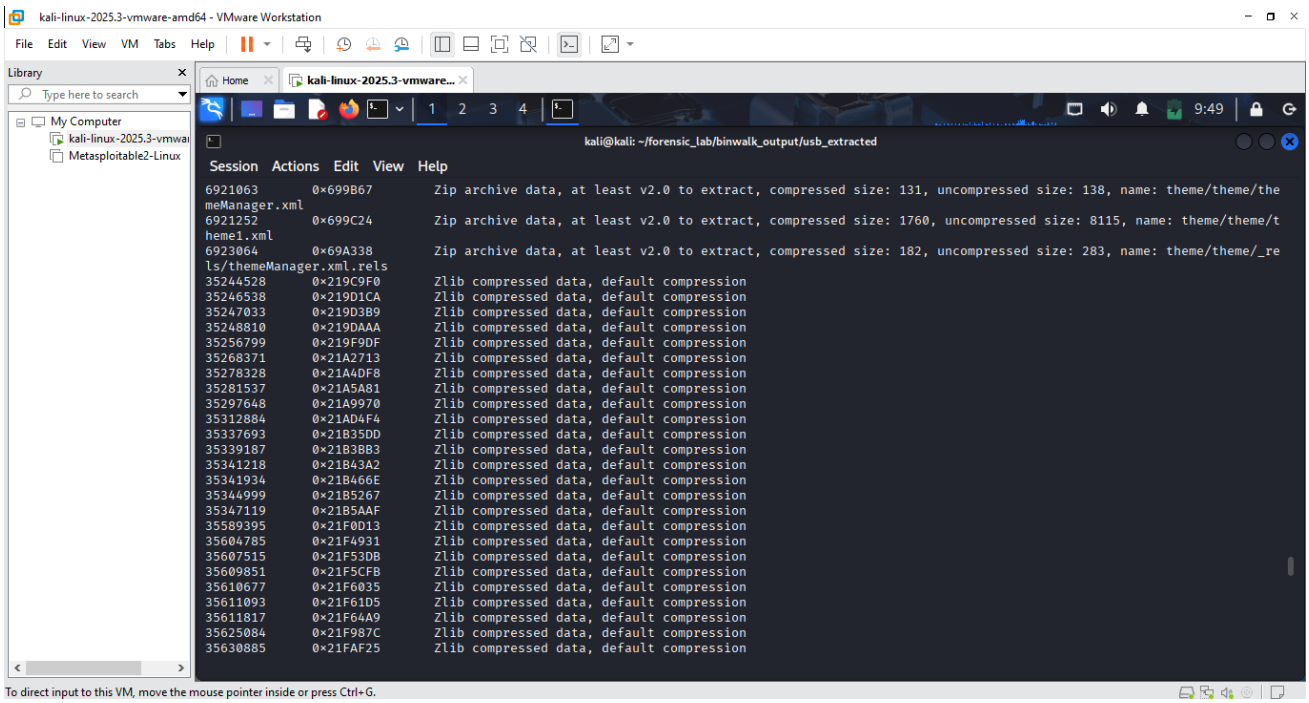

Fig 4.7 Automatic Binwalk extraction page 1.

Explanation

The automatic process continued to identify and extract compressed structures. Numerous zlib-related objects were detected during this stage.

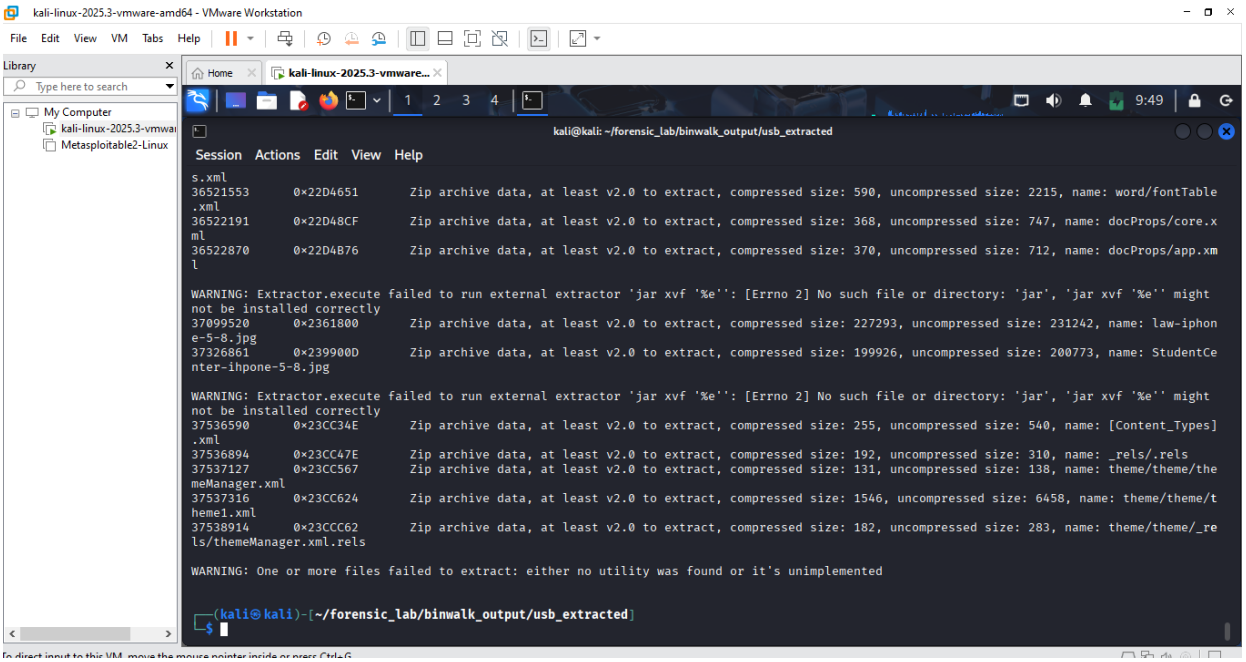

Fig 4.8 Automatic Binwalk extraction page 2.

Explanation

This screenshot records additional extractor warnings, including instances where the required external utility could not be found. Binwalk therefore completed only the extraction operations supported by the available utilities.

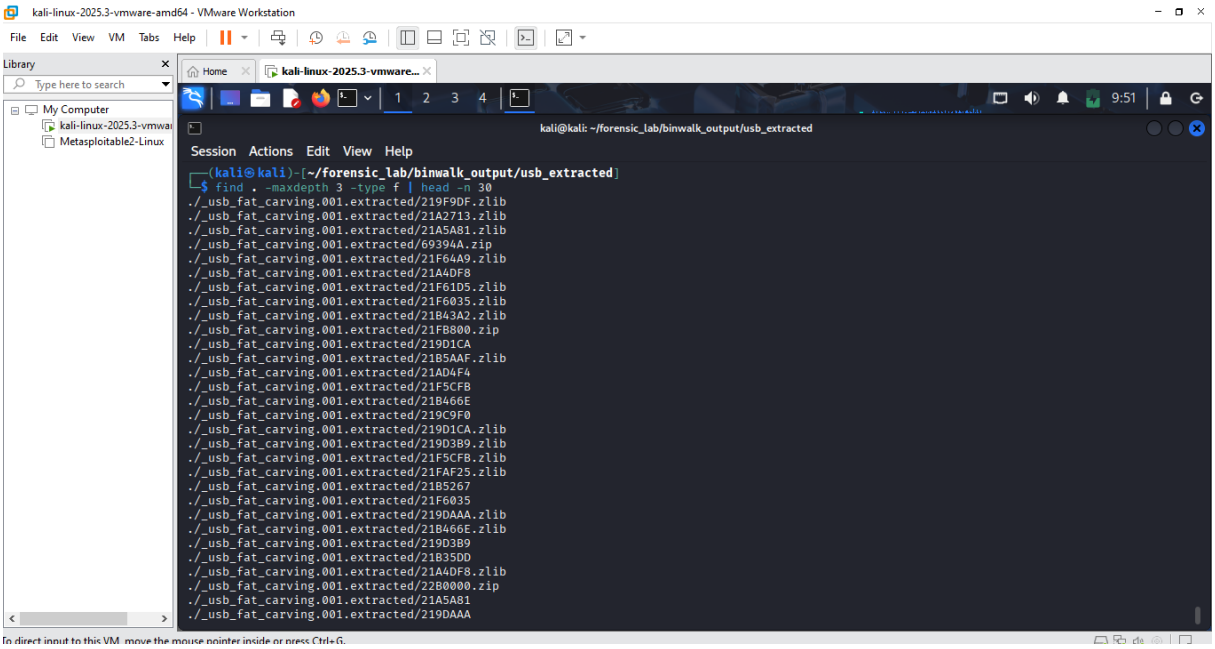

Fig 4.9 Automatic Binwalk extraction page 3.

Explanation

This continuation shows more of the automatically generated extraction files. It documents the large number of derived objects created during the Binwalk process.

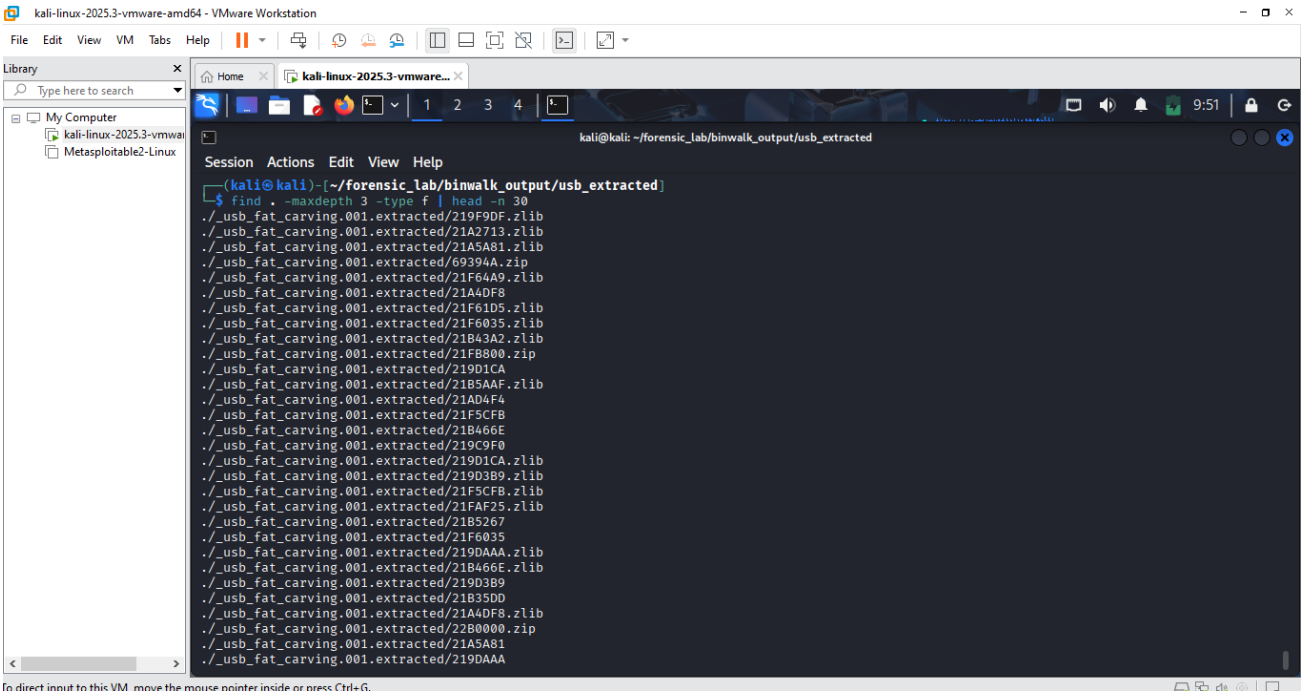

Fig 5.0 Automatic Binwalk extraction page 4.

Explanation

This continuation shows more of the automatically generated extraction files. It documents the large number of derived objects created during the Binwalk process.

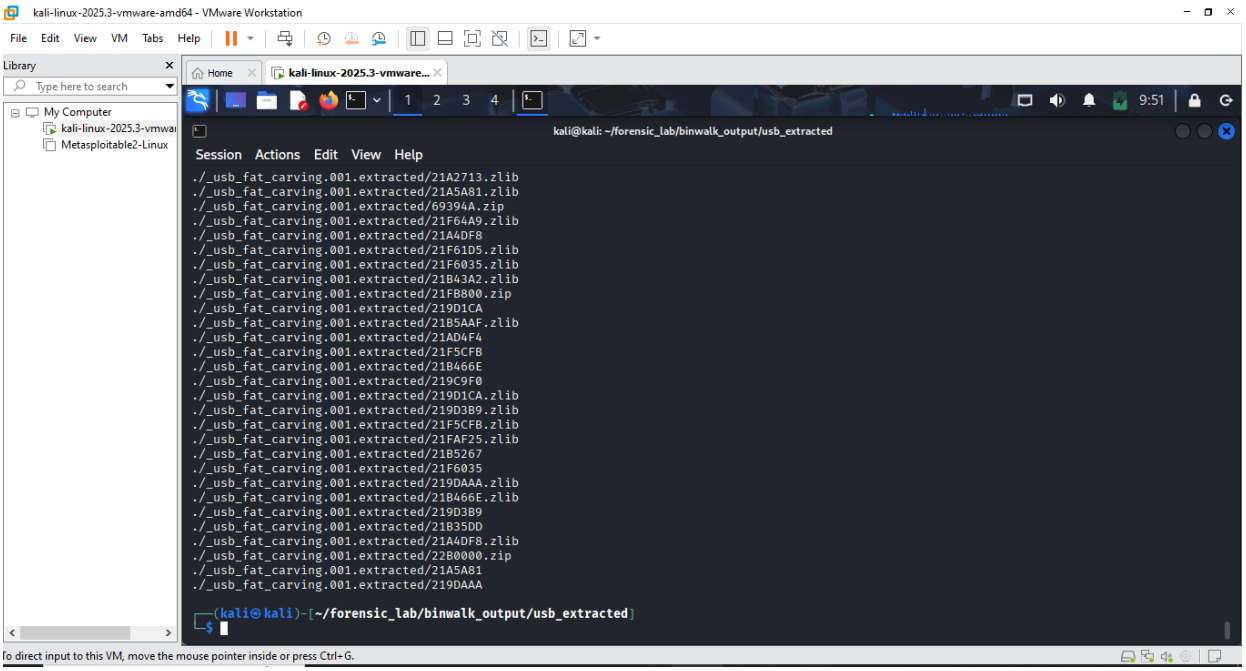

Fig 5.0.1 Automatic Binwalk extraction page 5.

Demonstrating a manual Binwalk-guided extraction

start = 421888

end-of-ZIP record = 519568

footer length = 22

519568 + 22 - 421888 = 97,702 bytes

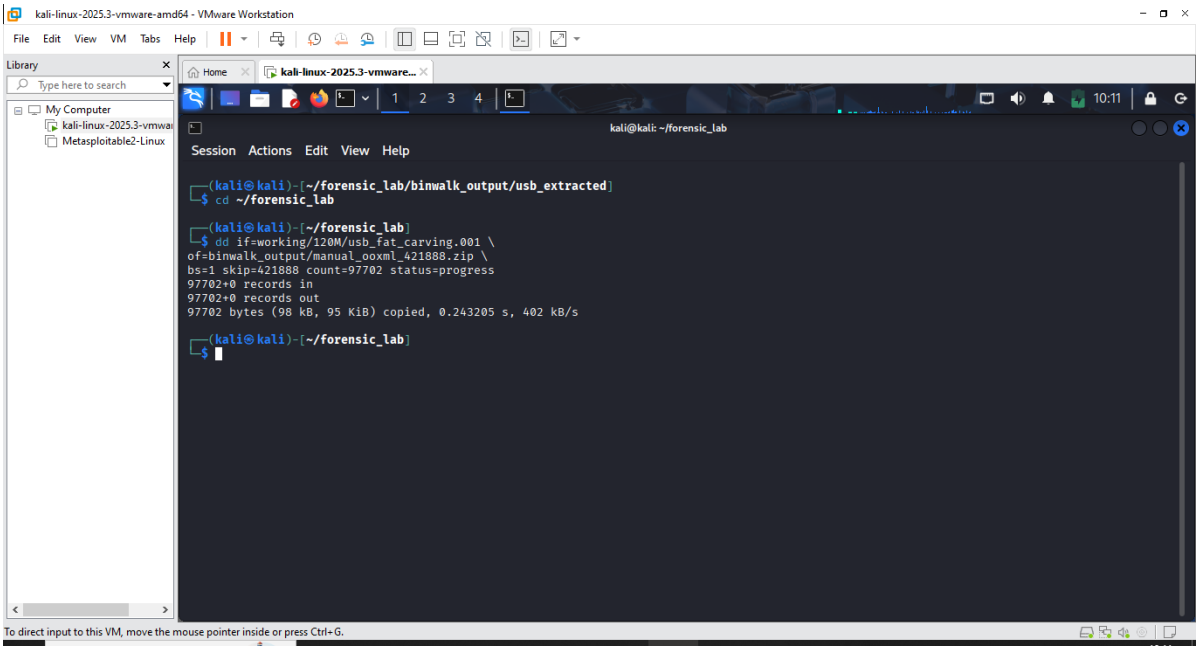

Fig 5.1 identify and extracting bytes.

Explanation

Using the start and end information identified through Binwalk, dd was used to extract exactly 97,702 bytes beginning at decimal offset 421888. This created the manually recovered OOXML candidate.

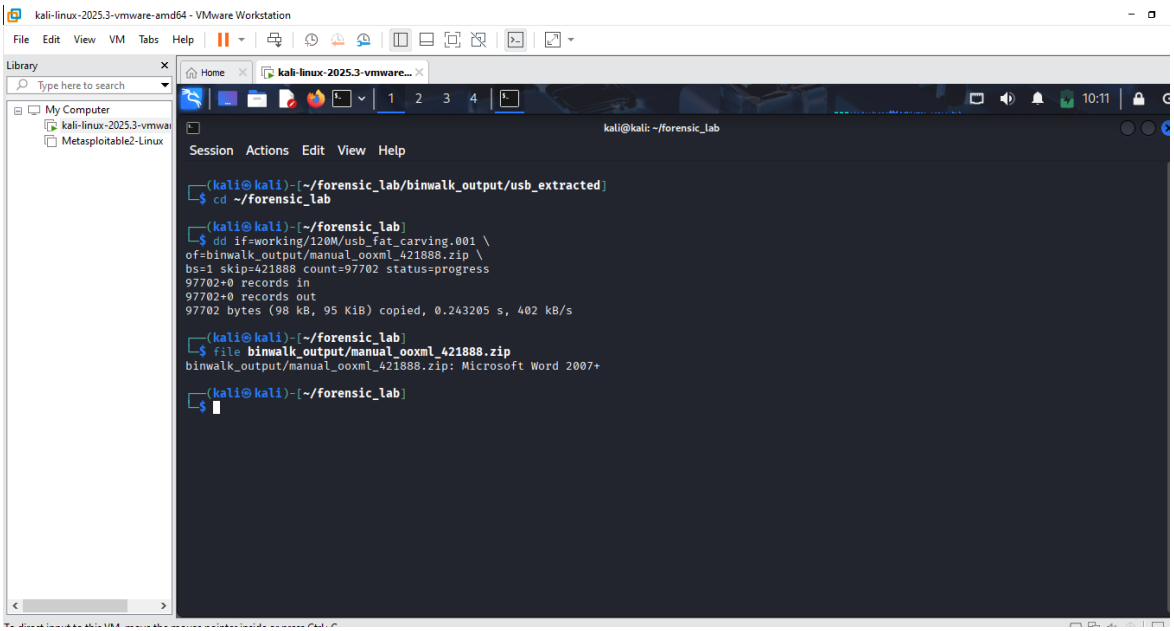

Fig 5.2 Testing the ZIP structure.

Explanation

The recovered object was examined with the file command and recognised as Microsoft Word 2007+. This provided an initial indication that the manually extracted data represented a valid Office Open XML document structure.

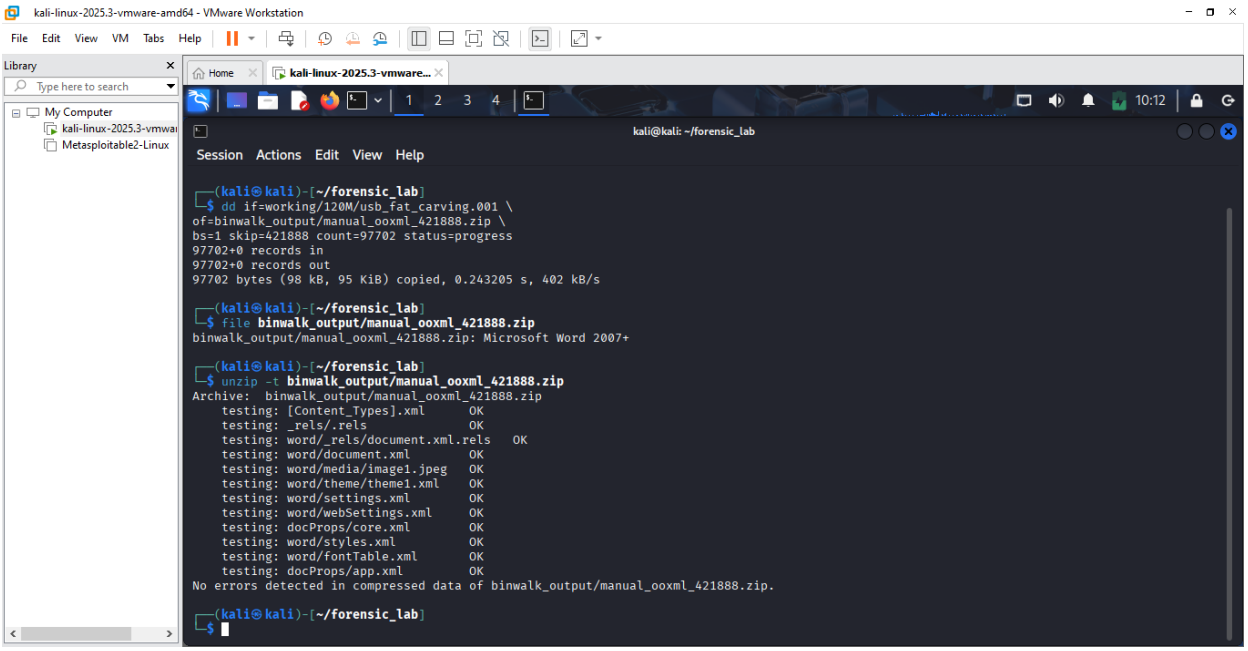

Fig 5.3 Inspecting its contents.

Explanation

unzip -t tested the manually recovered OOXML archive and reported no errors in the compressed data. This demonstrated that the extracted ZIP structure was internally valid.

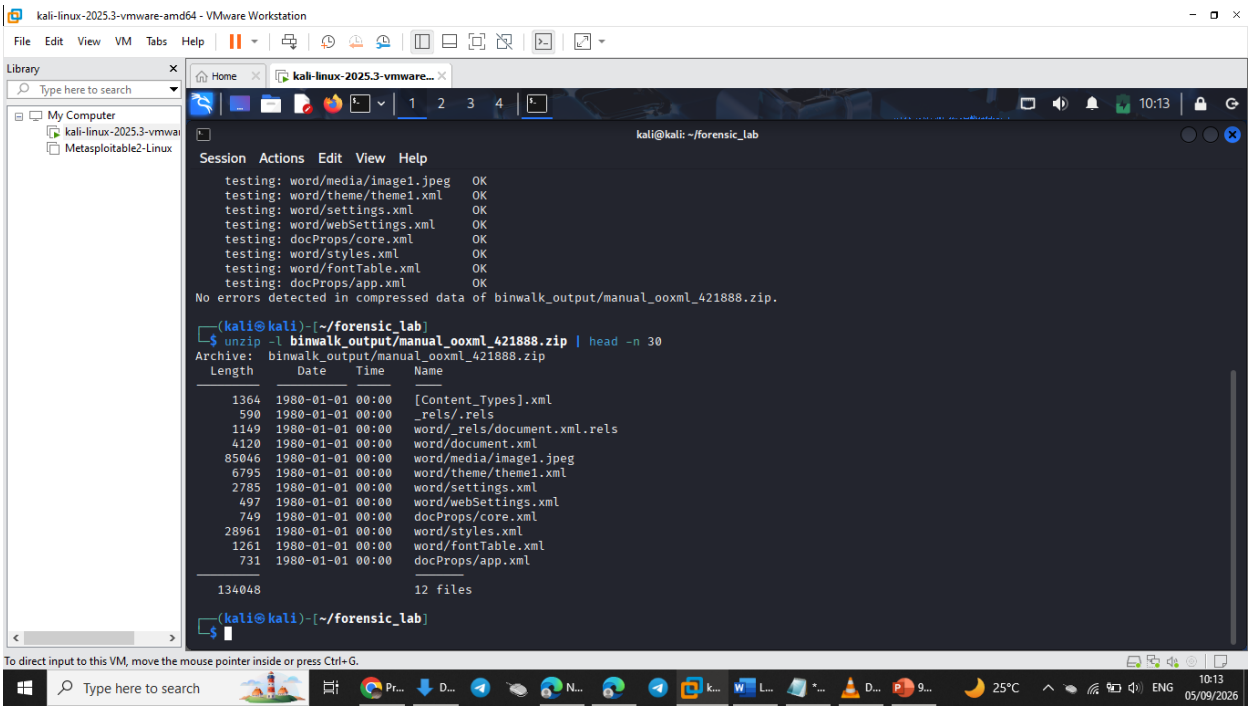

Fig 5.4 Inspecting its contents page 1.

Explanation

unzip -l displayed the files stored inside the recovered object, including [Content_Types].xml, word/document.xml, relationship files, document properties, styles, and word/media/image1.jpeg. These entries are consistent with the Word OOXML structure documented in the lab.

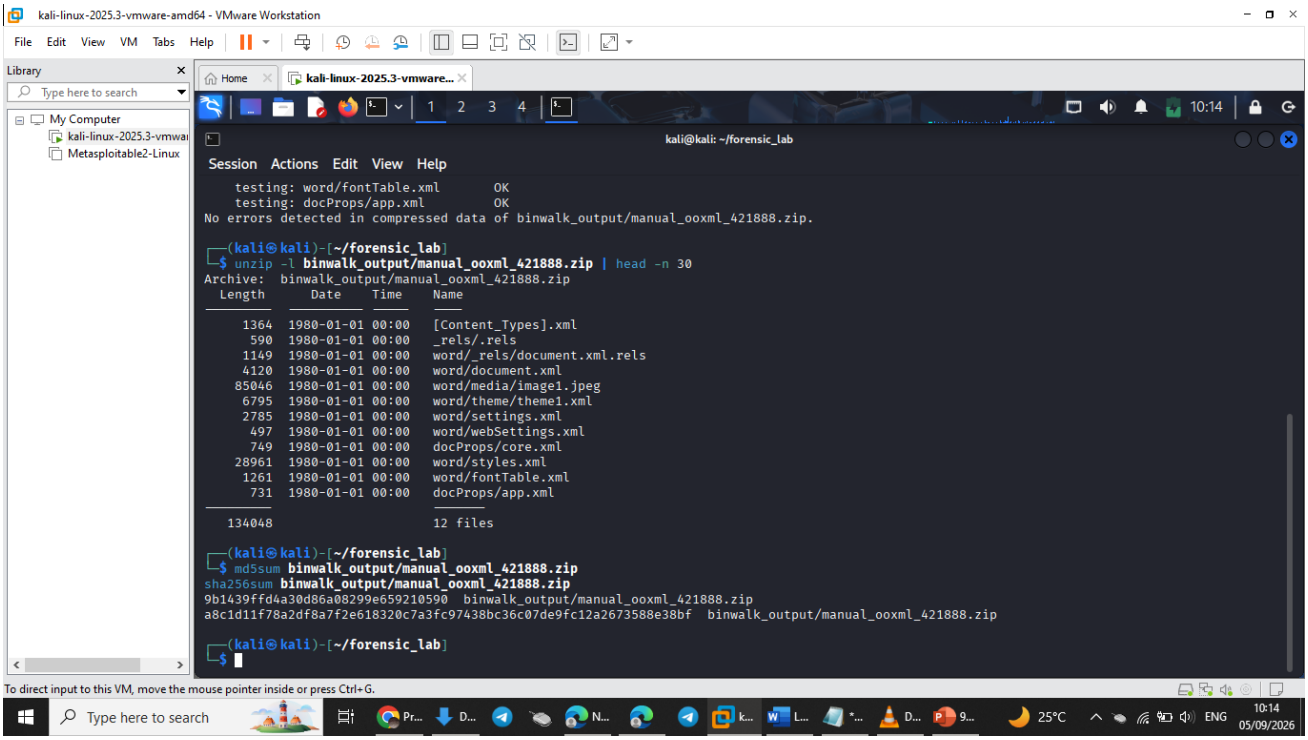

Fig 5.5 Hashing using md5sum and sha256sum.

Explanation

MD5 and SHA-256 hashes were generated for the manually extracted OOXML file. Recording these values preserved an integrity reference for the recovered object after successful validation.

The instructor-referenced File_carving.docx was not available in the supplied evidence. In accordance with the lab instructions, Binwalk practice was therefore completed using the supplied USB forensic image. Binwalk identified an Office Open XML ZIP structure beginning at decimal offset 421888 (0x67000). The object was manually extracted using dd. Validation with unzip -t reported no errors, while unzip -l confirmed a Microsoft Word OOXML structure containing word/document.xml, relationship files, document properties, styles and an embedded JPEG. Cryptographic hashes were recorded for the recovered object.

Part F - Carve Deleted Files with Scalpel

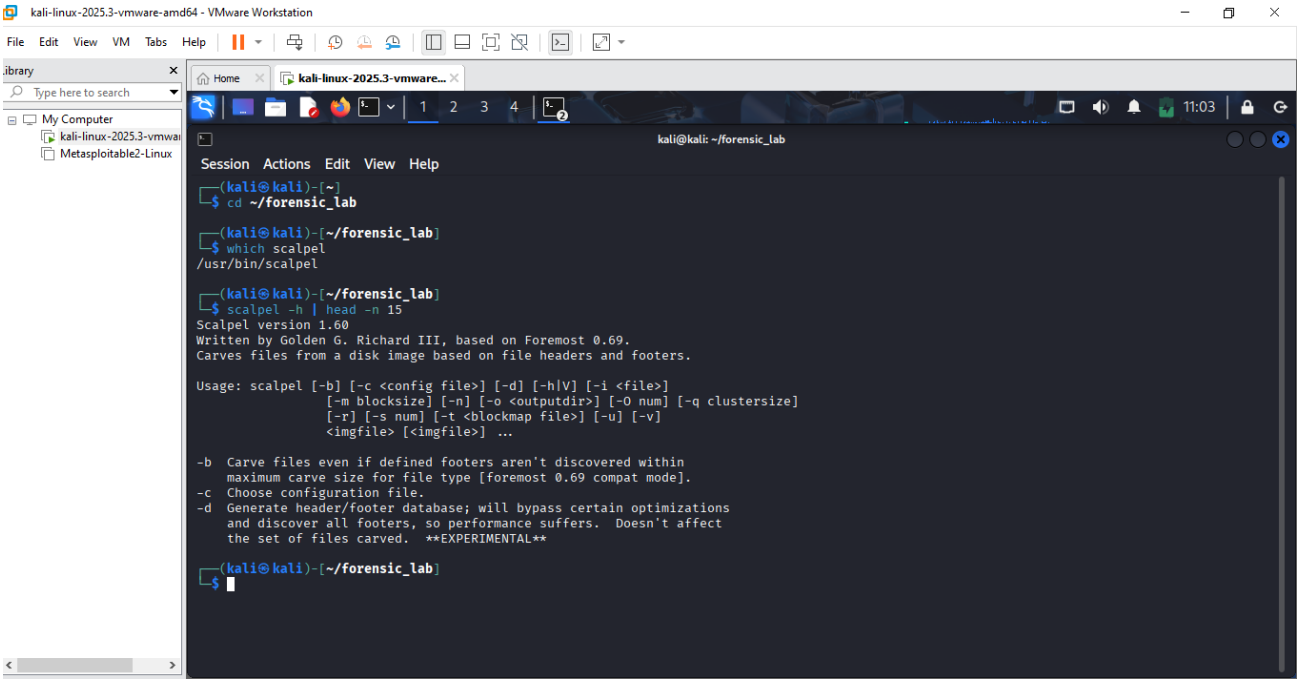

Fig 5.6 Checking if Scalpel is installed.

Explanation

The system was checked for the Scalpel executable, and the help information confirmed that Scalpel was available for the carving stage of the laboratory.

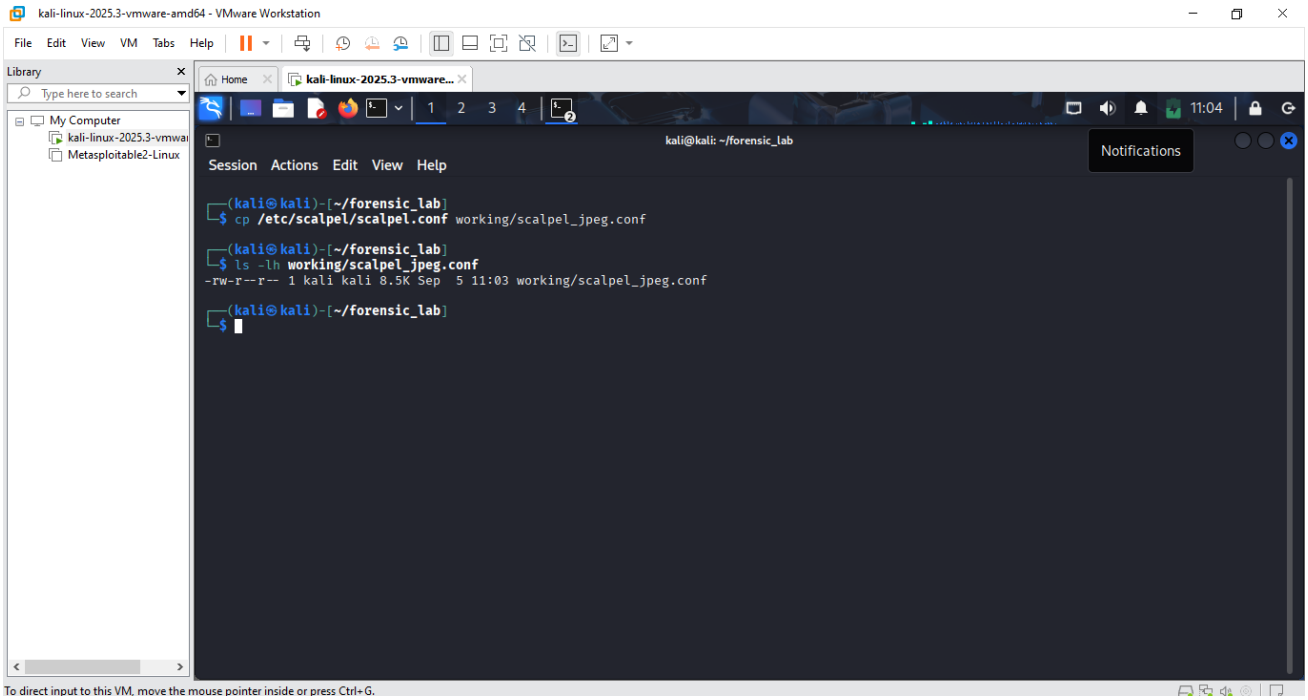

Fig 5.7 Making a working copy of the Scalpel configuration.

Explanation

The system Scalpel configuration was copied into the working directory. This allowed the carving rules to be adjusted without altering the original system configuration file.

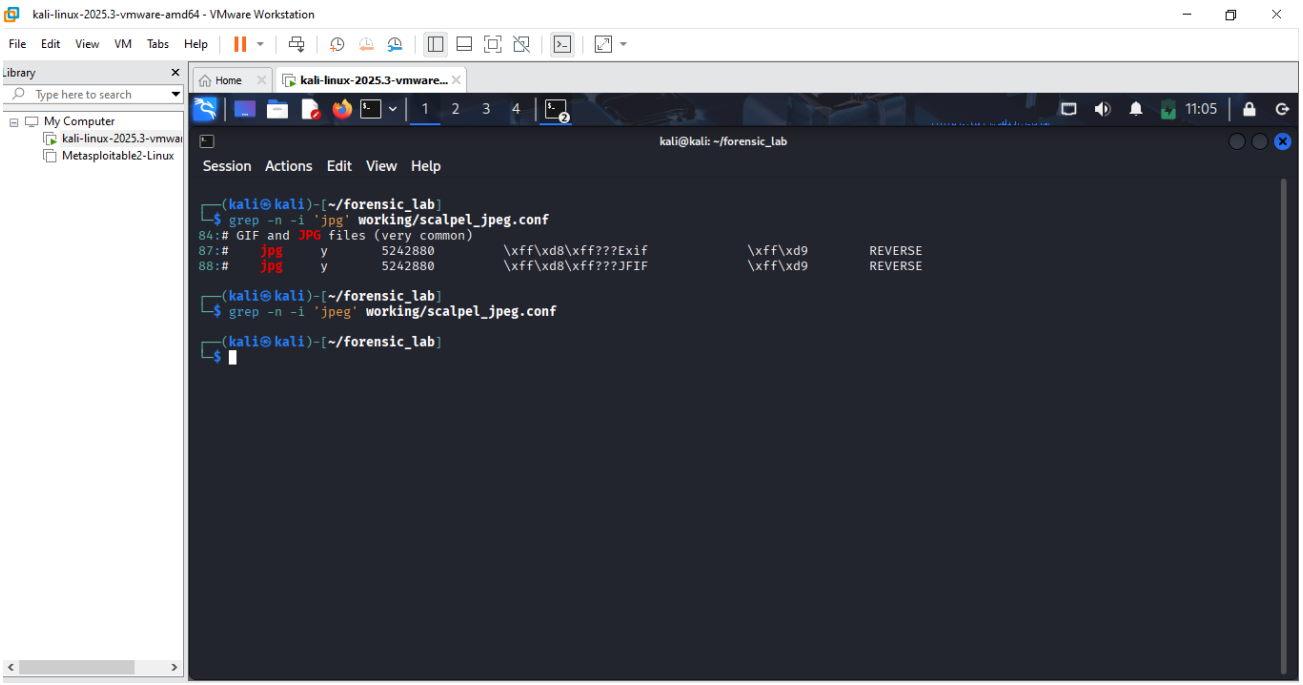

Fig 5.8 Locating all JPEG-related configuration.

Explanation

The configuration was searched for JPEG rules, revealing separate EXIF- and JFIF-based signatures. At this point, the entries were visible and ready to be enabled for the required carving exercise.

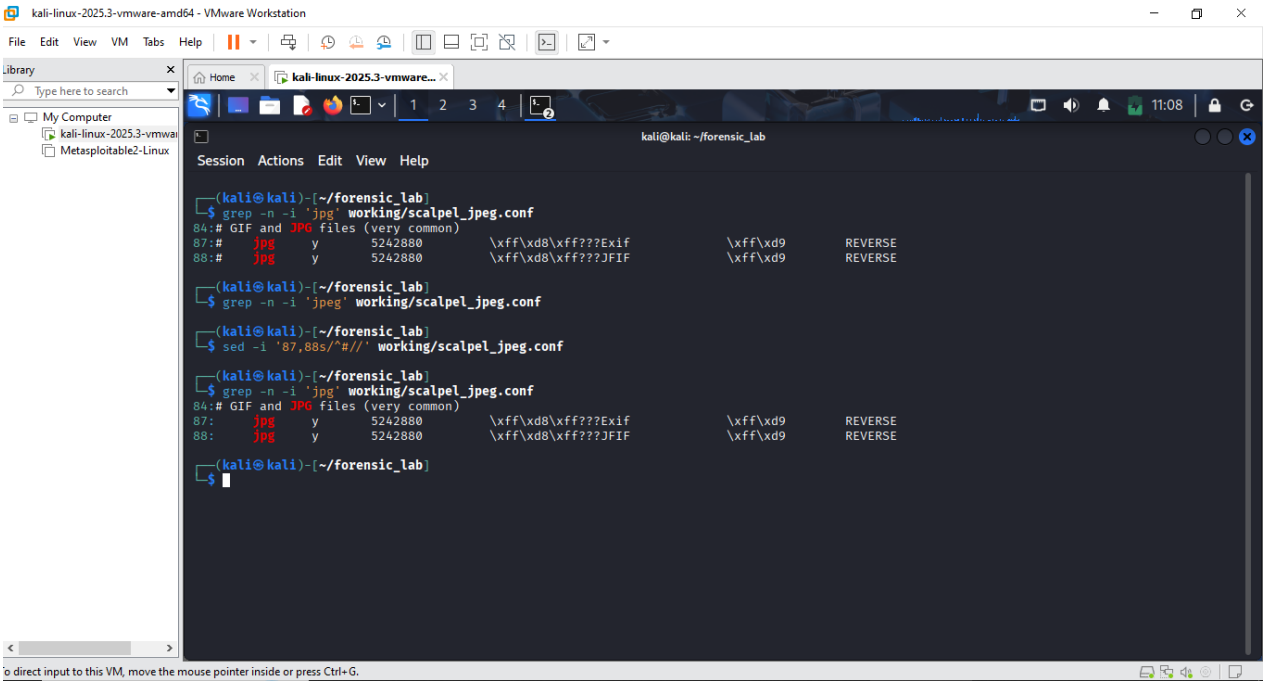

Fig 5.9 Evidence that the JPEG/JPG types were enabled in the Scalpel configuration.

Explanation

The relevant JPEG configuration lines were uncommented, enabling both EXIF- and JFIF-based JPEG carving. A second search confirmed that the rules were now active.

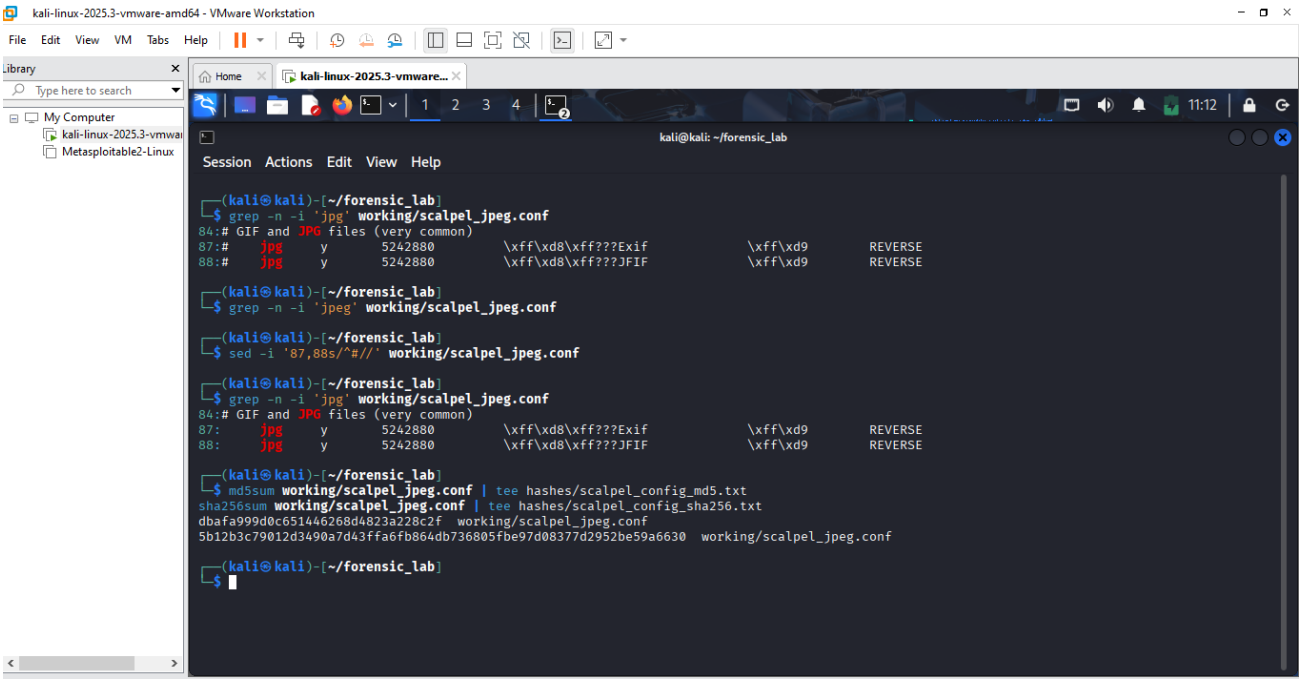

Fig 6.0 Preserving the configured file's hash.

Explanation

MD5 and SHA-256 hashes were calculated for the modified Scalpel configuration. This created a record of the exact configuration used to produce the carving results.

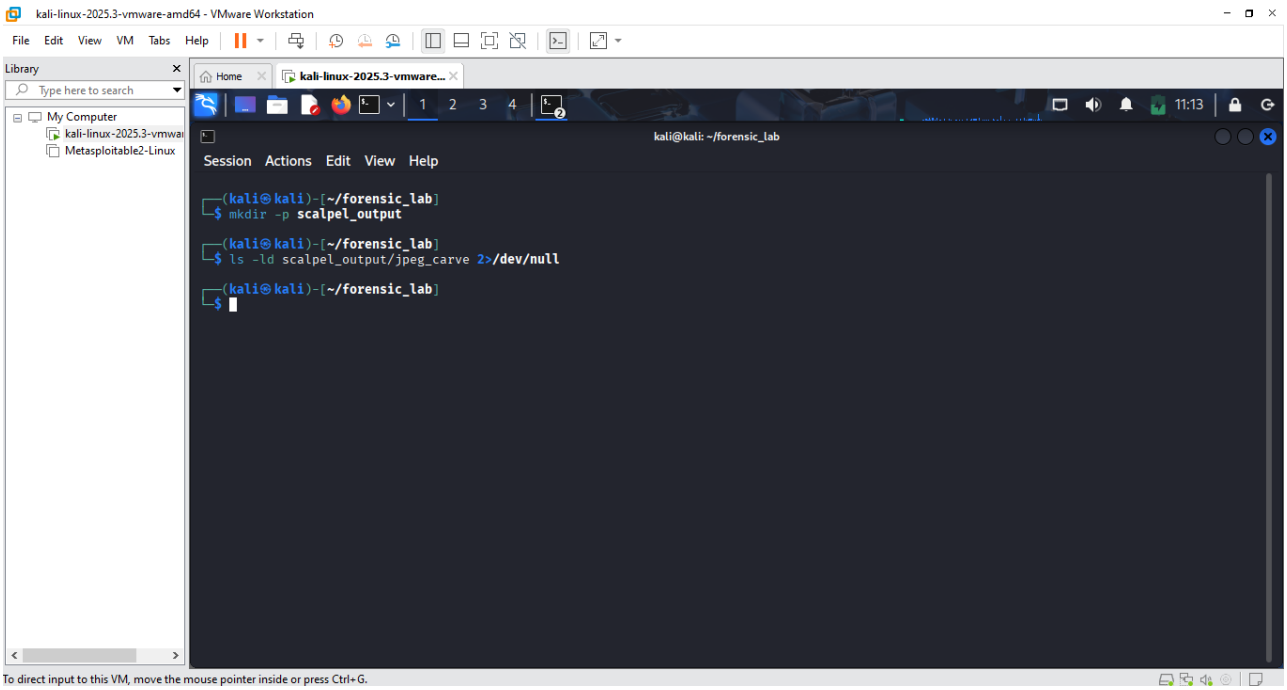

Fig 6.1 Making sure the parent output directory exists.

Explanation

The output directory for Scalpel results was created and checked before carving began. This ensured that the recovered files and audit information would be stored in a dedicated location.

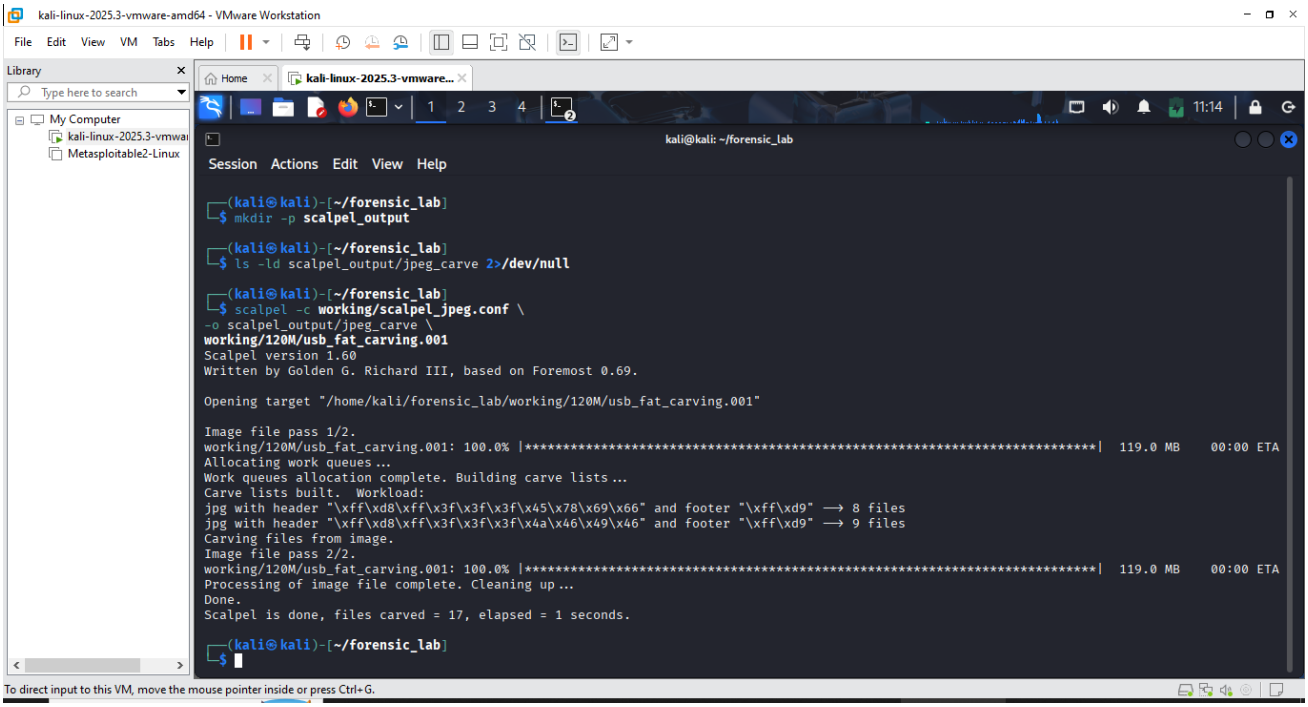

Fig 6.2 Scaple carving.

Explanation

Scalpel was run against usb_fat_carving.001 using the custom JPEG configuration. The two-pass process completed successfully and reported a total of 17 carved files.

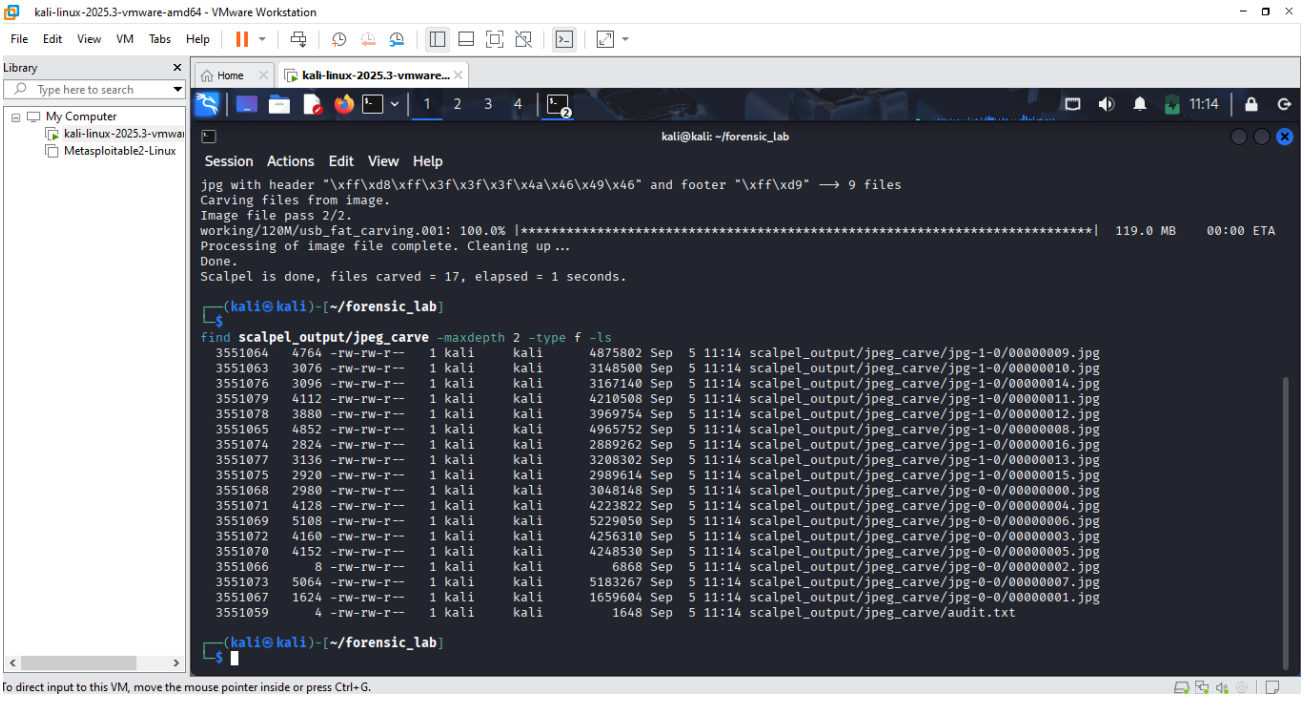

Fig 6.3 Scaple carving for jpeg.

Explanation

The carved JPEG files were listed after the operation completed. The output shows files recovered into separate JPEG directories together with audit.txt.

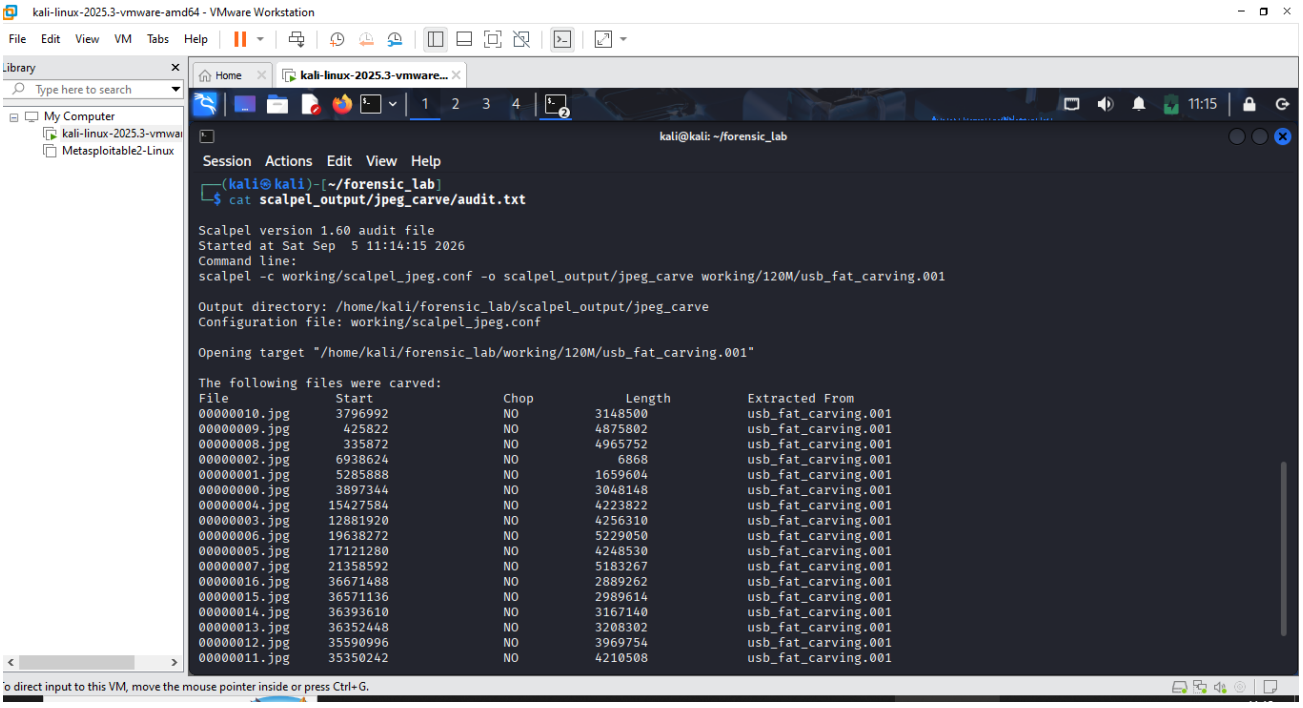

Fig 6.4 Locating the audit file.

Explanation

Audit.txt was opened to review the results generated by Scalpel. The audit information recorded each carved filename, start offset, file length, chop status, and the source forensic image.

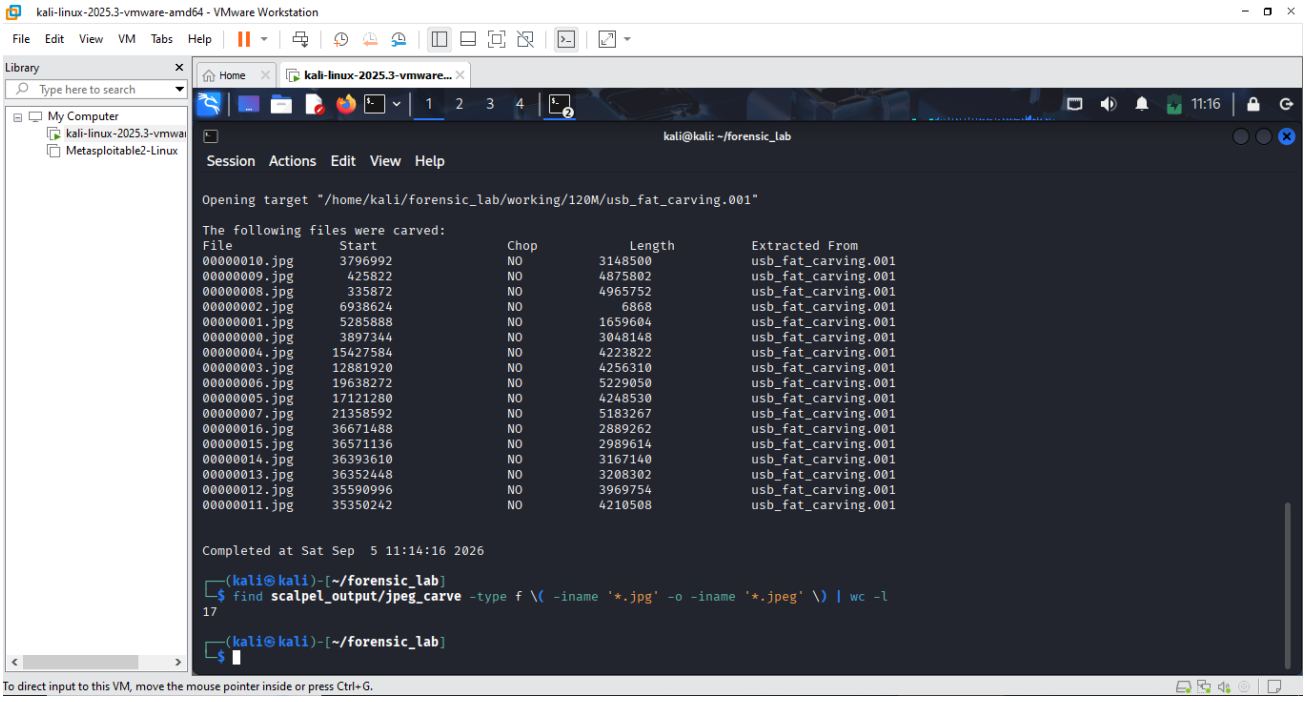

Fig 6.5 counting the carved JPEGs.

Explanation

A find command was used to count files with .jpg or .jpeg extensions in the carving output. The result confirmed that 17 JPEG files had been recovered.

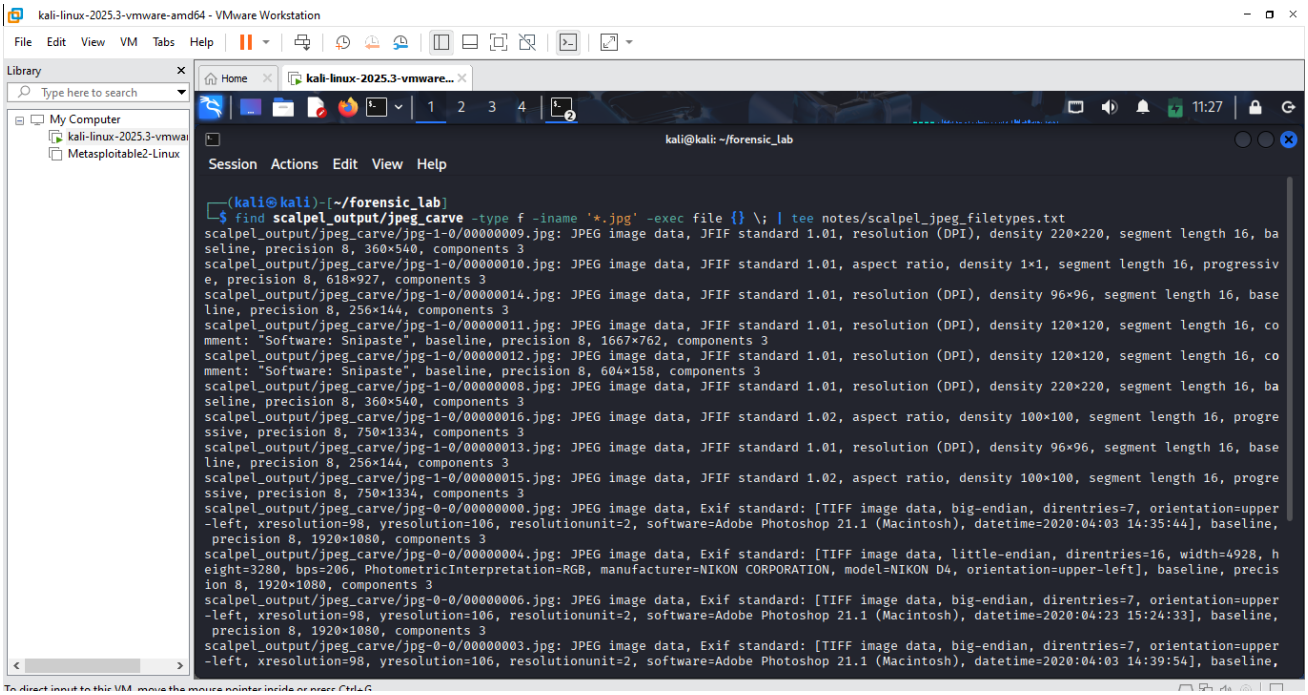

Fig 6.6 Verify the recovered JPEGs.

Explanation

The file utility was run against the recovered images to check their recognized formats and image characteristics. The output confirmed that the carved items were being identified as JPEG image data.

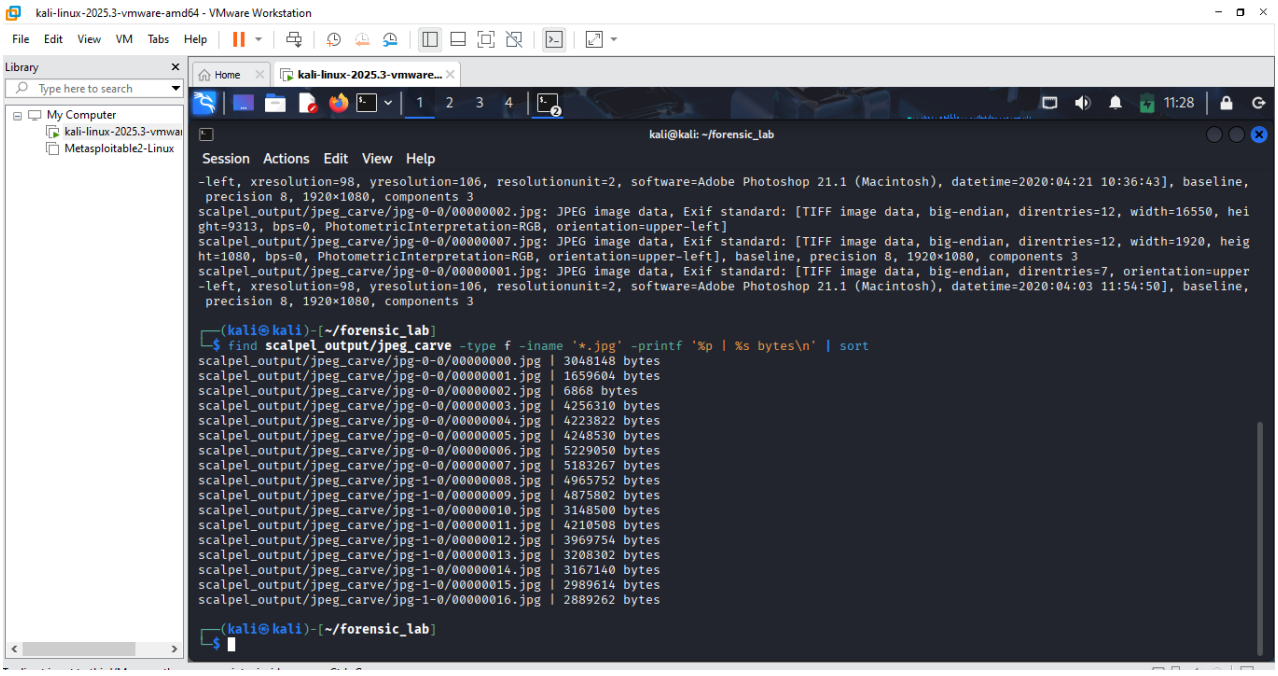

Fig 6.7 Verify the recovered JPEGs page 1.

Explanation

The carved JPEGs were listed together with their file sizes. This provided a simple inventory showing that the recovered images varied in size rather than all being identical artefacts.

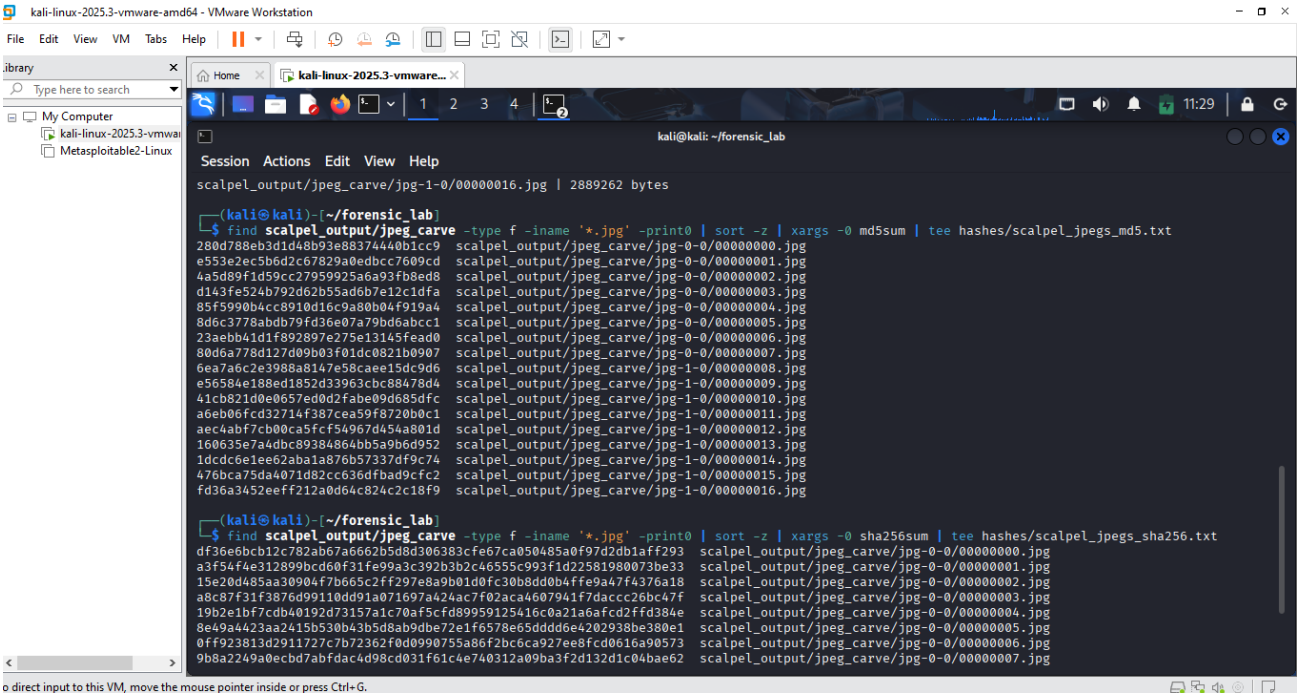

Fig 6.8 Hash all 17 carved JPEGs using md5sum and sha256sum.

Explanation

MD5 hashes were calculated for the complete set of carved JPEG files and saved to a hash record. SHA-256 processing was also started so that stronger integrity values would be retained.

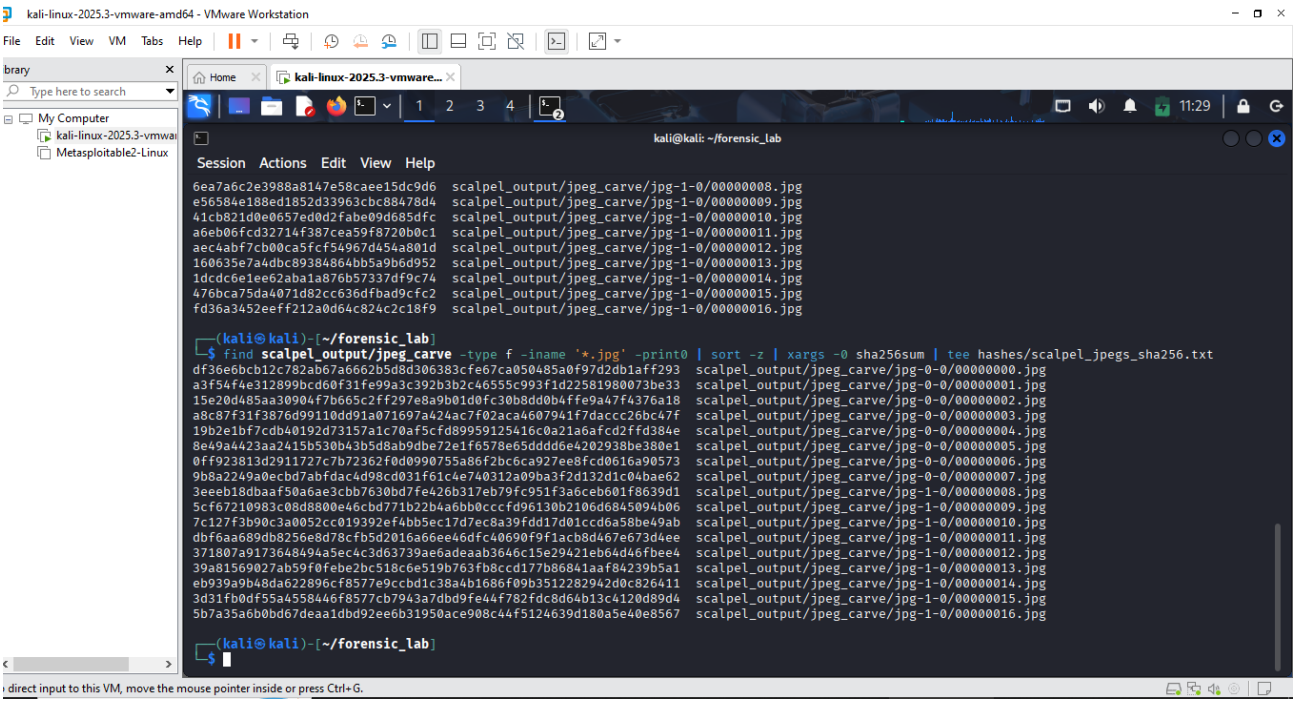

Fig 6.9 Hash all 17 carved JPEGs using md5sum and sha256sum page 1.

Explanation

This screenshot completes the SHA-256 hash output for the 17 recovered JPEGs. The stored hash records provide individual integrity references for each carved image.

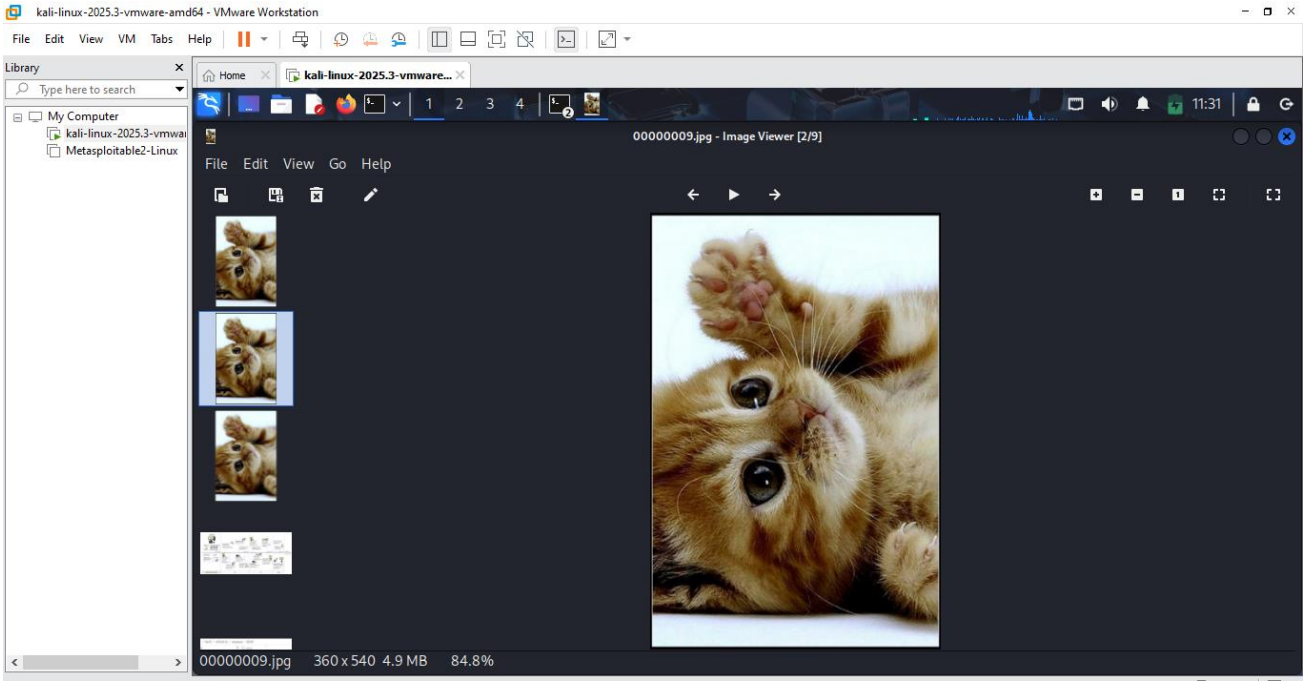

Fig 7.0 Opening recovered JPEGs 00000009.

Explanation

The first selected carved JPEG was successfully opened in the Kali image viewer and displayed a kitten image. Its successful rendering confirmed that Scalpel had recovered usable JPEG content rather than only a signature fragment.

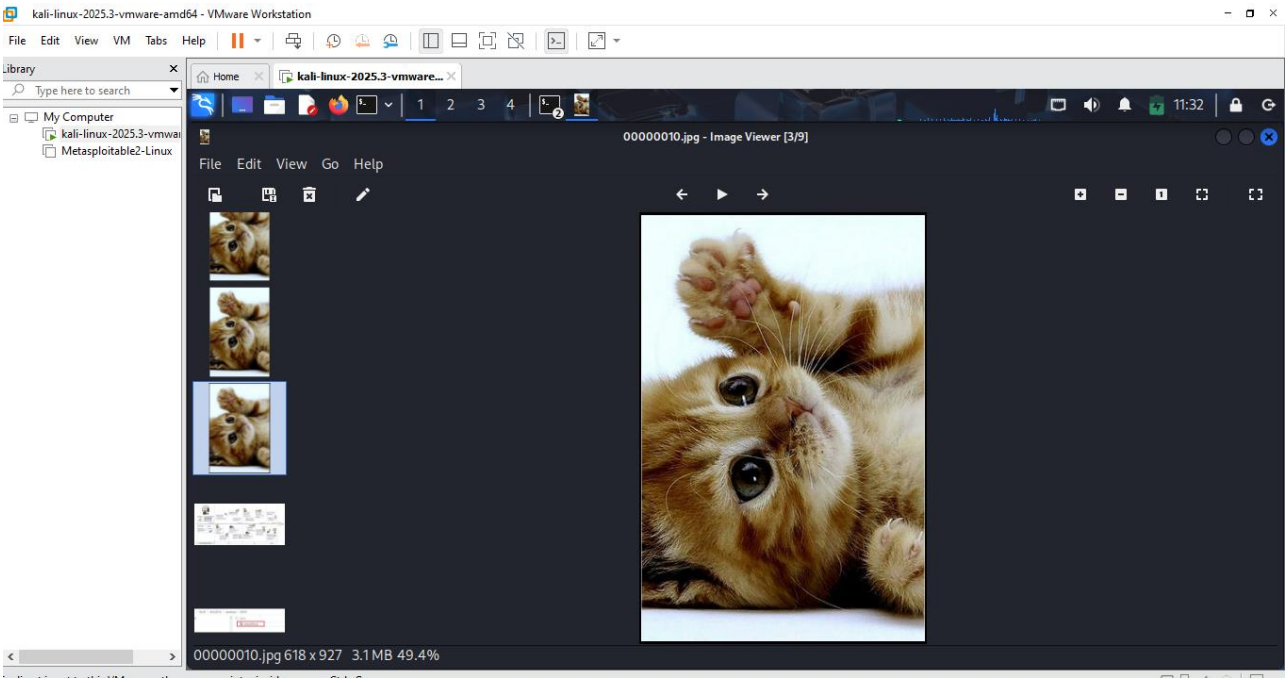

Fig 7.1 Opening recovered JPEGs 00000010.

Explanation

A second recovered JPEG was opened successfully and also displayed correctly in the image viewer. This provided another visible confirmation that the carving process had produced recoverable image evidence.

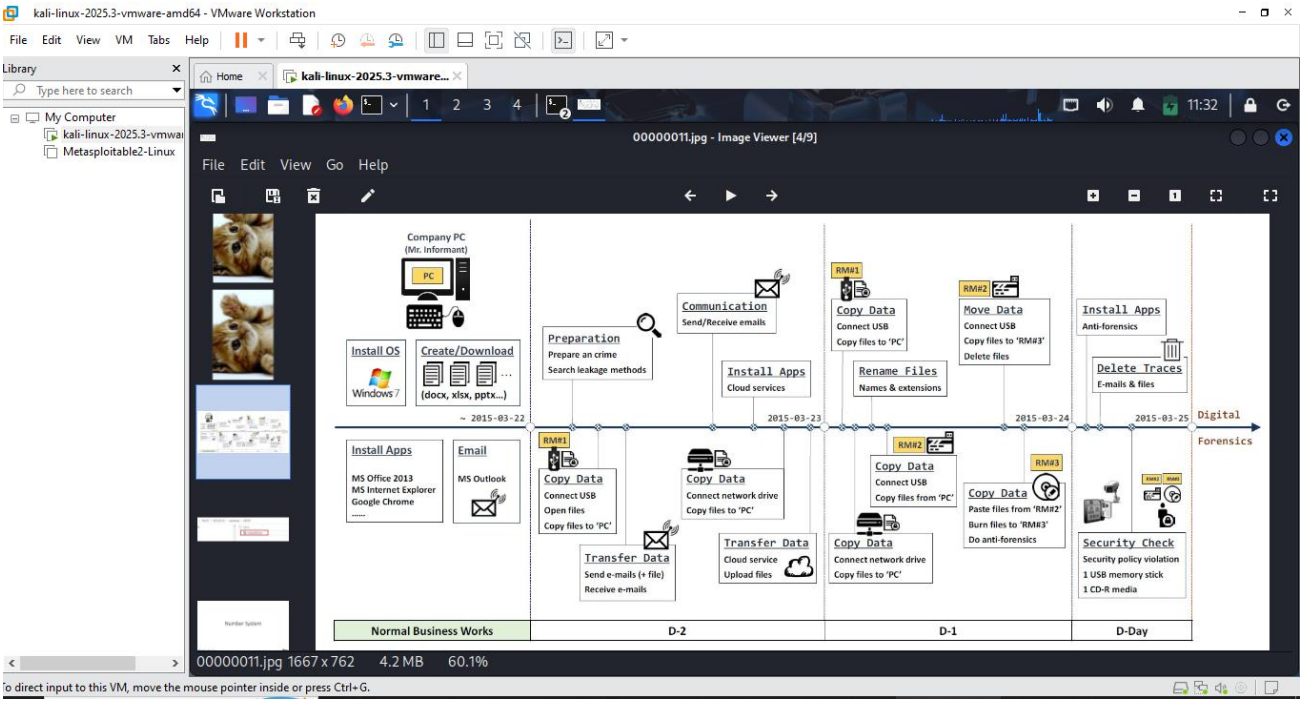

Fig 7.2 Opening recovered JPEGs 00000011.

The third example opened successfully and displayed a digital-forensics workflow diagram. Together, the three opened files demonstrated that several carved JPEG artefacts were usable after recovery.

Explanation

Scalpel was configured to carve both EXIF- and JFIF-based JPEG files. The carving process completed successfully and recovered 17 JPEG artefacts from the USB forensic image. audit.txt documented the start offset, file length and extraction source for each carved artefact. MD5 and SHA-256 hashes were generated for all recovered JPEG files. Multiple carved images were successfully opened using the Kali image viewer, including 00000009.jpg, 00000010.jpg and 00000011.jpg, confirming that usable image data had been recovered.

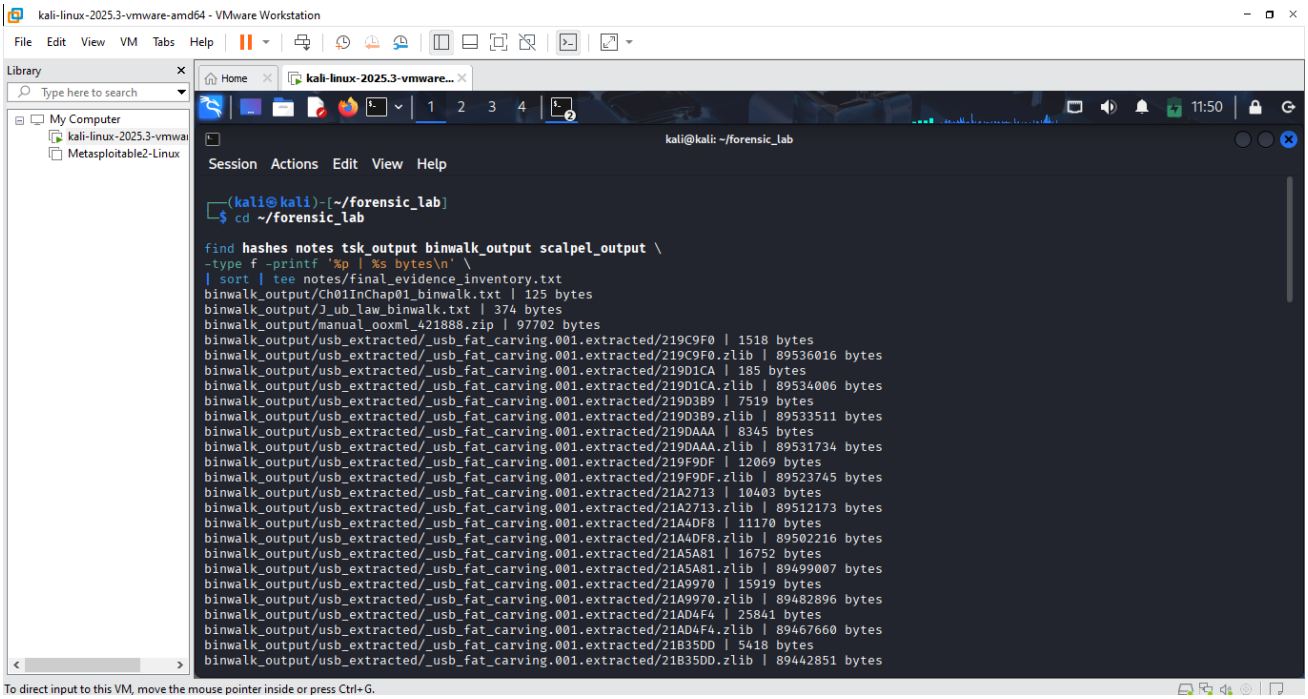

Fig 7.3 Evidence packaging.

Explanation

An inventory was generated for the hashes, notes, TSK output, Binwalk output, and Scalpel results. This began the process of documenting exactly which supporting files had been generated during the investigation.

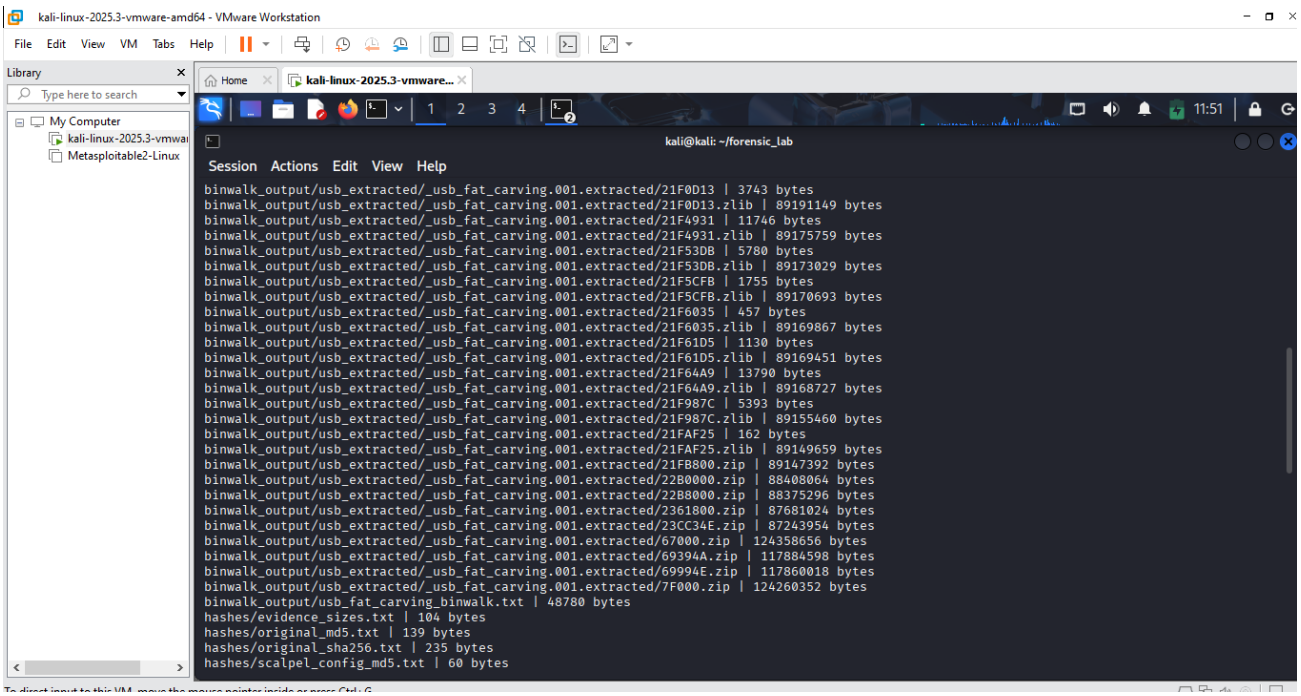

Fig 7.4 Evidence packaging page 1.

Explanation

The evidence inventory continued through the large Binwalk extraction output and other generated files. File

paths and sizes were recorded so that the contents of the laboratory workspace could be reviewed.

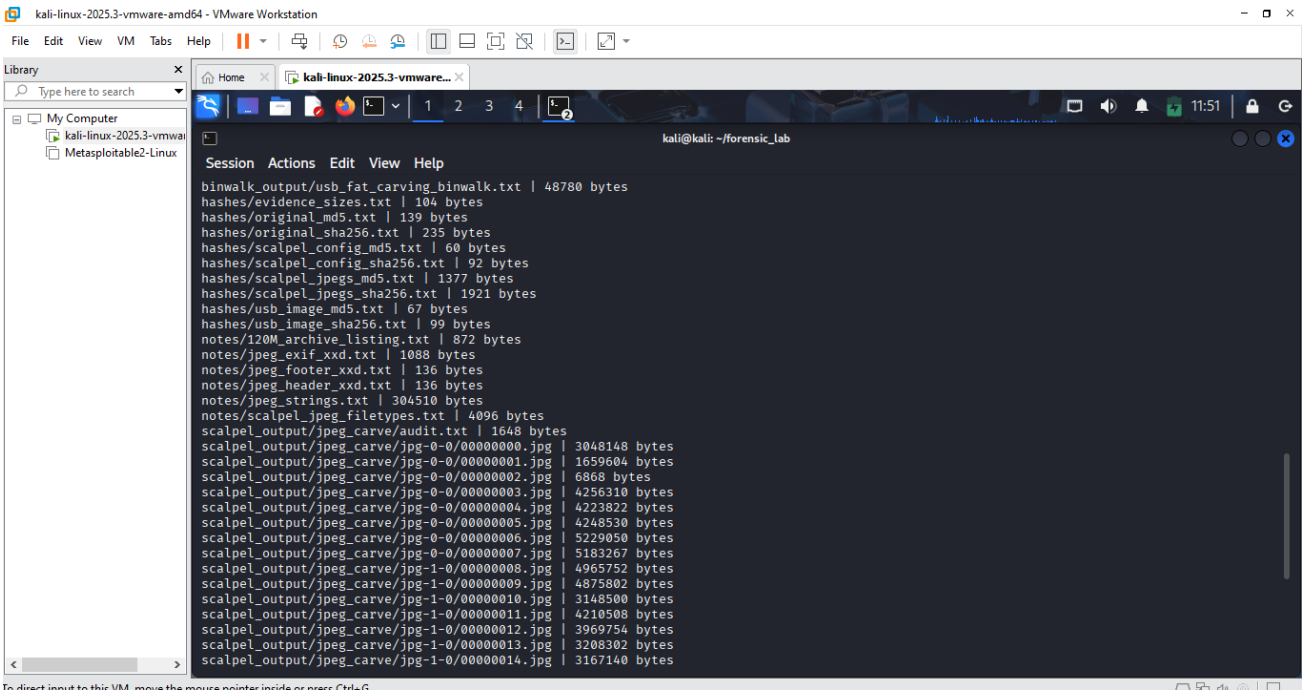

Fig 7.5 Evidence packaging page 2.

Explanation

This screenshot continues the inventory and includes hash files, notes, Scalpel output, and other supporting material. It demonstrates that the recovered and generated evidence was being catalogued before packaging.

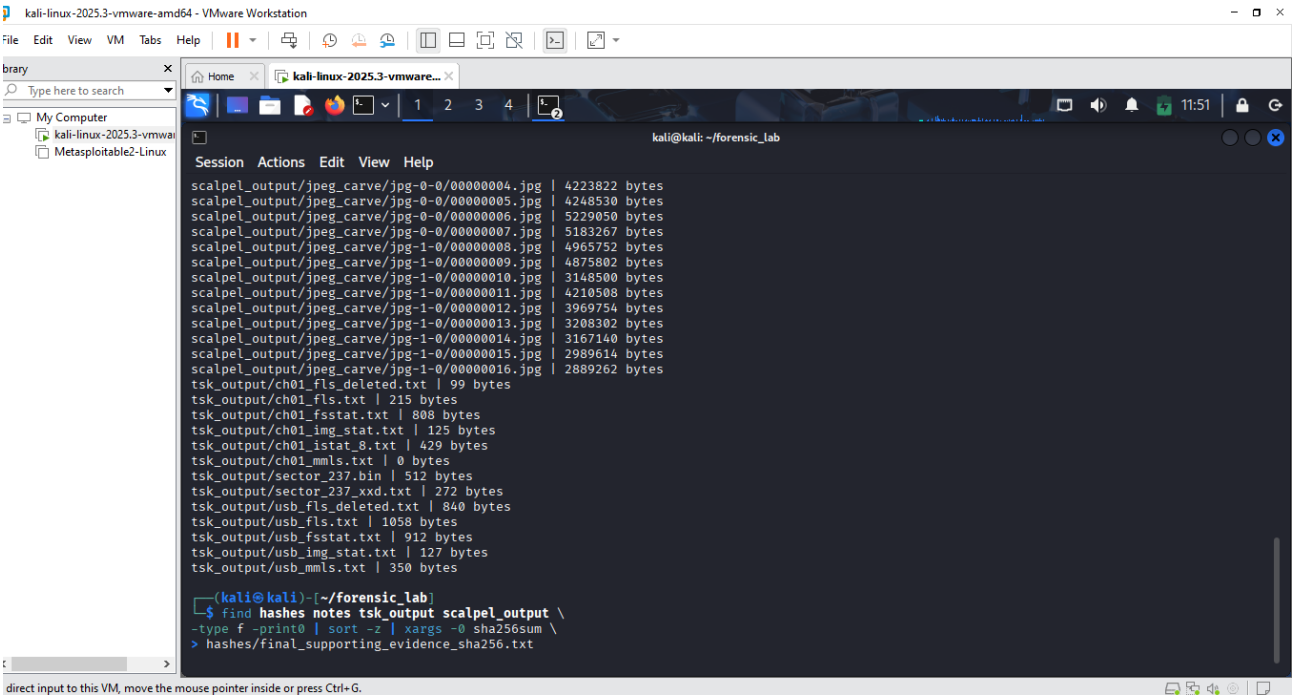

Fig 7.6 Evidence packaging page 3.

Explanation

The inventory reaches the TSK output files and additional supporting records. A SHA-256 manifest command was then started to generate integrity values for the collected supporting evidence.

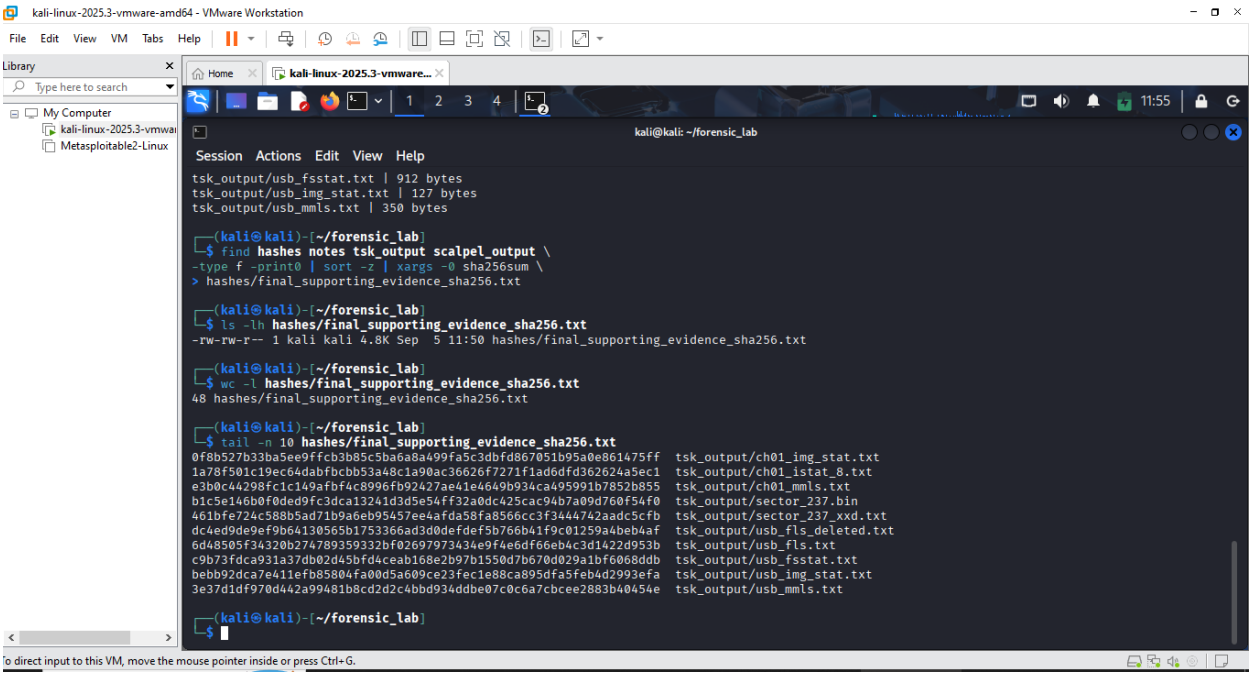

Fig 7.7 Evidence packaging page 4.

Explanation

The completed supporting-evidence SHA-256 file was checked by displaying its size, number of entries, and final records. This provided confirmation that the supporting material had been included in a cryptographic manifest.

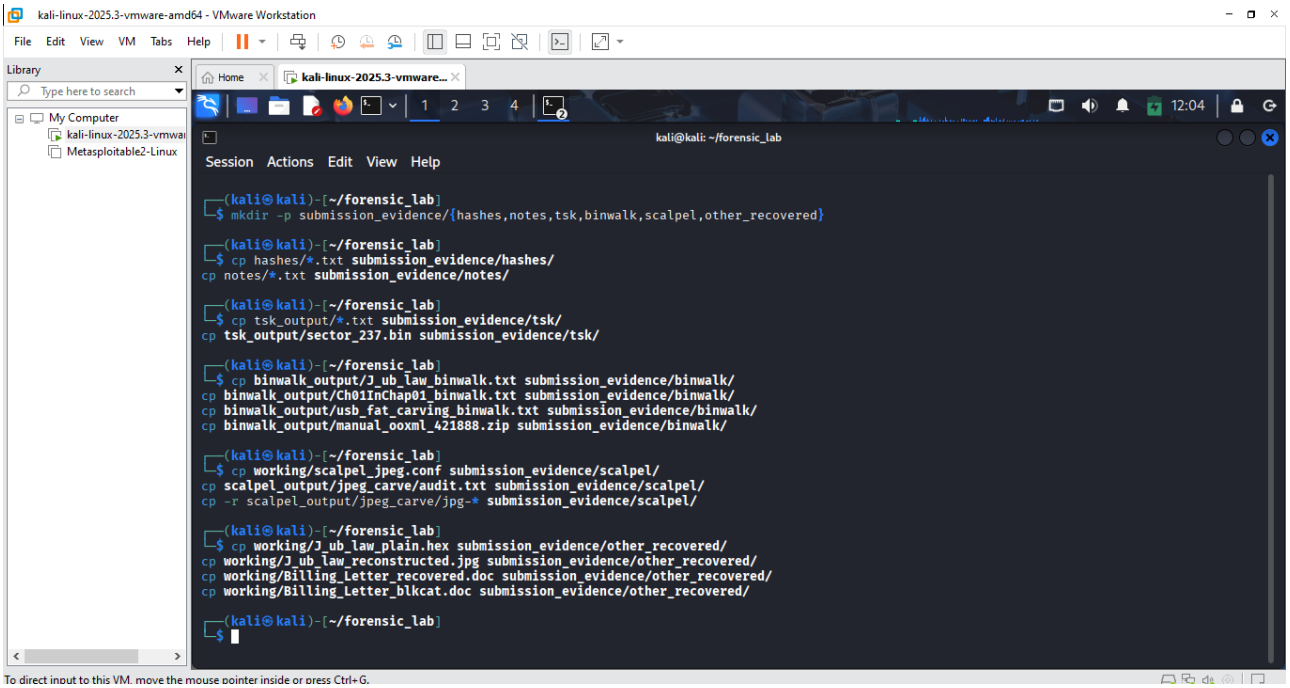

Fig 7.8 Creating a clean submission-evidence folder.

Explanation

A dedicated submission_evidence structure was created and populated with selected hashes, notes, TSK output, Binwalk results, Scalpel evidence, and recovered files. This separated the material intended for submission from unnecessary temporary or automatically generated data.

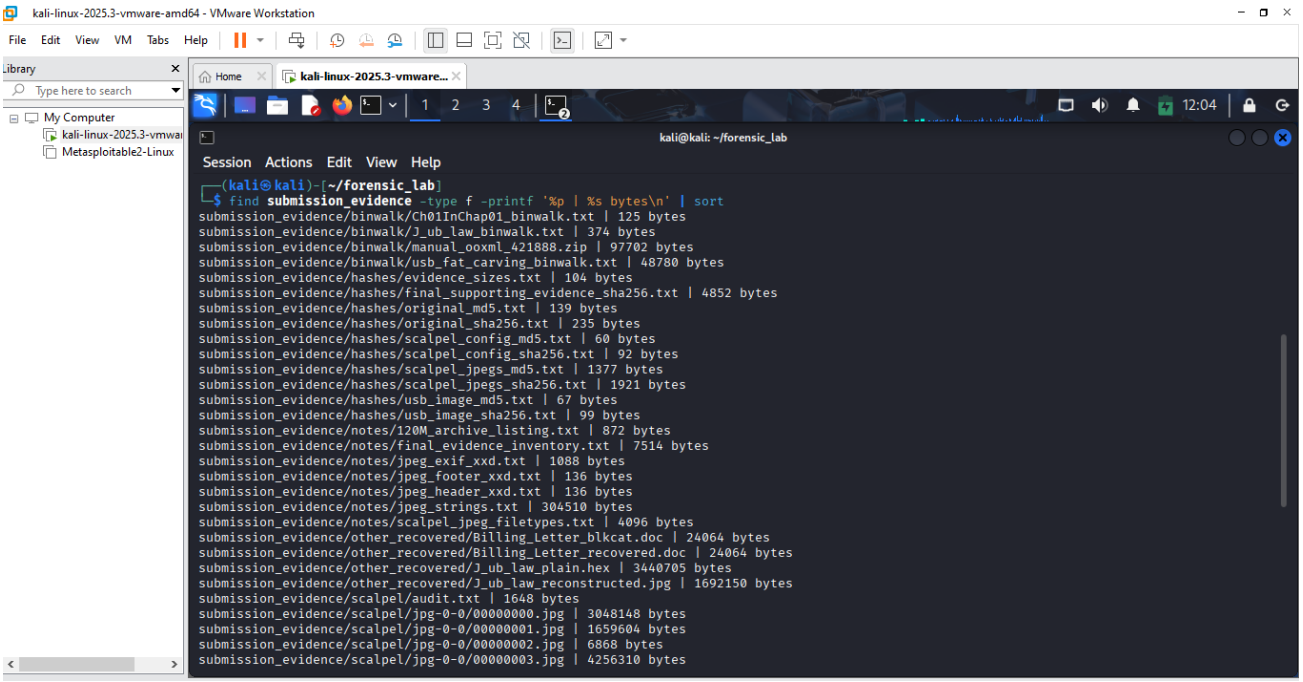

Fig 7.9 Final package.

Explanation

The contents of the clean submission folder were listed with their paths and sizes. This provided a final review of the evidence selected for inclusion in the supporting archive.

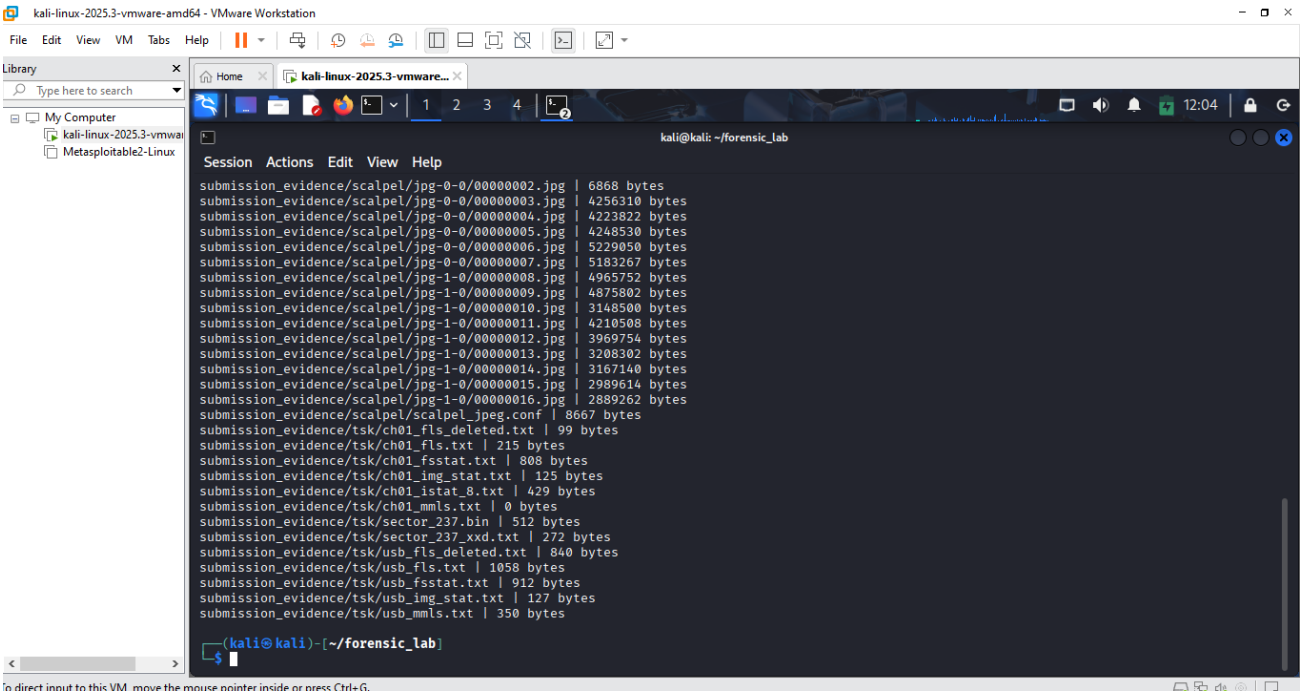

Fig 8.0 Final package page 1.

Explanation

The package listing continued through the carved JPEG files and TSK outputs. It confirmed that recovered images, audit records, and forensic command outputs had been included.

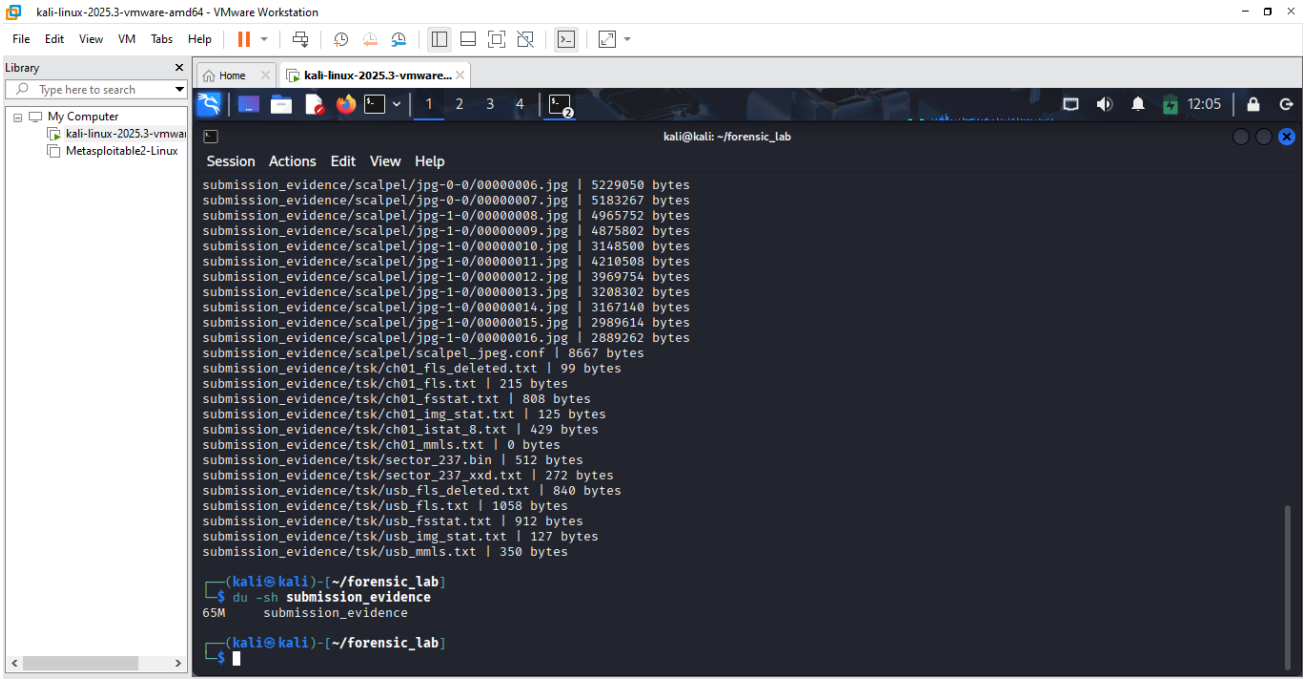

Fig 8.1 Final package page 2.

Explanation

The remainder of the final evidence directory was displayed and its overall size was checked. This completed the inventory stage before the final archive was created.

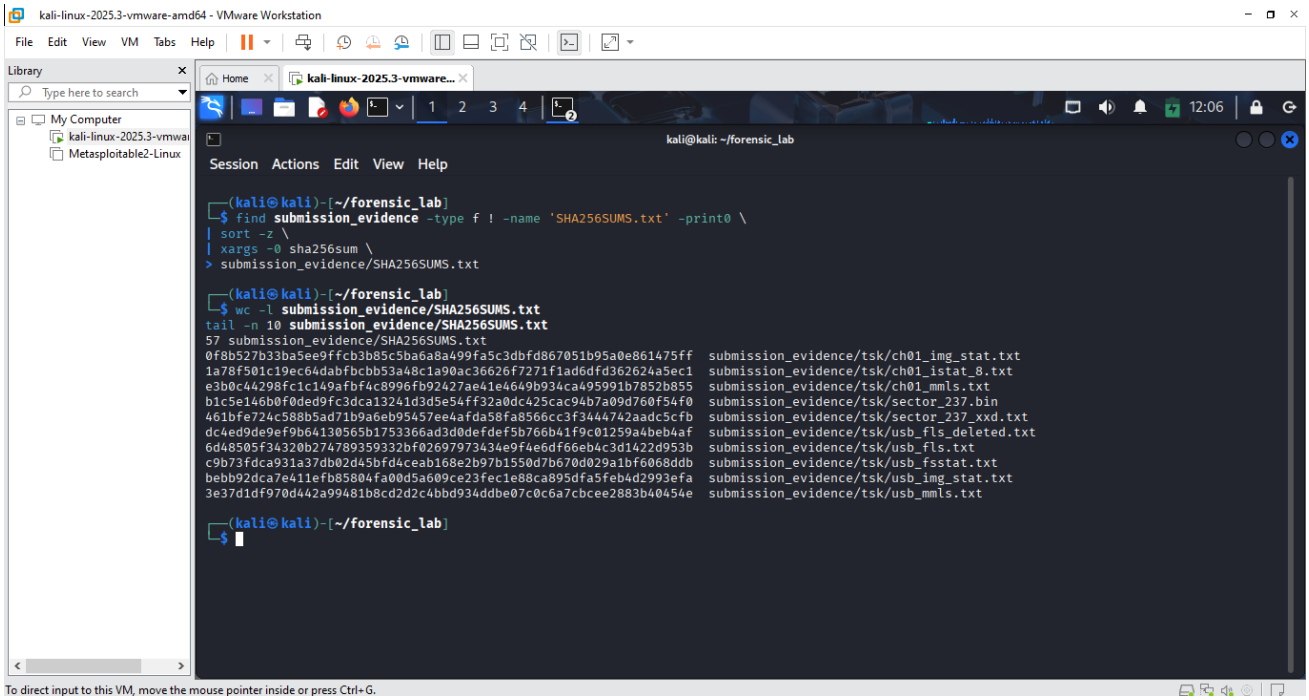

Fig 8.2 Hashing the submission_evidence.

Explanation

A SHA-256 manifest was generated specifically for the files inside the final submission folder. The displayed entries show that the packaged supporting evidence received individual integrity hashes before compression.

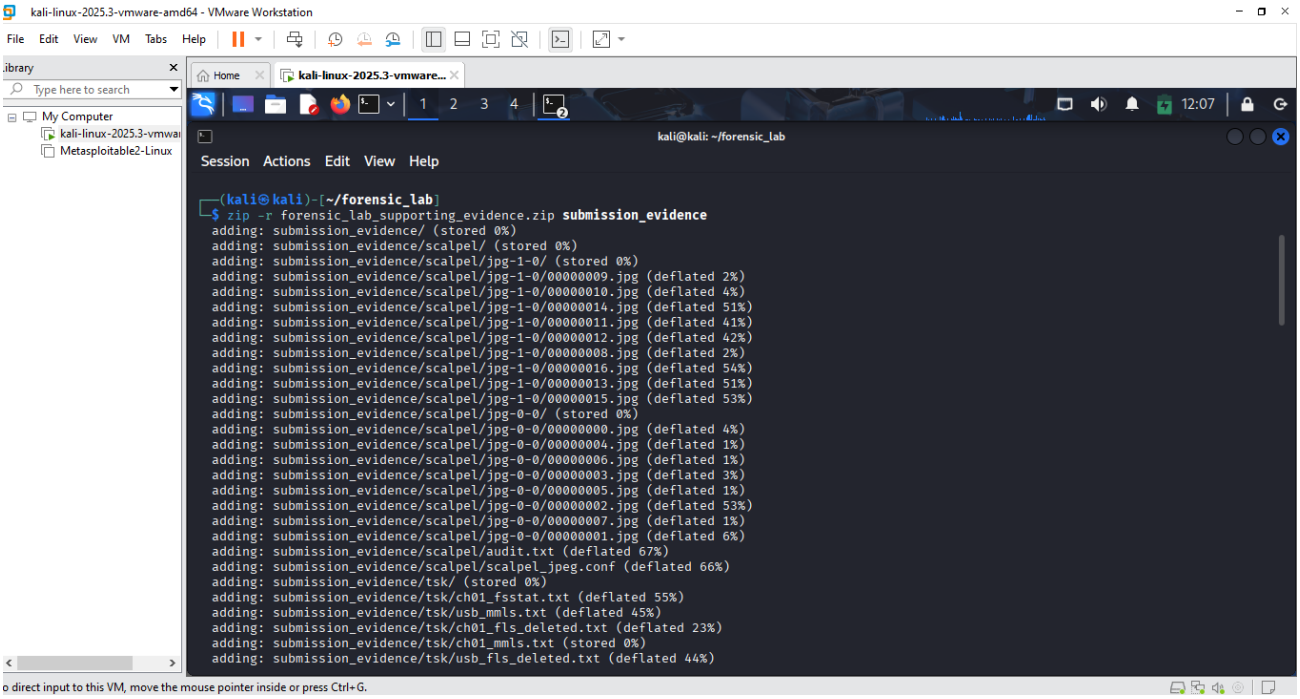

Fig 8.3 Creating the required evidence ZIP.

Explanation

The clean submission_evidence directory was compressed into the required supporting-evidence ZIP archive. The command output shows the individual files being added to the archive.

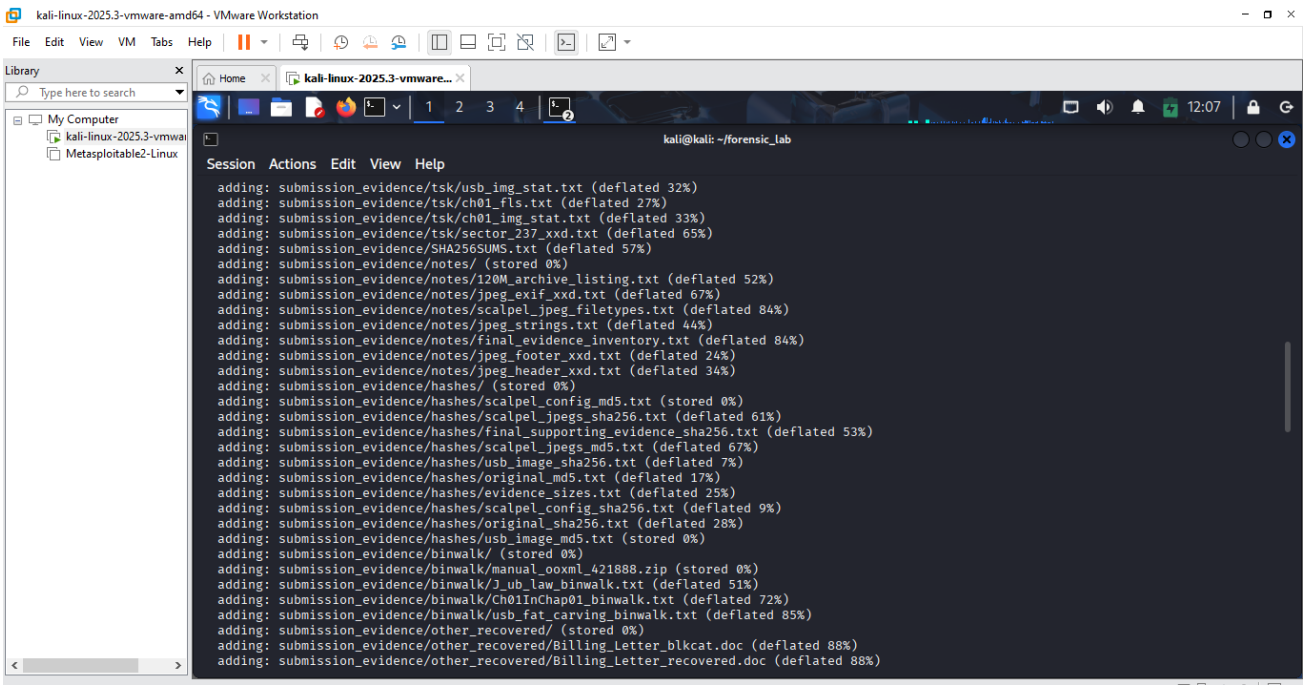

Fig 8.4 Creating the required evidence ZIP page 1.

Explanation

This screenshot continues the compression process as additional notes, hashes, Binwalk records, and recovered evidence were added to the ZIP. It documents the contents included in the final submission archive.

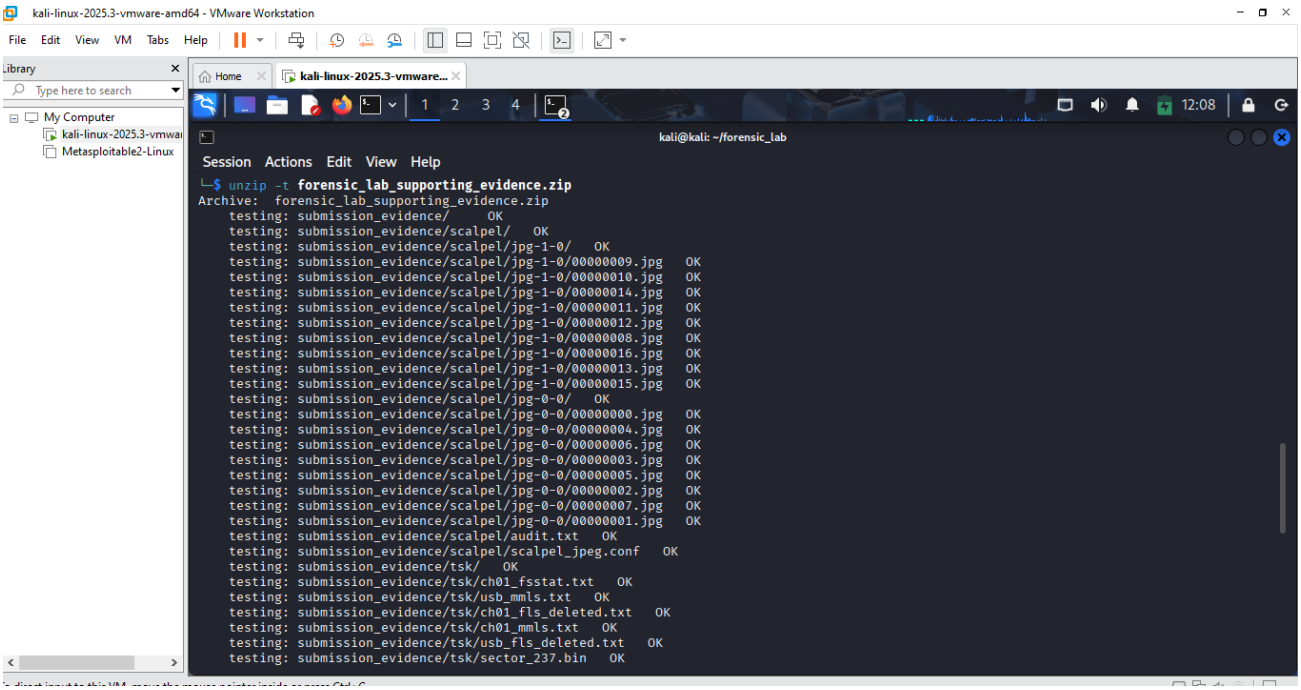

Fig 8.5 Creating the required evidence ZIP page 2.

Explanation

The finished ZIP archive was tested using unzip -t. The displayed entries were reported as OK, showing that the archived supporting files could be read successfully without detected compression errors.

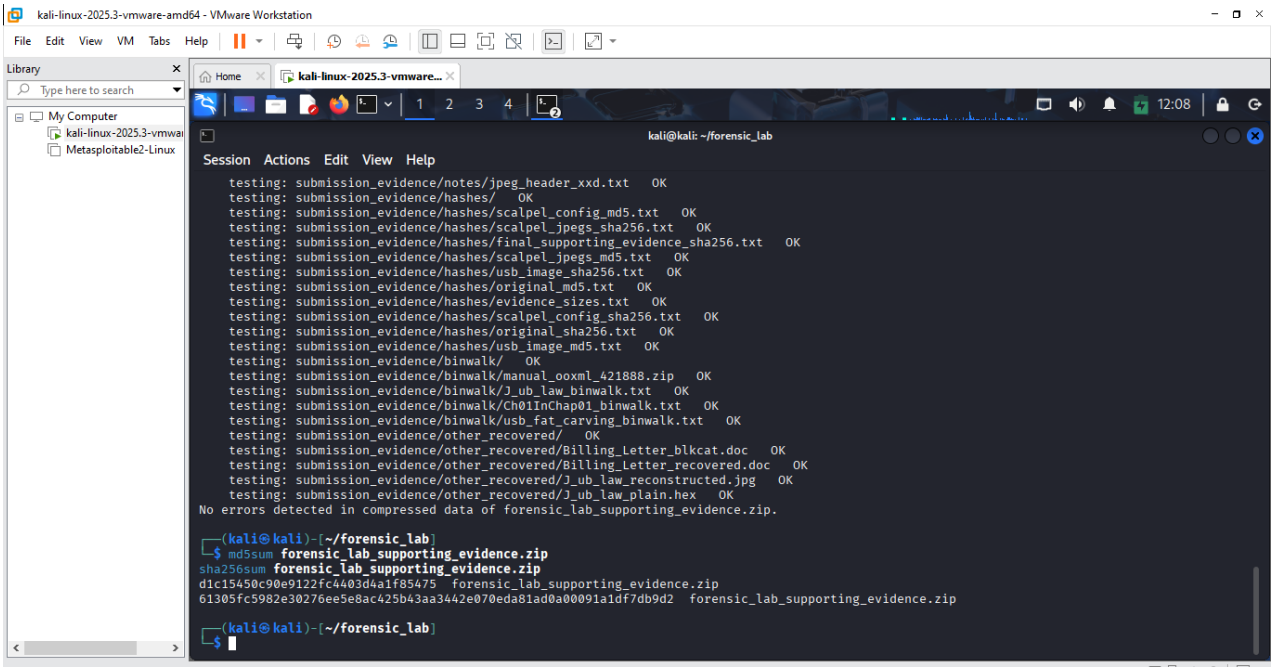

Fig 8.6 Creating the required evidence ZIP page 3.

Explanation

The ZIP verification completed without detected compressed-data errors, after which MD5 and SHA-256 hashes were calculated for the final archive. This provided the last integrity record for the supporting-evidence package before submission.

Conclusion

The laboratory successfully demonstrated how file recovery can continue even when normal file-system information is unavailable or unreliable. By combining hexadecimal examination, file-system analysis, embedded-file discovery, direct data-unit extraction, and signature-based carving, useful evidence was identified and recovered from the supplied forensic images. Integrity checks were incorporated throughout the process using MD5 and SHA-256 hashing.

The investigation also showed the value of working from the evidence actually available rather than relying on assumed offsets or placeholder values. The FAT16 partition offset was confirmed as sector 128 through mmls, while metadata addresses and sectors used with The Sleuth Kit came directly from the examined images. The absence of File_carving.docx was documented as a limitation, and the permitted alternative Binwalk workflow was completed using the USB forensic image instead.

Overall, the practical produced recoverable files, verified reconstructions, embedded-content extraction, 17 carved JPEG artefacts, documented hashes, an audit trail, and a validated supporting-evidence archive. This provided a complete and repeatable record of the forensic workflow while preserving the original evidence.

Screenshots

References

ICDFA Materials.